

# RoKoko<sup>\*</sup>: Lattice-based Succinct Arguments, a Committed Refinement

Michael Klooß<sup>1</sup>, Russell W. F. Lai<sup>2</sup>, Ngoc Khanh Nguyen<sup>3</sup>, Michał Osadnik<sup>2</sup>, and Lorenzo Tucci<sup>2</sup>

<sup>1</sup> KASTEL, Karlsruhe Institute of Technology, Karlsruhe, Germany

<sup>2</sup> Aalto University, Espoo, Finland

<sup>3</sup> King’s College London, London, UK

**Abstract.** We present RoKoko, a new lattice-based succinct argument system that achieves a linear-time prover alongside polylogarithmic communication and verifier complexity. Asymptotically, our construction improves upon RoK and Roll (ASIACRYPT 2025), the first post-quantum SNARK with  $\tilde{O}(\lambda)$  proof size, by a multiplicative factor of  $\Theta(\log \lambda)$ . Practically, our system yields proofs around 150 KB, while outperforming the state-of-the-art polynomial commitment scheme Greyhound (CRYPTO 2024) with  $\sim 100\times$  faster verification, similar prover time, and competitive proof size. Our framework natively supports (tensor-)structured relations, such as polynomial evaluation and sumcheck relations.

At a high level, our construction follows the recursive split-and-fold paradigm: the prover first *splits* the witness into  $\rho$  sub-witnesses, sends the corresponding cross-terms, and then *folds* them into a single witness that is shorter by a factor of  $\rho$  using verifier challenges. Prior works typically restrict  $\rho = O(1)$  to preserve succinct verification and maintain the optimal  $\tilde{O}(\lambda)$  proof size. We overcome this “constant barrier”, which enables larger  $\rho$  and thereby reduces the proof size. To achieve this, we introduce the following technical contributions.

- (i) *Committed folding.* Instead of sending  $O(\rho)$  cross-terms in the clear, the prover commits to the messages and later proves that the committed vector satisfies the verification relations. This enables a larger shrinking factor, reducing the number of recursion rounds. While this strategy has been successfully used in LaBRADOR (CRYPTO 2023), our main novelty is to do so while preserving succinct verification.
- (ii) *Recursive commitments.* We generalise the double-commitment technique from LaBRADOR into a framework for recursive commitments, yielding further compression in commitment size. This results in concrete improvements in communication within each recursion round.
- (iii) *Sumcheck-driven structured recursion.* We extend the sumcheck framework from SALSAA (ePrint 2025/2124) to prove substantially more complex constraints arising in our construction, including correctness of random projections, inner-product claims and well-formedness of recursive commitments, and open for future extensions. Expressing these constraints as sumcheck relations takes considerable effort, but the resulting protocols compose seamlessly with the structured recursion, yielding linear-time proving and succinct verification.

## 1 Introduction

Succinct argument systems [Kil92, Mic94] allow a prover to convince a verifier of a statement’s validity using communication that is significantly shorter than the witness itself. This powerful compression capability underlies some of the most impactful applications in modern cryptography, including verifiable outsourced computation [KP16, Lee21, GSW23], verifiable fully-homomorphic encryption [GBK<sup>+</sup>24, LLZ<sup>+</sup>25, ABPS24, ZLW<sup>+</sup>25], aggregate signatures [BK20, AAB<sup>+</sup>24], and verifiable delay functions [LM23, OKC<sup>+</sup>25].

As quantum computers threaten the security foundations of group-based cryptography, the demand for post-quantum secure succinct arguments has become increasingly urgent. Lattice-based constructions offer a compelling direction as they rely on well-studied hardness assumptions believed

---

<sup>\*</sup> also known as “late baRoK”.

to remain secure against quantum attacks, while their rich algebraic structure enables efficiency advantages that hash-based approaches cannot easily match.

### 1.1 Two Approaches in Lattice-Based Succinct Arguments

At the core of lattice-based arguments lies an efficient proof of knowledge of a low-norm witness vector  $\mathbf{w} \in \mathcal{R}^m$  satisfying

$$\mathbf{F}\mathbf{w} = \mathbf{y} \bmod q \quad \text{and} \quad \|\mathbf{w}\| \leq \beta, \quad (1)$$

where  $\mathcal{R} = \mathbb{Z}[\zeta]$  denotes the ring of integers of the  $f$ -th cyclotomic field  $\mathcal{K} := \mathbb{Q}(\zeta)$  of degree  $\varphi = \varphi(f)$ . While the linear relation is straightforward, the challenge is proving that  $\mathbf{w}$  is short: existing constructions either prove a relaxed variant of the equation, affecting parameter choices, or add machinery to enforce shortness.

A prominent instance of the former category is the lattice adaptation of the Bulletproofs protocol [BLNS20, AL21, ACK21, BCS21] which was the first lattice-based argument system with  $\text{poly}(\log m, \lambda)$  proof size. Let  $\rho \in \mathbb{N}$  be the shrinking factor. To prove (1), the prover and verifier partition  $\mathbf{F}$  as  $[\mathbf{F}_0 \mid \mathbf{F}_1 \mid \cdots \mid \mathbf{F}_{\rho-1}]$ , while the prover decomposes the witness  $\mathbf{w} := (\mathbf{w}_i)_{i \in [\rho]}$  into  $\rho$  smaller vectors  $\mathbf{w}_0, \dots, \mathbf{w}_{\rho-1} \in \mathcal{R}^{m/\rho}$ . The prover then transmits all  $\rho^2$  cross-terms  $\mathbf{g}_{i,j} := \mathbf{F}_i \mathbf{w}_j \bmod q$  for all pairs  $(i, j) \in [\rho]^2$ , to the verifier, who immediately checks that

$$\sum_{i=0}^{\rho-1} \mathbf{g}_{i,i} = \sum_{i=0}^{\rho-1} \mathbf{F}_i \mathbf{w}_i = \mathbf{F}\mathbf{w} = \mathbf{y} \pmod{q}. \quad (2)$$

The verifier then samples challenges  $c_0, \dots, c_{\rho-1} \leftarrow \mathcal{C}$  from a challenge space  $\mathcal{C} \subseteq \mathcal{R}$  consisting of short ring elements. The key observation is that the folded witness  $\mathbf{z} := \sum_{i=0}^{\rho-1} c_i \mathbf{w}_i \in \mathcal{R}^{m/\rho}$  satisfies

$$\left( \sum_{i=0}^{\rho-1} c_i \mathbf{F}_i \right) \mathbf{z} = \sum_{i=0}^{\rho-1} \sum_{j=0}^{\rho-1} c_i c_j \mathbf{g}_{i,j} \bmod q \quad \text{and} \quad \|\mathbf{z}\| \leq \beta_z \quad (3)$$

for an appropriately chosen bound  $\beta_z$ . As this statement retains the structure of (1), the prover can recursively repeat the procedure  $O(\log m)$  times until the final folded witness is small enough to reveal directly.

As analysed in [AL21, ACK21, BCS21], the lattice Bulletproofs protocol suffers from several efficiency drawbacks. Most notably, knowledge soundness critically depends on *subtractive sets*  $\mathcal{C}$ , in which the difference of any two distinct  $c, c' \in \mathcal{C}$  must be invertible in  $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$  with a short inverse. This limits  $\mathcal{C}$  to polynomial size [AL21], implying an inverse-polynomial soundness error; amplifying it to  $2^{-\lambda}$  then increases communication by a factor  $\Omega(\lambda/\log \lambda)$ . As a consequence, lattice Bulletproofs have large communication, both in theory and in practice.

Subsequent work in this area has diverged into two main directions. One path focuses on constructing succinct arguments with concretely efficient proof sizes, while the other aims at asymptotically efficient arguments with polylogarithmic verification. We elaborate on both directions below.

*Path 1: Concrete efficiency and proof size.* The lattice Bulletproofs protocol was substantially advanced by LaBRADOR [BS23], which introduces several key modifications. The first is that it departs from subtractive sets and instead adopts the random-projection technique of [BL17, LNS21,

[GHL22, LNP22, BS23]. To demonstrate that  $\mathbf{w}$  is (approximately) short, the verifier samples a random *integer* matrix  $\mathbf{J} \in \mathbb{Z}^{r \times m\varphi}$  with small entries (e.g. from  $\{-1, 0, 1\}$ ). The prover computes the projected vector  $\mathbf{v} := \mathbf{J}\mathbf{cf}(\mathbf{w}) \bmod q \in \mathbb{Z}_q^r$ , where  $\mathbf{cf}(\mathbf{w}) \in \mathbb{Z}^{m\varphi}$  is the standard coefficient embedding of the ring vector  $\mathbf{w}$ . The verifier checks that  $\mathbf{v}$  has small norm, while the prover additionally proves  $\mathbf{v}$  well-formed; this is a linear relation over  $\mathcal{R}_q$  [LNP22]. Crucially, this removes the dependence on subtractive sets, so the protocol needs no repetitions.

The second change is that the prover no longer transmits all  $\rho^2$  cross-terms  $\mathbf{g}_{i,j}$ . Instead, the prover commits to the concatenated vector  $\mathbf{g} := (\mathbf{g}_{i,j})_{i \in [\rho], j \in [\rho]}$ , and later proves that the committed value satisfies the linear constraints required for verification, namely those in Equations (2) and (3). This *committed folding* approach allows LaBRADOR to select large shrinking factors  $\rho_i = O(m_i^{1/3})$  at each round (where  $m_i$  is the witness size entering round  $i$ ), yielding only  $O(\log \log m)$  recursive rounds.

Finally, in LaBRADOR the witness  $\mathbf{w}$  is committed in a double-commitment fashion<sup>4</sup>. Specifically, the commitment  $\mathbf{t}_{\text{out}}$  to a short vector  $\mathbf{w}$  is defined by

$$\mathbf{t}_{\text{in}} := (\mathbf{F}_{\text{in}} \otimes \mathbf{I}_\rho) \mathbf{w} \bmod q, \quad \text{and} \quad \mathbf{t}_{\text{out}} := \mathbf{F}_{\text{out}}(\mathbf{G}^{-1}(\mathbf{t}_{\text{in}})) \bmod q,$$

where  $\mathbf{G}^{-1}(\cdot)$  denotes the standard gadget decomposition map.

This two-layer design enables a reduction in the final commitment size: the (Module-)SIS rank of the outer commitment matrix  $\mathbf{F}_{\text{out}}$  becomes essentially independent of the witness length  $m$ , unlike in classical Ajtai commitments [Ajt96].

As a consequence of the aforementioned improvements, LaBRADOR achieves concretely efficient proof sizes, around 60 KB even for large statements (with witness consisting of  $\approx 2^{20}$  ring elements). Its main drawback is the linear verification time. Indeed, even processing the random projection matrix  $\mathbf{J}$  already takes time  $\Omega(m)$ . The subsequent work Greyhound [NS24] addresses this limitation by invoking LaBRADOR as a subroutine, where each invocation is on a witness of length  $O(\sqrt{m})$ . As a result, Greyhound inherits the appealing proof sizes of LaBRADOR while achieving sublinear verification time.

*Path 2: Succinct verification.* Motivated by the growing interest in constructing SNARKs from Polynomial Interactive Oracle Proofs, there has been much research on lattice-based polynomial commitment schemes (PCS). In this direction, [CLM23] first adapted lattice Bulletproofs to support succinct verification by endowing the commitment matrix  $\mathbf{F}$  with a tensor structure. This modification leads to a commitment that is binding under a new assumption, called the Vanishing SIS (vSIS) assumption. Independently, [BCS23] achieved fast verification for lattice Bulletproofs via additional preprocessing. However, both constructions inherit all the underlying limitations of lattice Bulletproofs itself [BCS23, CLM23].

A different line of work [AFLN24, CMNW24, KLN024b] develops succinct arguments with polylogarithmic verification time by applying FRI-style [BBHR18] split-and-fold techniques. These protocols still require repetitions due to the use of subtractive sets, but they yield more efficient proofs than Bulletproofs, since they involve only  $O(\rho)$  cross-terms, compared to  $O(\rho^2)$  required in lattice Bulletproofs.

The reliance on subtractive sets was later removed by [KLN025], which adapts the random projection strategy to demonstrate shortness. Directly using LaBRADOR-style random projections is infeasible here due to the goal of achieving fast verification. To address this, the authors employ

<sup>4</sup> In [BS23], the authors refer to this as “inner” and “outer” commitments.

tensor-structured projection matrices of the form  $\mathbf{J} := \mathbf{I} \otimes \mathbf{J}'$ , where  $\mathbf{I}$  is the identity matrix and  $\mathbf{J}'$  is a short random challenge matrix with  $O(\lambda)$  rows and columns. Consequently, the verifier only needs to operate on  $\mathbf{J}'$ , and with a carefully designed split-and-fold protocol, succinct verification becomes possible. As a result, [KLNO25] achieves asymptotically optimal  $\tilde{O}(\lambda)$ -sized proofs, notable since all prior post-quantum argument systems, including hash-based ones [ACFY24, ACFY25], only obtained  $\tilde{O}(\lambda^2)$  proofs. Despite this asymptotic, concrete efficiency remains limited as proof sizes scale to a few megabytes (with witness of  $\approx 2^{23}$  ring elements).

While random projections can demonstrate that the committed witness is approximately short, they cannot on their own certify exact norm bounds. To address this, [KLNO24b, KLNO25] showed how to prove inner products between committed vectors modulo  $q$ . Provided that  $\mathbf{w}$  is sufficiently short, no modular wrap-around occurs, enabling the prover to establish inner products over  $\mathbb{Z}$ . Building on this, SALSAA [KLOT25] adds sumcheck protocols [LFKN92] to verify exact  $\ell_2$  norm and, unlike [KLNO24b, KLNO25], enjoys a linear prover.

*Bridging the gap.* LaBRADOR [BS23] achieves remarkably small proof sizes, owing to its use of super-constant folding, double-layer commitments, and random projections to establish shortness. However, its principal limitation is the linear verification time, and even with the improvement offered by Greyhound [NS24], the verifier remains more than two orders of magnitude slower than hash-based alternatives. Conversely, argument systems that support succinct verification are still far from concretely competitive. Although recent work incorporates structured Johnson–Lindenstrauss projections into this setting, it remains unclear whether the other techniques underpinning LaBRADOR adapt to the succinct-verifier regime while preserving practical efficiency. This raises a question: can one design an argument system that admits polylogarithmic verification while achieving proof sizes and concrete performance similar to LaBRADOR and Greyhound?

## 1.2 Our contributions

We present RoKoko – a lattice-based argument system that unifies the strengths of both paths, achieving nearly the best of both worlds:

- (i) **Succinct verification:** polylogarithmic verifier runtime, matching the best Path 2 constructions;
- (ii) **Optimal prover:** linear-time proving, matching the lower bound;
- (iii) **Compact proofs:** proof sizes substantially improving on prior succinct-verification work, narrowing the gap to the LaBRADOR line;
- (iv) **Rich expressiveness:** native support for structured relations, polynomial commitments, and natural arithmetisations such as R1CS.

*Technical innovations.* Our main technical contribution is to adapt committed superconstant folding – a technique pioneered by LaBRADOR [BS23] – to the vSIS-based framework that supports succinct verification. In the LaBRADOR line, the verifier performs work proportional to the witness size, so the folding structure can remain opaque to the verifier. In the succinct-verification regime, however, the verifier must efficiently check consistency of the folded instances without implicitly reading the full witness, which requires the folding to respect the algebraic structure of the underlying commitment scheme. Concretely:

- **Structured superconstant folding.** We show how to fold by a superconstant factor within the vSIS-based framework while preserving the structure needed for succinct verification. We

extend the random projection technique of [KLNO25] to handle this more aggressive folding, yielding a significant reduction in the number of rounds.

- **Recursive commitments.** Inspired by [BS23, BC25b], we introduce a recursive commitment structure where the outermost commitment – the only one the verifier inspects explicitly – has low rank independent of the witness size. This compresses the commitment component of the proof, which dominates proof size in prior constructions from Path 2 (the succinct-verification regime); such recursive commitments were not previously available in the structured setting.
- **Sumcheck-based linear-time proving.** Building on [KLOT25], we integrate sumcheck protocols to efficiently prove well-formedness of the recursive commitments and correctness of projections. The sumcheck machinery suits the recursive commitment structure well: the prover shows each commitment layer is correctly formed without materialising intermediate witnesses, achieving linear-time proving without sacrificing succinctness.
- **Implementation and evaluation.** We implement RoKoko and conduct experiments, demonstrating that our improvements translate into practice: competitive proof sizes alongside faster verification compared to prior lattice-based constructions. We also give a complete asymptotic analysis and instantiate RoKoko as a SNARK for structured relations and as a PCS.
- **Incomplete-NTT library.** As a side contribution, we provide the most efficient open-source standalone implementation, to date, of the “almost-splitting” cyclotomic ring arithmetic underpinning our construction.

We summarise the concrete performance of RoKoko compared to prior post-quantum SNARKs and PCSs in Table 1. Compared to SALSAA [KLOT25], the closest prior work with succinct verification, RoKoko achieves an order-of-magnitude improvement in proof size and prover runtime, and 30–40× faster verification. Compared to Greyhound [NS24], which optimises for proof size and concrete efficiency but lacks succinct verification, RoKoko offers up to 50–100× faster verification while remaining competitive in proof size and prover runtime. The trade-off between RoKoko and Greyhound is that the former achieves polylogarithmic verification time at the cost of a slight increase in proof size and reliance on the vSIS assumption, while the latter has smaller proofs and relies on the more standard Module-SIS assumption but without polylogarithmic verification. We defer a detailed comparison of RoKoko against prior work to Section 9.5.

### 1.3 Related work

**Group- and hash-based succinct arguments.** Group-based arguments such as Groth16 [Gro16], PLONK [GWC19] and others [PHGR13, GGPR13, WTs<sup>+</sup>18, MBKM19, XZZ<sup>+</sup>19, BFS20, CHM<sup>+</sup>20] achieve strong efficiency but rely on assumptions like discrete logarithm, and are thus insecure against quantum adversaries. Hash-based argument systems, such as FRI [BBHR18], Ligero [AHIV23], Aurora [BCR<sup>+</sup>19], Brakedown [GLS<sup>+</sup>23], and WHIR [ACFY25], offer post-quantum security and transparent setup. In particular, WHIR achieves extremely efficient verification time. However, hash-based schemes generally fall short in prover efficiency and proof size compared to lattice-based alternatives.

**Non-succinct lattice-based arguments.** Non-succinct lattice-based arguments [LNSW13, BLS19, ALS20, LNP22] have proof size and verification time linear in the witness size, offering high concrete efficiency for small relations. These protocols typically focus on the zero-knowledge property rather than succinctness.

**Table 1.** Concrete performance of our work vs. prior post-quantum SNARKs/PCSs ( $|\pi|$  = proof size in KB). Hash-based: Brakedown [GLS<sup>+</sup>23], Ligerio [AHIV17], FRI [BBHR18], WHIR [ACFY25]. Lattice-based: CMNW24 [CMNW24], HSS24 [HSS24], Papercraft [OKC<sup>+</sup>25], KLNO25 [KLNO25], Greyhound [NS24], SALSAA [KLOT25] ( $\varphi=256$ ). For non-lattice schemes and HSS24, times and sizes are from the original papers. For Greyhound and SALSAA, we ran their open-source code ourselves, and due to the scheme’s memory requirement exceeding 64 GB, the experiments marked with † could not be performed on the target machine (cf. Remark 6). A comparison of the runtimes on a different, server-grade machine is provided in Table 8. The committed witness bit-width is 32 bits for RoKoko, matching the bit-width reported in Greyhound [NS24]. For SALSAA, we reduced the number of threads to 1 to remove the parallelisation effect.

| Scheme     | $\#\mathbb{Z}_q = 2^{26}$ |         |        |         | $\#\mathbb{Z}_q = 2^{28}$ |         |        |         | $\#\mathbb{Z}_q = 2^{30}$ |       |        |         |
|------------|---------------------------|---------|--------|---------|---------------------------|---------|--------|---------|---------------------------|-------|--------|---------|
|            | Comm.                     | P       | V      | $ \pi $ | Comm.                     | P       | V      | $ \pi $ | Comm.                     | P     | V      | $ \pi $ |
| Brakedown  | 36s                       | 3.21s   | 0.703s | 49157   | 150s                      | 13s     | 2.56s  | 93767   | 605s                      | 48.6s | 2.96s  | 181948  |
| Ligerio    | 39.9s                     | 3.11s   | 0.196s | 7256    | 169s                      | 12.4s   | 0.402s | 14383   | 717s                      | 50s   | 0.846s | 28631   |
| FRI        | 168s                      | 185s    | 0.041s | 740     | -                         | -       | -      | -       | -                         | -     | -      | -       |
| WHIR       | 210s                      | -       | 5.4ms  | 689     | 860s                      | -       | 6.1ms  | 770     | 3700s                     | -     | 6.6ms  | 848     |
| CMNW24     | -                         | -       | -      | 1546    | -                         | -       | -      | -       | -                         | -     | -      | 5296    |
| HSS24      | 188s                      | -       | 1.07s  | 48640   | -                         | -       | -      | -       | -                         | -     | -      | -       |
| Papercraft | -                         | -       | -      | -       | 1593s                     | -       | 5.10s  | 18940   | 6986s                     | -     | 6.00s  | 29790   |
| KLNO25     | -                         | -       | -      | 2181    | -                         | -       | -      | 2604    | -                         | -     | -      | 3152    |
| Greyhound  | 4.80s                     | 1.41s   | 0.42s  | 46      | 18.62s                    | 5.41s   | 0.75s  | 53      | †                         | †     | †      | 53      |
| SALSAA     | 1.71s                     | 108.59s | 0.289s | 1005    | 6.76s                     | 439.73s | 0.359s | 1232    | †                         | †     | †      | 1459    |
| RoKoko     | 1.13s                     | 1.36s   | 7.50ms | 157     | 4.31s                     | 2.97s   | 7.97ms | 157     | 18.1s                     | 8.92s | 13.1ms | 157     |

**Folding schemes.** Orthogonal to “monolithic” SNARKs discussed above (following the terminology of [BCFW25]), a line of works [BC25a, FKNP24, BMNW25a, BMNW25b, NS25, BC25b, BCFW25] have proposed post-quantum folding schemes (or accumulation schemes) that can be used as building blocks of incrementally-verifiable computation protocols [Val08] and proof-carrying data [CT10]. Folding schemes also compile generically into SNARKs [KST21].

Among recent quantum-safe folding schemes, several rely on lattice assumptions [BC25a, FKNP24, NS25, BC25b]. As in our setting, the prover must prove (exact) shortness of the witness. To achieve this, [BC25a, NS25, BC25b] apply the sumcheck protocol to prove the  $\ell_\infty$  norm bound. However, as the communication complexity scales linearly with the range, this approach becomes impractical for large  $\ell_\infty$  bounds and is not well-suited to prove  $\ell_2$  norms.

## 2 Technical Overview

We give a high-level overview of our lattice-based succinct argument construction, its applications and its implementation.

### 2.1 Background

Let  $\mathcal{R} \cong \mathbb{Z}[x]/\langle \Phi_f(x) \rangle$  be a cyclotomic ring and  $\mathcal{R}_q = \mathcal{R}/q\mathcal{R}$  for a prime  $q$ . Works on lattice-based succinct arguments typically consider a sufficiently expressive relation defined over  $\mathcal{R}_q$  where the witness is a matrix  $\mathbf{W} \in \mathcal{R}^{m_w \times r}$  of norm at most some  $\beta_w$ . The goal is always to progressively shrink the witness dimension  $m_w$  until it is small enough to be sent in the clear. Two intertwined challenges must be addressed:

1. *Witness folding.* The basic idea of shrinking the witness  $\mathbf{W}$  is to fold  $\mathbf{W}$  into  $\mathbf{Wc}$  by some verifier challenge  $\mathbf{c}$ , and then repack the witness into a smaller matrix  $\mathbf{W}'$ .<sup>5</sup> The *shrink factor*  $\rho$ , i.e. the

<sup>5</sup> Prior works typically describe this in the other way around – first repack the witness to create many columns then fold to fewer columns. The fold-then-split perspective is more convenient for this work.

rate at which the witness dimension is reduced, is roughly the dimension of  $\mathbf{c}$ . What distinguishes the semi-<sup>6</sup> and fully-succinct settings is that, in the latter, the verifier must be able to check (among other constraints due to norm control) the correctness of folding in polylogarithmic time.

2. *Norm control*. While shrinking the witness dimension, we must keep the witness norm at around the same level, say  $\beta_{\mathbf{w}}$ . This splits into two sub-challenges. *Completeness*, i.e. the honest prover can produce a valid witness for the reduced instance. Folding  $\mathbf{W}$  into  $\mathbf{W}\mathbf{c}$  may increase the norm; the standard fix is to decompose the folded witness  $\mathbf{W}\mathbf{c}$ , giving a net dimension reduction when the shrinking by  $\mathbf{c}$  outweighs the decomposition expansion. We take this for granted and omit it henceforth. *Extraction*, i.e. the extractor can recover a valid witness for the original instance. As usual for lattice-based arguments, it recovers not an exact witness but a “relaxed” one  $(\mathbf{W}, \mathbf{s})$  with a short “slack” vector  $\mathbf{s}$ , relaxed in two ways: it only guarantees  $\|\mathbf{W} \cdot \text{diag}(\mathbf{s})\| \leq \beta'_{\mathbf{w}}$  (not necessarily  $\|\mathbf{W}\| \leq \beta'_{\mathbf{w}}$ ), and the bound may be looser than the original ( $\beta'_{\mathbf{w}} > \beta_{\mathbf{w}}$ ). This too can accumulate throughout composition.

Numerous prior works address subsets of these challenges with different trade-offs; we highlight the most relevant.

Regarding *witness folding*, [CLM23] proposes using vSIS commitments so that the witness can be folded in a structured way that enables a polylogarithmic verifier. This method is refined in [KLNO24b] and adopted in [KLNO25, KLOT25]. In [KLNO24b, KLNO25, KLOT25], folding the witness by a factor  $\rho$  requires communication proportional to  $\rho$  per round, forcing them to settle on  $\rho = O(1)$ . LaBRADOR [BS23] achieves a shrink factor of  $\rho = O(m_{\mathbf{w}}^{1/3})$  with essentially constant communication per round by having the prover *commit* to the cross-terms instead of sending them in the clear. However, this method of folding breaks whatever structure the original relation might have, causing the verifier to run in time proportional to  $m_{\mathbf{w}}$ .

Regarding *norm control*, LaBRADOR [BS23] uses random projections to eliminate the slack vector  $\mathbf{s}$ . However, the projection matrices are unstructured, which again causes the verifier to run in time proportional to  $m_{\mathbf{w}}$ . RoK and Roll [KLNO25] adapted this technique to the fully-succinct setting by introducing structured projections. Then, to force the extracted witness norm  $\beta'_{\mathbf{w}}$  to be the same as the original norm  $\beta_{\mathbf{w}}$ , SALSAA [KLOT25] adapted the sumcheck protocol to prove the exact norm of the witness with a prover that runs in strict- (as opposed to quasi-)linear time, improving upon the norm-check RoK of [KLNO24b].

The main contribution of this work is to extend the committed folding technique (explored in the literature in [BS23] or [BC25b] in the context of folding schemes) to the fully-succinct setting while retaining all techniques introduced in [KLNO24b, KLNO25, KLOT25] for structure-preserving norm control. This will ultimately allow us to achieve a more favourable trade-off than the state-of-the-art. Our construction of succinct arguments is built by composing a small number of reductions of knowledge (RoKs) which map between two relations – a committed-linear relation  $\Xi_{\text{COM}}^{\text{lin}}$  and a functional-sumcheck relation  $\Xi_{\text{COM}}^{\text{sum}}$ , both defined over  $\mathcal{R}_q$ . While either relation can be taken as the “starting” relation, in this exposition we start from the committed-linear relation  $\Xi_{\text{COM}}^{\text{lin}}$  which is easier to explain. In other words, we will build a succinct argument system for  $\Xi_{\text{COM}}^{\text{lin}}$ .

## 2.2 Core Design

At a high level, in  $\Xi_{\text{COM}}^{\text{lin}}$  the prover’s witness  $\text{wit}$  mainly consists of a matrix  $\mathbf{W} \in \mathcal{R}^{m_{\mathbf{w}} \times r}$  satisfying linear constraints  $\mathbf{F}_i \mathbf{W} = \mathbf{H}_i \mathbf{Y}_i \bmod q$  for all  $i$ , where we can think of  $\mathbf{Y}_1$  as a commitment of  $\mathbf{W}$ .

<sup>6</sup> In the semi-succinct setting, only the proof size but not the verifier time must be polylogarithmic in the witness size.

The matrices  $\mathbf{Y}_i$  on the right-hand side are auxiliary witness components which are themselves (recursively) committed. The main witness component  $\mathbf{W}$  additionally satisfies a linear constraint of the form  $\mathbf{I}^T \mathbf{W} \mathbf{r} = t \bmod q$ . Intuitively, think of  $\mathbf{W}$  as a very tall matrix, e.g. of height  $m_w = 2^{20}$ , while  $\mathbf{Y}_i$  is relatively much smaller. Furthermore, the witness components  $\mathbf{W}, \mathbf{Y}_i$  must be *low-norm*.

Our goal is to iteratively reduce a large  $\Xi_{\text{COM}}^{\text{lin}}$  instance into progressively smaller ones while maintaining *full-succinctness* – the verifier’s runtime (and hence the proof size) must be at most  $\text{poly}(\lambda, \log |\text{wit}|) = \text{poly}(\lambda, \log m_w r)$ . In a nutshell, each round of our protocol consists of three (families of) RoKs:  $\Pi^{\text{proj}}$  for random projection,  $\Pi^{\text{fold-split}}$  for folding, splitting and encoding constraints into functional-sumcheck claims, and  $\Pi^{\text{lin}}$  for turning sumcheck claims back into linear constraints. Symbolically, each round is a sequential composition of

$$\Pi^{\text{fold-split}} \circ \Pi^{\text{proj}} : \Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{sum}} \quad \text{and} \quad \Pi^{\text{lin}} : \Xi_{\text{COM}}^{\text{sum}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}.$$

After each round, the witness dimension  $m_w$  is reduced by a factor of  $\rho$  while other parameters remain essentially unchanged. We explain these RoKs in detail.

*Committed random projection*  $\Pi^{\text{proj}} : \Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$ . This RoK addresses *norm control*: inspired by [KLNO25], it is a self-reduction on  $\Xi_{\text{COM}}^{\text{lin}}$  that eliminates the extraction-side norm slack. The key idea relies on a variant of the Johnson–Lindenstrauss (JL) lemma: a random matrix  $\mathbf{J} \in \mathbb{Z}^{n_{\text{rp}} \times m_{\text{rp}}}$  with small i.i.d. entries maps  $\mathbf{x} \in \mathbb{Z}^{m_{\text{rp}}}$  so that the ratio  $\|\mathbf{J}\mathbf{x}\|/\|\mathbf{x}\|$  lies in a bounded interval  $[\alpha_{\text{rp}}, \beta_{\text{rp}}]$  with high probability – the norm is scaled up but tightly controlled. Naively, one would apply such a  $\mathbf{J}$  to the entire witness of dimension  $m_w$ , but the verifier would then need to read all  $n_{\text{rp}} \times m_w$  entries of  $\mathbf{J}$ , breaking succinctness. Instead, the prover applies a *structured* variant  $(\mathbf{I} \otimes \mathbf{J})$  where  $\mathbf{J} \in \mathbb{Z}^{n_{\text{rp}} \times m_{\text{rp}}}$  with  $m_{\text{rp}} \ll m_w$ : the verifier only samples the small matrix  $\mathbf{J}$ , and the block-diagonal structure preserves the JL guarantee on each block. The JL guarantee ensures that a short projection image implies the original witness is short, i.e. without slack.

We instantiate this idea in three variants. The *coarse* variant ( $\Pi^{\text{proj-c}}$ ) projects ring elements directly and is used while  $m_w \geq m_{\text{rp}}$ . The *fine* variant ( $\Pi^{\text{proj-f}}$ ) projects their coefficients via the trace-dual embedding (used once  $m_w$  drops below  $m_{\text{rp}}$ , relaxing the dimension requirement to  $m_{\text{rp}} \mid \varphi m_w$ ), and then batches the projection and lifts to constraints over  $\mathcal{R}_q$ . The *lift-and-batch* variant ( $\Pi^{\text{proj-fl}}$ ) generalises the fine variant by exploiting the chain of intermediate subfields  $\mathbb{F}_{q^a}$  of  $\mathcal{R}_q$  to gradually lift the projection image instead of immediately lifting to  $\mathcal{R}_q$ , yielding further asymptotic improvements. All variants share the same  $(\mathbf{I} \otimes \mathbf{J})$  structure, and – crucially – the prover *commits* to the (gadget-decomposed) projection image rather than sending it in the clear.

Whereas *coarse* projections are similar to the random “structured” projection protocol in [KLNO25] aside from operating on different input and output relations, the *fine* and *lift-and-batch* variants introduce additional structure not present in the corresponding variant from [KLNO25]. This structure serves to keep the verifier efficient as the projection matrix grows wider – a necessity when matching the compression effect of larger shrinking factors  $\rho$ . In [KLNO25], the *fine* and *lift-and-batch* projection image (presented jointly as “unstructured”) has linear communication in both the projection height and the witness width, and is sent in the clear; in our setting, widening the witness and increasing the projection height to accommodate a large  $\rho$  would make this image prohibitively large for uncompressed transmission. Committing to the image reduces that overhead.

*Committed folding and splitting*  $\Pi^{\text{fold-split}} : \Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{sum}}$ . This RoK primarily addresses *witness folding* but also encodes all resulting constraints into sumcheck claims. The protocol starts from a committed-linear instance of width  $r$ , samples a low-norm challenge  $\mathbf{c} \in \mathcal{C}^r$ , folds the witness to  $\mathbf{W}\mathbf{c}$ ,

and packs all derived data into  $\widehat{\mathbf{w}}$ . It then computes the norm witness  $v = \langle \widehat{\mathbf{w}}, \widehat{\mathbf{w}} \rangle$  and reshapes  $\widehat{\mathbf{w}}$  to a new witness matrix  $\mathbf{U}$  of width  $r'$ . Then, it derives a commitment  $\mathbf{F}\mathbf{U} = \mathbf{G}_{\ell'} \mathbf{Y} \bmod q$  (with  $\mathbf{Y}$  being further recursively committed). Rather than returning a new  $\Xi_{\text{COM}}^{\text{lin}}$  with additional constraints, the second part of this RoK encodes all constraints – folded linear constraints, commitment well-formedness, right/global linear constraints and the exact norm check via  $v$  – into sumcheck claims while preserving succinct verification (in a similar manner to [KLOT25]). Due to the specific structure of the linear constraints arising in our protocol – they are either Kronecker row-tensors (originating from vSIS commitment keys) or sparse matrices (from auxiliary constraints) – the resulting multilinear extensions are efficiently evaluable, enabling a succinct sumcheck verifier.

*Linearisation with sumcheck*  $\Pi^{\text{lin}}: \Xi_{\text{COM}}^{\text{sum}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$ . This RoK completes each round by re-encoding the sumcheck claims as linear constraints, enabling the next iteration. It executes the sumcheck protocol as a subroutine to reduce all accumulated sumcheck claims into evaluation claims on the committed witness. In our construction, it retains the constraint matrices  $(\text{com}, \mathbf{F}, \mathbf{H})$  from the original instance and linearises the sumcheck claims into structured linear constraints, yielding a fresh  $\Xi_{\text{COM}}^{\text{lin}}$  instance with  $m_{\mathbf{w}} \mapsto m_{\mathbf{w}}/\rho^7$ . Our key optimisation is *subfield batching*: while combining multiple sumcheck claims via a random linear combination is standard, we additionally apply a random  $\mathbb{F}_{q^a}$ -linear map  $\Phi: \mathcal{R}_q \rightarrow \mathbb{F}_{q^a}$  (for a subfield  $\mathbb{F}_{q^a} \subseteq \mathcal{R}_q$ ) that collapses the  $\varphi/a$  components of the  $\mathbb{F}_{q^a}$ -decomposition of  $\mathcal{R}_q$  into one, so that each sumcheck round communicates only a single  $\mathbb{F}_{q^a}$  element. This keeps verifier work and communication low. This can be viewed as a generalisation and formalisation of batching over NTT slots introduced in [BC25a, KLOT25, GLLO26] and orthogonal sumcheck-over-coefficients techniques from [NOZ26, HMZ25].

### 2.3 Putting Everything Together

*Composition and asymptotics.* The full composition proceeds in two phases: first, a *coarse* chain  $(\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-c}})$  is iterated while  $m_{\mathbf{w}} \geq m_{\text{rp}}$ ; once  $m_{\mathbf{w}}$  drops below  $m_{\text{rp}}$ , the *fine* chain  $(\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-f}})$  takes over for the remaining rounds<sup>8</sup>. With  $\rho = \lambda$ , the full composition yields a prover runtime of  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations (linear-time), verifier runtime of  $O(\lambda^3 \log m_{\mathbf{w}} / \log \lambda)$ , and proof size of  $O(\lambda \log^3 m_{\mathbf{w}} / \log^2 \lambda)$  bits. Compared to SALSAA [KLOT25], which is effectively limited to  $\rho = O(1)$  because their verifier complexity grows linearly in  $\rho$ , we reduce proof size by a factor of  $\log \lambda$  while matching prover complexity, at the cost of a theoretically larger verifier runtime. Practically, our verifier optimisations significantly reduce the overhead of processing random projection constraints, improving upon the verifier time of SALSAA as shown in our experiments.

*Applications.* A succinct argument for  $\Xi_{\text{COM}}^{\text{lin}}$  immediately yields a polynomial commitment scheme (PCS): the polynomial evaluation claim  $p(\mathbf{v}) = t$  is a linear constraint  $\mathbf{I}^T \mathbf{W} \mathbf{r} = t \bmod q$  on the committed coefficient matrix  $\mathbf{W}$ , which is already a  $\Xi_{\text{COM}}^{\text{lin}}$  instance (as noted above). If we instead take the more expressive  $\Xi_{\text{COM}}^{\text{sum}}$  as the starting relation, we obtain a SNARK for R1CS: the R1CS constraints are naturally expressible as sumcheck constraints captured by  $\Xi_{\text{COM}}^{\text{sum}}$ , though the verifier must additionally evaluate multilinear extensions on public R1CS matrices.

<sup>7</sup> The “fresh”  $\Xi_{\text{COM}}^{\text{lin}}$  instance does not shrink  $m_{\mathbf{w}}$  by the full folding width  $r$ , because the witness accumulates additional data over the course of the composition – projection images, auxiliary recursive-commitment information, gadget-decomposition overhead, etc. The parameter  $\rho$  therefore captures the *effective* compression factor per complete round, which is smaller than the number of columns  $r$  folded in  $\Pi^{\text{fold-split}}$ .

<sup>8</sup> In asymptotic analysis in Section 8, we use *lift-and-batch* projections for the second chain, which allows us to achieve a better asymptotic proof size. This variant is more complex and may not yield practical improvements for the parameters we consider.

*Implementation.* We implement RoKoko in Rust, targeting almost-splitting power-of-two cyclotomic rings ( $\mathcal{R}_q \cong (\mathbb{F}_{q^2})^{\varphi/2}$ , with  $\varphi = 128$ ,  $q \approx 2^{50}$ ). Almost-splitting rings (residue degree  $e = 2$ ) provide NTT slots  $\mathbb{F}_{q^2}$  large enough for sufficient per-round soundness while admitting efficient *incomplete-NTT* arithmetic, with ring multiplication reducing to  $\varphi/2$  independent quadratic products. We release the ring arithmetic layer as an independent open-source library. On top of it, we build a sumcheck module with a generic interface for expressing sumcheck relations, and instantiate it to deliver a PCS. For a PCS over  $2^{26}$  coefficients in  $\mathbb{Z}_q$  (single-threaded,  $\lambda = 128, \kappa = 2^{-100}$ ), our implementation produces a 157 KB proof in 2.5 s (commit + prove), with verification in 7.5 ms. Compared to SALSAA [KLOT25], this is a  $\sim 6\times$  reduction in proof size,  $\sim 45\times$  faster proving, and  $\sim 40\times$  faster verification; compared to Greyhound [NS24], we offer  $\sim 56\times$  faster verification (scaling up to  $\sim 100\times$  at larger instances, as shown in Table 8).

## 2.4 Paper Organisation

We define the relations  $\Xi_{\text{COM}}^{\text{lin}}$  and  $\Xi_{\text{COM}}^{\text{sum}}$  in Section 4. The committed random projection protocols  $\Pi^{\text{proj-c}}$  and  $\Pi^{\text{proj-f}}$  are presented in Section 5, committed folding  $\Pi^{\text{fold-split}}$  in Section 6, and the linearisation-with-sumcheck protocol  $\Pi^{\text{lin}}$  in Section 7. The composition of these RoKs and the asymptotic analysis are given in Section 8, and implementation details with experimental results in Section 9.

## 3 Preliminaries

For  $n \in \mathbb{N}$ , we write  $[n] := \{0, 1, \dots, n-1\}$ , i.e. we count from 0. For vectors, we typically use lowercase bold letters, e.g.  $\mathbf{v}$ , and for matrices we use uppercase bold letters, e.g.  $\mathbf{M}$ . For a matrix  $\mathbf{M}$ , we write  $\text{vec}(\mathbf{M})$  for the (column-style) vectorisation of  $\mathbf{M}$ . Conversely, we write  $\mathbf{V} = \text{reshape}_k(\mathbf{v})$  to reshape a vector into a  $k$ -column matrix, such that  $\text{vec}(\mathbf{V}) = \mathbf{v}$ , assuming compatible dimensions. Recall that for matrices  $\mathbf{A}, \mathbf{B}, \mathbf{C}$  with compatible dimensions the mixed-product property  $\text{vec}(\mathbf{ABC}) = (\mathbf{C}^T \otimes \mathbf{A}) \cdot \text{vec}(\mathbf{B})$  holds, where  $\otimes$  denotes the Kronecker product. We use  $\mathbf{A} \parallel \mathbf{B}$  to denote the horizontal concatenation of matrices  $\mathbf{A}$  and  $\mathbf{B}$  with the same number of rows. We use  $(\mathbf{A}, \mathbf{B})$  to denote the vertical concatenation of matrices  $\mathbf{A}$  and  $\mathbf{B}$  with the same number of columns. We use  $\mathbf{A} \bullet \mathbf{B}$  to denote the row-wise tensor product of matrices  $\mathbf{A}$  and  $\mathbf{B}$  with the same number of rows. To highlight the row-tensor structure of a matrix  $\mathbf{A} = \mathbf{A}_0 \bullet \dots \bullet \mathbf{A}_{k-1}$  defined over ring  $\mathcal{R}$  (resp.,  $\mathcal{R}_q$ ), we write  $\mathbf{A} \in \mathcal{R}^{n \times 2^{\otimes k}}$  (resp.,  $\mathcal{R}_q^{n \times 2^{\otimes k}}$ ). We write  $\text{minpowtwo}(x)$  to denote the smallest power of two greater than or equal to  $x$ .

### 3.1 Algebraic Number Theory

Throughout, we let  $\mathcal{K} = \mathbb{Q}(\zeta)$  be a cyclotomic field with conductor  $\mathfrak{f}$  and degree  $\varphi = \varphi(\mathfrak{f})$  and  $\mathcal{R} = \mathcal{O}_{\mathcal{K}}$  its ring of integers. We write  $\text{trace} = \text{trace}_{\mathcal{K}/\mathbb{Q}}$  for the field trace and, by abuse of notation, the natural adaptation of the map from  $\mathcal{R}_q$  to  $\mathbb{Z}_q$ . If  $\mathbf{M}$  is a matrix then  $\text{trace}(\mathbf{M})$  denotes taking trace entry-wise. For a prime modulus  $q \in \mathbb{N}$ , we write  $\mathcal{R}_q := \mathcal{R}/q\mathcal{R}$ . We denote by  $\mathcal{R}^\times$  and  $\mathcal{R}_q^\times$  the sets of units in  $\mathcal{R}$  and  $\mathcal{R}_q$  respectively.

*Embeddings.* We endow  $\mathcal{R}$  with two geometries via the coefficient embedding  $\text{cf}_{\mathbf{b}} : \mathcal{R} \rightarrow \mathbb{Z}^\varphi$  (for a given basis  $\mathbf{b}$ ) and the canonical embedding  $\sigma : \mathcal{K} \rightarrow \mathbb{C}^\varphi$  (of  $\mathcal{K}$ ). Specifically, for a given  $\mathbb{Z}$ -basis  $\mathbf{b} = (b_i)_{i \in [\varphi]}$  of  $\mathcal{R}$  and an element  $x = \sum_{i \in [\varphi]} x_i b_i \in \mathcal{R}$ , we write  $\text{cf}_{\mathbf{b}}(x) := (x_i)_{i \in [\varphi]}$  and

$\sigma(x) := (\sigma_j(x))_{j \in [\varphi]}$ , where  $\sigma_j \in \text{Gal}(\mathcal{K}/\mathbb{Q})$ . We define the powerful basis of  $\mathcal{R}$  as  $\mathbf{b} = (1, \zeta, \dots, \zeta^{\varphi-1})$  for prime-power conductor  $\mathfrak{f}$ . The basis generalises to the composite conductor  $\mathfrak{f} = \prod_{i \in [k]} \mathfrak{f}_i^{e_i}$  for prime  $\mathfrak{f}_i$  via tensor product,  $\mathbf{b} = \bigotimes_{i \in [k]} \left(1, \zeta_{\mathfrak{f}_i}, \dots, \zeta_{\mathfrak{f}_i}^{\varphi(\mathfrak{f}_i)-1}\right)$ . If  $\mathbf{b}$  is the standard powerful basis, we may omit  $\mathbf{b}$  from the subscript of  $\text{cf}_{\mathbf{b}}$ . We extend the notation of  $\text{cf}_{\mathbf{b}}$  and  $\sigma$  naturally to vectors, i.e. if  $\mathbf{x} = (x_i)_{i \in [m]}^T \in \mathcal{R}^m$ , then  $\text{cf}_{\mathbf{b}}(\mathbf{x}) := (\text{cf}_{\mathbf{b}}(x_i))_{i \in [m]}^T$  and  $\sigma(\mathbf{x}) := (\sigma(x_i))_{i \in [m]}^T$  are defined as vectors in  $\mathbb{Z}^{m\varphi}$  (resp.  $\mathbb{C}^{m\varphi}$ ) obtained by stacking the individual  $\text{cf}_{\mathbf{b}}(x_i)$  (resp.  $\sigma(x_i)$ ). For matrices, we define  $\text{cf}_{\mathbf{b}}$  and  $\sigma$  entry-wise so that  $\text{cf}_{\mathbf{b}}$  is a map from  $\mathcal{R}^{n \times m}$  to  $\mathbb{Z}^{n\varphi \times m}$  and  $\sigma$  is a map from  $\mathcal{R}^{n \times m}$  to  $\mathbb{C}^{n\varphi \times m}$ . The inverse of  $\text{cf}_{\mathbf{b}}$  is denoted by  $\text{cf}_{\mathbf{b}}^{-1}$ , and we write  $\text{cf}^{-1}$  when  $\mathbf{b}$  is the powerful basis. We write  $\bar{\cdot}$  for the *conjugation automorphism* of  $\mathcal{K}$ , the norm-preserving involution induced by  $\zeta \mapsto \zeta^{-1}$  (equivalently, complex conjugation under  $\sigma$ ), applied entry-wise to vectors and matrices.

**Lemma 1 (Dual modulo prime [KLNO25, Lemma 1]).** *Let  $q$  be an unramified prime, i.e.  $q \nmid \mathfrak{f}$ , and  $\mathcal{R}$  be the ring of integers of a cyclotomic field  $\mathcal{K}$ . For any  $\mathbb{Z}$ -basis  $\mathbf{b} = (b_i)_{i \in [\varphi]} \in \mathcal{R}^{\varphi}$  of  $\mathcal{R}$ , there exists  $\mathbf{b}^{\vee} = (b_i^{\vee})_{i \in [\varphi]} \in \mathcal{R}^{\varphi}$  such that  $\text{trace}(b_i \cdot b_j^{\vee}) = \delta_{i,j} \bmod q$ , where  $\delta_{i,j}$  denotes the Kronecker delta, for all  $i, j \in [\varphi]$ .*

While considering the coefficient embedding with respect to the vector  $\mathbf{b}^{\vee}$ , when  $\mathbf{b}$  is a powerful basis, we may omit  $\mathbf{b}^{\vee}$  and write only  $\text{cf}_{\vee}$ .

*Norms and Norm Conversion.* The coefficient  $\ell_p$ -norm and canonical  $\ell_p$ -norm of an element  $x \in \mathcal{R}$  are defined as  $\|x\|_p := \|\text{cf}_{\mathbf{b}}(x)\|_p$  and  $\|x\|_{\sigma,p} := \|\sigma(x)\|_p$  respectively, where  $\|\cdot\|_p$  is the standard  $\ell_p$ -norm on vectors. We extend the definitions naturally to vectors  $\mathbf{x} \in \mathcal{R}^m$  by defining  $\|\mathbf{x}\|_p := \|\text{cf}_{\mathbf{b}}(\mathbf{x})\|_p$  and  $\|\mathbf{x}\|_{\sigma,p} := \|\sigma(\mathbf{x})\|_p$ . For a matrix  $\mathbf{B} = (\mathbf{b}_0, \dots, \mathbf{b}_{m-1}) \in \mathcal{R}^{n \times m}$ , we define  $\|\mathbf{B}\|_p = \max_{i \in [m]} \|\mathbf{b}_i\|_p$  and similarly for  $\|\mathbf{B}\|_{\sigma,p}$ . By default, we use “norm” to refer to the  $\ell_2$ -norm over the coefficient embedding, i.e.  $\|\mathbf{x}\| := \|\text{cf}_{\mathbf{b}}(\mathbf{x})\|_2$ .

**Lemma 2 (Relations between norms ([KLNO24a, Lemma 1])).** *Let  $x \in \mathcal{K} = \mathbb{Q}(\zeta_{\mathfrak{f}})$  and  $\varphi = \varphi(\mathfrak{f})$ . Let  $\hat{\mathfrak{f}}$  be  $\mathfrak{f}$  if  $\mathfrak{f}$  is odd and  $\mathfrak{f}/2$  if it is even; let  $\text{rad}(\mathfrak{f})$  be the radical (i.e. the product of all primes dividing  $\mathfrak{f}$ ). Then we have  $\|x\|_2 \leq \sqrt{\frac{\text{rad}(\mathfrak{f})}{\hat{\mathfrak{f}}}} \|x\|_{\sigma,2}$  and  $\|x\|_{\sigma,2} \leq \sqrt{\hat{\mathfrak{f}}} \cdot \|x\|_2$ .*

*Operator norm and expansion factor.* Fix the coefficient embedding  $\text{cf}$  above. For  $c \in \mathcal{R}_q$ , let  $\|c\|_{\text{op}} := \sup_{t \in \mathcal{R}_q \setminus \{0\}} \frac{\|t \cdot c\|}{\|t\|}$ . For some set  $S \subseteq \mathcal{R}_q$ , we let  $\gamma_S := \max_{c \in S} \|c\|_{\text{op}}$ .

*Challenge set.* Given any ring  $\mathcal{R}_q$ , we call a set  $\mathcal{C}$  a *challenge set* if for all  $c_1 \neq c_2 \in \mathcal{C}$  the difference  $c_1 - c_2$  is invertible (modulo  $q$ ). For example, any field embedded in the ring is a challenge set. We say that  $\mathcal{C}$  is of norm  $\gamma$  if  $\gamma_{\mathcal{C}} \leq \gamma$ , i.e.  $\|c\|_{\text{op}} \leq \gamma$  for all  $c \in \mathcal{C}$ .

*Gadget decomposition.* Let  $\mathbf{x} \in \mathcal{R}^m$  be of norm at most  $\beta$  and dimension  $m$ . Let  $\mathbf{y} := \mathbf{G}_{\ell}^{-1}(\mathbf{x}) \in \mathcal{R}^{\ell m}$ , i.e. the gadget decomposition of  $\mathbf{x}$  into  $\ell m$  coefficients in  $\mathcal{R}$ . We denote the inverse operation by  $\mathbf{x} = \mathbf{G}_{\ell}(\mathbf{y})$ . We omit the subscript  $\ell$  when it is clear from the context. The (most pessimistic) bound on the base  $b$  of the gadget decomposition is  $b \geq \lceil \beta^{1/\ell} \rceil$ . Then, the norm of  $\mathbf{y}$  is at most  $\beta' = \sqrt{\varphi m \ell} \cdot b/2$ . We write  $\beta' = \text{dcmp}_{m,\ell}(\beta)$  or simply  $\beta' = \text{dcmp}(\beta)$  to denote this transformation. We also write  $\beta = \text{cmp}_{m,\ell}(\beta')$  or simply  $\beta = \text{cmp}(\beta')$  to denote the inverse transformation.

*NTT decomposition.* We assume that the cyclotomic polynomial  $\Phi_{\mathfrak{f}}(X)$  splits modulo  $q$  into  $\varphi/e$  irreducible factors of equal degree  $e$ , i.e.  $\Phi_{\mathfrak{f}}(X) \equiv \prod_{j=0}^{\varphi/e-1} f_j(X) \pmod{q}$ . By the Chinese Remainder Theorem, this induces a ring isomorphism  $\text{NTT}: \mathcal{R}_q \rightarrow \mathbb{F}_{q^e}^{\varphi/e}$ ,  $a \mapsto (a \bmod f_j)_{j \in [\varphi/e]}$ . The inverse map  $\text{NTT}^{-1}: \mathbb{F}_{q^e}^{\varphi/e} \rightarrow \mathcal{R}_q$  recovers the ring element from its components via CRT reconstruction.

### 3.2 Polynomials and Multilinear Extension

Let  $\mathbf{x} = (x_0, \dots, x_{\mu-1})$  and  $\mathbf{z} = (z_0, \dots, z_{\mu-1})$  be formal variables. We write  $\mathcal{R}_q[\mathbf{z}]^{\leq \delta}$  for the set of all (multivariate) polynomials over  $\mathcal{R}_q$  with variables  $\mathbf{z}$  with individual degree at most  $\delta$ . For a vector  $\mathbf{w} \in \mathcal{R}_q^m$  for  $m = 2^\mu$ , we write  $\text{MLE}[\mathbf{w}](\mathbf{z}) \in \mathcal{R}_q[\mathbf{z}]^{\leq 1}$  for the multilinear extension of (the function  $\{0, 1\}^\mu \rightarrow \mathcal{R}_q$  whose truth table is represented by)  $\mathbf{w}$ . We identify  $\{0, 1\}^\mu = [2^\mu]$  via bit decomposition with most significant bits first. We define the equality polynomial  $\text{eq} \in \mathcal{R}_q[\mathbf{x}, \mathbf{z}]$  as

$$\text{eq}(\mathbf{x}, \mathbf{z}) := \prod_{i \in [\mu]} (x_i \cdot z_i + (1 - x_i) \cdot (1 - z_i)).$$

For boolean inputs  $\mathbf{x}, \mathbf{z} \in \{0, 1\}^\mu$ ,  $\text{eq}(\mathbf{x}, \mathbf{z}) = 1$  if  $\mathbf{x} = \mathbf{z}$  and 0 otherwise. Sometimes, we abuse notation and write  $\text{eq}(\mathbf{z}) \in \mathcal{R}_q[\mathbf{x}]$  to denote the equality polynomial being partially evaluated at  $\mathbf{z} \in \mathcal{R}_q^\mu$ .

For  $\mathbf{x} = (x_0, \dots, x_{k-1}) \in \mathcal{R}_q^k$ , we write  $\text{tensor}(\mathbf{x}) = ((1 - x_0, x_0) \otimes \dots \otimes (1 - x_{k-1}, x_{k-1}))^T \in \mathcal{R}_q^{2^k}$ . By definition, it holds that  $\text{MLE}[\mathbf{w}](\mathbf{x}) = \text{tensor}(\mathbf{x})^T \cdot \mathbf{w}$ .

### 3.3 Computational Assumptions

*Vanishing Short Integer Solution (vSIS)* is a variant of the SIS problem, where the adversary is given access to a family of matrices  $\mathbf{A}_{n, 2^\mu}$  and must find a short vector  $\mathbf{w}$  in the kernel of some matrix  $\mathbf{A}_{n, 2^\mu}$ . The main distinction between vSIS and the ring-based variant of SIS is that the matrix  $\mathbf{A}_{n, 2^\mu} \in \mathcal{R}_q^{n \times 2^{\otimes \mu}}$  is not sampled uniformly at random but instead has a structured form: it is obtained via iterated row-tensor products of smaller matrices from the family, i.e.  $\mathbf{A}_{n, 2^\mu} = \mathbf{B}_{n, 0} \bullet \mathbf{B}_{n, 1} \bullet \dots \bullet \mathbf{B}_{n, \mu-1}$  for  $\mathbf{B}_{n, i} \in \mathcal{R}_q^{n \times 2}$  for  $i \in [\mu]$ .

**Definition 1 (vSIS Assumption (adapted from [CLM23])).** Let  $\text{par} = (\mathcal{R}, q, n, m, \beta)$  be parametrised by  $\lambda$ . A vSIS setup  $\text{Setup} = \text{Setup}_{\text{par}}$  is a PPT algorithm that takes as input  $(1^\lambda, 1^N, 1^M)$ , where  $N, M \in \mathbb{N}$ , and outputs an efficient description of family  $\mathfrak{A}$  of matrices  $\mathbf{A}_{n, 2^\mu} \in \mathcal{R}_q^{n \times 2^{\otimes \mu}}$ , for all  $n \leq N$ ,  $\mu \leq \log_2 M$ . The vSIS<sub>par</sub> assumption w.r.t. Setup states that, for any PPT adversary  $\mathcal{A}$ , the following advantage function is negligible:

$$\text{Adv}_{\text{par}, \mathcal{A}}^{\text{vSIS}}(\lambda) := \Pr \left[ \begin{array}{l} \mathbf{w} \in \mathcal{R}^m, n \leq N, m \leq M \\ m \text{ is a power of } 2 \\ \mathbf{A}_{n, m} \mathbf{w} = \mathbf{0} \bmod q \\ 0 \neq \|\mathbf{w}\|_2 \leq \beta \end{array} \middle| \begin{array}{l} (1^N, 1^M, st) \leftarrow \mathcal{A}(1^\lambda, \text{par}) \\ \mathfrak{A} \leftarrow \text{Setup}(1^\lambda, 1^N, 1^M) \\ \mathbf{w} \leftarrow \mathcal{A}(st, \mathfrak{A}) \end{array} \right].$$

### 3.4 Cryptographic Primitives

**Reduction of Knowledge (RoK)** In [KP23], ternary relations  $\Xi \subseteq \{0, 1\}^* \times \{0, 1\}^* \times \{0, 1\}^*$  are considered, where a tuple  $(\text{ppar}, \text{stmt}, \text{wit}) \in \Xi$  consists of public parameters  $\text{ppar}$ , statement  $\text{stmt}$ , and witness  $\text{wit}$ . For simplicity,  $\text{ppar}$  is omitted when it is clear from the context. Our definition of a reduction of knowledge is a streamlined variant of [KLNO24b].

**Definition 2 (Reduction of Knowledge (adapted from [KLNO24b])).** Let  $\Xi_0, \Xi_1$  be ternary relations. A reduction of knowledge (RoK)  $\Pi$  from  $\Xi_0$  to  $\Xi_1$ , short  $\Pi: \Xi_0 \rightarrow \Xi_1$ , is defined by two PPT algorithms  $\Pi = (\text{P}, \text{V})$ , the prover  $\text{P}$ , and the verifier  $\text{V}$ , with the following interface:

- $P(\text{ppar}, \text{stmt}_0, \text{wit}_0) \rightarrow (\text{stmt}_1, \text{wit}_1)$ : Interactively reduce the input statement  $(\text{ppar}, \text{stmt}_0, \text{wit}_0) \in \Xi_0$  to an output statement  $(\text{ppar}, \text{stmt}_1, \text{wit}_1) \in \Xi_1$  or  $\perp$ .
- $V(\text{ppar}, \text{stmt}_0) \rightarrow \text{stmt}_1$ : Interactively reduce the task of checking the input statement  $(\text{ppar}, \text{stmt}_0)$  w.r.t.  $\Xi_0$  to checking the output statement  $(\text{ppar}, \text{stmt}_1)$  w.r.t.  $\Xi_1$ .

A RoK  $\Pi$  is  $\epsilon$ -complete (with completeness error  $\epsilon$ ) if for any honest protocol run on correct inputs  $(\text{ppar}, \text{stmt}_0, \text{wit}_0) \in \Xi_0$ , the prover outputs a witness  $\text{wit}_1$  for the reduced statement  $\text{stmt}_1$  that the verifier outputs, i.e.  $(\text{ppar}, \text{stmt}_1, \text{wit}_1) \in \Xi_1$ , except with probability at most  $\epsilon$ . We say  $\Pi$  is *perfectly complete* if  $\epsilon = 0$ . A RoK  $\Pi$  is *knowledge sound* from  $\Xi_0^{\text{KS}}$  to  $\Xi_1^{\text{KS}}$  with knowledge error  $\kappa(\text{ppar}, \text{stmt})$  if there is a *black-box* expected polynomial-time extractor  $\text{Ext}$ , which succeeds with probability  $\epsilon - \kappa(\text{ppar}, \text{stmt})$  if the malicious prover outputs a valid witness for the reduced statement with probability  $\epsilon$  (on verifier's input  $(\text{ppar}, \text{stmt})$ ).<sup>9</sup>

**Definition 3 (Knowledge Soundness).** A reduction of knowledge  $\Pi = (P, V)$  from  $\Xi_0^{\text{KS}}$  to  $\Xi_1^{\text{KS}}$  is knowledge sound with knowledge error  $\kappa : \mathbb{N} \rightarrow [0, 1]$  if there exists a knowledge extractor  $\text{Ext}$  such that, for every input statement  $x$  and any (malicious) prover  $P^*$ , the extractor  $\text{Ext}^{P^*}(x)$  runs in expected time polynomial in  $|x|$  (counting calls to  $P^*$  as unit-cost operations) and outputs a witness  $w$  such that

$$\Pr[(x, w) \in \Xi_1^{\text{KS}}] \geq \epsilon(P^*, x) - \kappa(|x|),$$

where  $x$  is the input statement,  $w := \text{Ext}^{P^*}(x)$  is the extractor's candidate  $\Xi_1^{\text{KS}}$ -witness for  $x$ , and  $\epsilon(P^*, x)$  is the probability that the interaction  $(P^*, V)(x)$  makes the verifier output a statement for which  $P^*$  produces a valid  $\Xi_1^{\text{KS}}$ -witness.

We write  $\Xi_{\text{in}} \rightarrow \Xi_{\text{out}}$  for a RoK that reduces the relation  $\Xi_{\text{in}}$  to the relation  $\Xi_{\text{out}}$ ; completeness statements are phrased in this left-to-right direction. Knowledge-soundness statements instead use the reverse arrow:  $\Xi_{\text{ext}} \leftarrow \Xi_{\text{out}}$  reads right-to-left and denotes the *extraction direction* – from an accepting instance of the output relation  $\Xi_{\text{out}}$ , the extractor produces a witness for  $\Xi_{\text{ext}}$ . In particular, a disjunction such as  $\Xi_{\text{sis}}^{\text{sis}} \vee \Xi_{\text{COM}}^{\text{lin}} \leftarrow \Xi_{\text{COM}}^{\text{lin-rel}}$  means that, given a valid instance of  $\Xi_{\text{COM}}^{\text{lin-rel}}$ , the extractor outputs a witness for either  $\Xi_{\text{sis}}^{\text{sis}}$  or  $\Xi_{\text{COM}}^{\text{lin}}$ .

### 3.5 Commitments

Commitment schemes allow one to commit to a message while hiding it and ensuring binding. Our construction uses the recursive Ajtai commitment  $\text{COM}_{\text{par}_{\text{com}}, \beta} = (\text{Setup}, \text{Com}_{\text{par}_{\text{com}}}, \text{Verify}_{\text{par}_{\text{com}}, \beta})$  defined in Fig. 1, where  $\text{par}_{\text{com}} = (d, \mathbf{l}, \mathbf{n})$  and  $\beta = (\beta_0, \dots, \beta_{d-1})$  are parameters consisting of (i) a depth  $d \in \mathbb{N}$ , (ii) a list  $\mathbf{l} = (\ell_0, \dots, \ell_{d-2})$  of  $d-1$  gadget decomposition lengths, (iii) a list  $\mathbf{n} = (n_0, \dots, n_{d-1})$  of  $d$  output dimensions and (iv) a list  $\beta = (\beta_0, \dots, \beta_{d-1})$  of  $d$  bounds on the norms of the witness and auxiliary information. Typically, we will write  $(\text{par}_{\text{com}}, \beta_y, \beta_x)$  to denote the verification configuration, where  $\beta_y$  serves as a bound for the witness and  $\beta_x$  serves as a bound for the remaining layers, i.e. we set  $\beta_0 = \beta_y$  and  $\beta_1 = \dots = \beta_{d-1} = \beta_x$ . We write  $m_x[\text{par}_{\text{com}}]$  to denote the combined dimension of all vectors in the auxiliary information, i.e.  $m_x[\text{par}_{\text{com}}] = \sum_{i \in [d-1]} n_i \cdot \ell_i$ . We will also write  $\beta_x[\text{par}_{\text{com}}]$  to denote the bound on the norm of the auxiliary information (obtained from its decomposition), i.e.  $\beta_x[\text{par}_{\text{com}}]^2 = \sum_{i \in [d-1]} \text{dcmp}_{n_i, \ell_i}(q/2)^2$ .

<sup>9</sup> We postulate a black-box extractor that handles any (unbounded) prover. Hence, though the notion is stated as non-adaptive, it is straightforward to use in adaptive settings. Moreover, our success bound  $\epsilon - \kappa$  is stricter than usual.

| $\text{Com}_{\text{par}_{\text{com}}}(\text{ck}, \mathbf{w} \in \mathcal{R}^{2^\mu}) \rightarrow (\text{com}, \text{aux})$              | $\text{Verify}_{\text{par}_{\text{com}}, \beta}(\text{ck}, \mathbf{w} \in \mathcal{R}^{2^\mu}, \text{com}, \text{aux})$ |
|-----------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| $(n_i)_{i \in [d]} := \mathbf{n}$                                                                                                       | $\beta' := (\beta_1, \dots, \beta_{d-1})$                                                                               |
| $\mathbf{y} := \mathbf{A}_{n_0, 2^\mu} \cdot \mathbf{w} \bmod q$                                                                        | $\mathbf{y}^* := \text{com}$                                                                                            |
| <b>if</b> $d = 1$ :                                                                                                                     | $(n_i)_{i \in [d]} := \mathbf{n}$                                                                                       |
| <b>return</b> $(\mathbf{y}, \emptyset)$                                                                                                 | $b_0 := (\ \mathbf{w}\ _2 \leq \beta_0)$                                                                                |
| $(\ell_i)_{i \in [d-1]} := \mathbf{l}$                                                                                                  | <b>if</b> $d = 1$ :                                                                                                     |
| $\tilde{\mathbf{x}} = \mathbf{G}_{\ell_0}^{-1}(\mathbf{y})$                                                                             | <b>return</b> $b_0 \wedge (\mathbf{y}^* = \mathbf{A}_{n_0, 2^\mu} \cdot \mathbf{w} \bmod q)$                            |
| $\mathbf{x} := \tilde{\mathbf{x}} \parallel \mathbf{0} \in \mathcal{R}^{\text{minpowtwo}(\ell_0 n_0)} \quad // \text{ Pad } \mathbf{0}$ | <b>else</b> :                                                                                                           |
| $\mathbf{l}' := (\ell_1, \dots, \ell_{d-2}) \quad \mathbf{n}' := (n_1, \dots, n_{d-1})$                                                 | $(\ell_i)_{i \in [d-1]} := \mathbf{l}$                                                                                  |
| $(\mathbf{y}^*, \mathbf{x}^*) := \text{Com}_{d-1, \mathbf{l}', \mathbf{n}'}(\text{ck}, \mathbf{x})$                                     | $\mathbf{l}' := (\ell_1, \dots, \ell_{d-2}) \quad \mathbf{n}' := (n_1, \dots, n_{d-1})$                                 |
| $\text{aux} := (\mathbf{x}^*, \mathbf{x})$                                                                                              | $(\mathbf{x}^*, \mathbf{x}) := \text{aux}$                                                                              |
| <b>return</b> $(\mathbf{y}^*, \text{aux})$                                                                                              | $b_1 := (\mathbf{A}_{n_0, 2^\mu} \cdot \mathbf{w} = (\mathbf{G}_{\ell_0} \parallel \mathbf{0})(\mathbf{x}))$            |
|                                                                                                                                         | $b_2 := \text{Verify}_{d-1, \mathbf{l}', \mathbf{n}', \beta'}(\text{ck}, \mathbf{x}, \mathbf{y}^*, \mathbf{x}^*)$       |
|                                                                                                                                         | <b>return</b> $b_0 \wedge b_1 \wedge b_2$                                                                               |

**Fig. 1.** Commitment scheme  $\text{COM}_{\text{par}_{\text{com}}, \beta}$  for compressing claims, where  $\text{par}_{\text{com}} = (d, \mathbf{l}, \mathbf{n})$  and  $\beta = (\beta_0, \dots, \beta_{d-1})$  are parameters. If  $\beta_0 = \beta_y$  and  $\beta_1 = \dots = \beta_{d-1} = \beta_x$  we write  $\text{COM}_{\text{par}_{\text{com}}, \beta_y, \beta_x}$  instead of  $\text{COM}_{\text{par}_{\text{com}}, \beta}$ . The algorithm  $\text{ck} \leftarrow \text{Setup}(1^\lambda)$  samples a vSIS family (Definition 1) via  $\text{ck} := \mathfrak{A} \leftarrow \text{vSIS.Setup}(1^\lambda, 1^N, 1^M)$ , where  $N, M$  are asymptotic maximal bounds, e.g.,  $N = M = \lambda$ . Recall that  $\mathfrak{A}$  specifies matrices  $\mathbf{A}_{n, 2^\mu} \in \mathcal{R}_q^{n \times 2^{\otimes \mu}}$  for all  $n \leq N, \mu \leq \log_2 M$ .

*Remark 1.*  $\text{Verify}_{\text{par}_{\text{com}}, \beta}$  has an additional parameter  $\beta$  compared to  $\text{Com}_{\text{par}_{\text{com}}}$ , which gives us the flexibility to discuss the verification of commitment openings with different norm bounds. In the context of our argument system, this flexibility allows us to capture the difference between the norm bounds that arises from decomposition and those from witness extraction.

We will write  $\#\text{constraints}(\text{par}_{\text{com}}) = \sum_{i \in [d]} n_i$  to denote the number of linear constraints over  $\mathcal{R}_q$  that need to be checked for the verification.<sup>10</sup>

### 3.6 Packing

We define the packing function  $\text{pack} : (\mathcal{R}_q^{m_0}, \dots, \mathcal{R}_q^{m_{k-1}}) \rightarrow \mathcal{R}_q^m$ , where each  $m_i$  is a power of two,  $m' = \sum_{i \in [k]} m_i$ , and  $m = 2^{\lceil \log m' \rceil}$  is the smallest power of two greater than or equal to  $m'$ . The function  $\text{pack}$  first reorders the input vectors by decreasing dimension (i.e. so that  $m_0 \geq m_1 \geq \dots \geq m_{k-1}$ ), then concatenates them and pads the result with zeros to obtain a vector of dimension  $m$ : For  $i \in [k]$ , we define the *offset*  $\omega_i := \sum_{j < i} m_j$  as the starting index of  $\mathbf{w}_i$  within the packed vector.

*Remark 2.* Since  $m' \leq m < 2m'$ , the zero-padding adds at most  $m - m' < m'$  entries, so less than half of the packed vector consists of zeros.

**Lemma 3 (Packing and Multilinear Extensions).** *Let  $\mathbf{w}_i \in \mathcal{R}_q^{m_i}$  for  $i \in [k]$  and suppose that  $m_0 \geq m_1 \geq \dots \geq m_{k-1}$  and all  $m_i$  are powers of two. Let  $\mathbf{w} = \text{pack}(\mathbf{w}_0, \dots, \mathbf{w}_{k-1}) \in \mathcal{R}_q^m$ . Let  $f = \text{MLE}[\mathbf{w}] \in \mathcal{R}_q[\mathbf{z}_0, \dots, \mathbf{z}_{\mu-1}]$  with  $\mu = \log m$ , and let  $f_i = \text{MLE}[\mathbf{w}_i] \in \mathcal{R}_q[\mathbf{z}_0, \dots, \mathbf{z}_{\mu_i-1}]$  with*

<sup>10</sup> Throughout the rest of the paper, we consider our commitment  $\text{COM}$  to be the “recursive Ajtai commitment” defined in Fig. 1. Nevertheless, if desired, one can instantiate  $\text{COM}$  with any commitment scheme that allows for the verification to be expressed as a set of linear low-norm constraints with auxiliary data of length and norm upper-bounded by parameters  $m_x[\text{par}_{\text{com}}]$  and  $\beta_x$ .

$\mu_i = \log m_i$  for each  $i \in [k]$ . Let  $\mathbf{p}_i \in \{0, 1\}^{\mu - \mu_i}$  be the binary representation of  $\omega_i/m_i$  (i.e. the  $(\mu - \mu_i)$ -bit prefix identifying the block where  $\mathbf{w}_i$  resides). Then, for all  $i \in [k]$ :

$$f(\mathbf{p}_i, \mathbf{z}) = f_i(\mathbf{z}) \bmod q.$$

*Proof.* We first show that the offset  $\omega_i$  is a multiple of  $m_i$ , i.e. that the lower-order  $\mu_i$  bits of  $\omega_i$  are all zero. Since the vectors are ordered by decreasing dimension, for each  $j < i$  we have  $m_j \geq m_i$ , and since each  $m_j$  is a power of two,  $m_j$  is a multiple of  $m_i$ . Therefore,  $\omega_i = \sum_{j < i} m_j$  is a sum of multiples of  $m_i$ , hence itself a multiple of  $m_i$ .

We now verify that  $\mathbf{p}_i$  correctly identifies the position of  $\mathbf{w}_i$ . Since  $\mathbf{w}_i$  starts at offset  $\omega_i$  and has dimension  $m_i = 2^{\mu_i}$ , the indices occupied by  $\mathbf{w}_i$  in  $\mathbf{w}$  are  $\{\omega_i, \omega_i + 1, \dots, \omega_i + m_i - 1\}$ .

For any index  $j \in \{0, \dots, m_i - 1\}$ , the corresponding global index in  $\mathbf{w}$  is  $\omega_i + j$ . Writing  $\omega_i + j$  in binary with  $\mu$  bits, the high-order  $(\mu - \mu_i)$  bits encode  $\lfloor (\omega_i + j)/m_i \rfloor$ . Since  $0 \leq j < m_i$ , we have  $\lfloor (\omega_i + j)/m_i \rfloor = \lfloor \omega_i/m_i \rfloor = \omega_i/m_i$  (using the divisibility established above). Thus, all indices in the range  $[\omega_i, \omega_i + m_i - 1]$  share the common prefix  $\mathbf{p}_i$ .

Now, by definition of the multilinear extension, for  $\mathbf{z} \in \{0, 1\}^{\mu_i}$  we have  $f(\mathbf{p}_i, \mathbf{z}) = w_{\langle \mathbf{p}_i, \mathbf{z} \rangle}$ , where  $\langle \cdot \rangle$  denotes the integer with the given binary representation. Since  $\langle \langle \mathbf{p}_i, \mathbf{z} \rangle \rangle = \langle \mathbf{p}_i \rangle \cdot m_i + \langle \mathbf{z} \rangle = \omega_i + \langle \mathbf{z} \rangle$ , we have

$$f(\mathbf{p}_i, \mathbf{z}) = w_{\omega_i + \langle \mathbf{z} \rangle} = \mathbf{w}_{i, \langle \mathbf{z} \rangle} = f_i(\mathbf{z}) \bmod q.$$

For our applications of `pack`, we can w.l.o.g. reorder the vectors in decreasing dimension. Hence Lemma 3 is applicable.

## 4 Relations

In this section, we define the relations that we will use in our reductions of knowledge. All relations implicitly take as parameters  $(\mathcal{R}, q)$ , which are typically omitted for brevity; If  $\Xi$  is a relation and  $(\text{par}, \text{stmt}, \text{wit}) \in \Xi$ , we write  $(\text{stmt}, \text{wit}) \in \Xi[\text{par}]$ . We recall that  $\mathfrak{A} \leftarrow \text{vSIS.Setup}(1^\lambda, 1^N, 1^M)$  is the vSIS family generation algorithm, which samples a family  $\mathfrak{A}$  of matrices  $\mathbf{A}_{n, 2^\mu}$  for all  $n \leq N$ ,  $\mu \leq \log_2 M$  as in Definition 1.

*Committed-linear relation (and its relaxed variant).* Let  $\text{COM} = \text{COM}_{\text{par}_{\text{com}}, \beta_y, \beta_x} = (\text{Setup}, \text{Com}, \text{Verify})$  be the commitment scheme from Fig. 1.

We define the committed-linear relation  $\Xi_{\text{COM}}^{\text{lin}}$  with respect to the commitment scheme  $\text{COM}$ :

- Parameters:  $\text{par} = (\text{ck} = \mathfrak{A}, k_{\text{lin}}, k_{\text{lr}}, n, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, \underbrace{(\text{par}_{\text{lin}, i})_{i \in [k_{\text{lin}}]}}_{(n_i, \text{par}_{\text{com}, i}, m_{y, i}, \beta_{x, i}, \beta_{y, i})}, \varrho)$ .
- Statement:  $\text{stmt} = ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}, \mathbf{b})$ , where:
  - $(\text{com}_i, \mathbf{F}_i \in \mathcal{R}_q^{n_i \times m_{\mathbf{w}}}, \mathbf{H}_i \in \mathcal{R}_q^{n_i \times m_{y, i}})_{i \in [k_{\text{lin}}]}$  with  $\mathbf{F}_0 \in \mathfrak{A}$  being a vSIS commitment key.
  - $(\mathbf{l}_j \in \mathcal{R}_q^{m_{\mathbf{w}}}, \mathbf{r}_j \in \mathcal{R}_q^r, t_j)_{j \in [k_{\text{lr}}]}$
  - $\mathbf{A} \in \mathcal{R}_q^{n \times \sum_{i \in [k_{\text{lin}}]} m_{y, i} \cdot r}$ ,  $\mathbf{b} \in \mathcal{R}_q^n$
- Witness:  $\text{wit} = (\mathbf{W} \in \mathcal{R}^{m_{\mathbf{w}} \times r}, (\mathbf{Y}_i \in \mathcal{R}^{m_{y, i} \times r}, \mathbf{x}_i \in \mathcal{R}^{m_{\mathbf{x}}[\text{par}_{\text{com}, i}]})_{i \in [k_{\text{lin}}]}, \mathbf{s} \in \mathcal{R}^r)$
- Constraints:
  - For all  $i \in [k_{\text{lin}}]$ :
    - \*  $\mathbf{F}_i \mathbf{W} = \mathbf{H}_i \mathbf{Y}_i \bmod q$
    - \*  $\text{COM.Verify}_{\text{par}_{\text{com}, i}, \beta_{y, i}, \beta_{x, i}}(\text{ck}, \text{vec}(\mathbf{Y}_i), \text{com}_i, \mathbf{x}_i) = 1$

- \*  $\|\text{vec}(\mathbf{Y}_i)\| \leq \beta_{\mathbf{y},i}, \|\mathbf{x}_i\| \leq \beta_{\mathbf{x},i}$
- For all  $j \in [k_{1r}], \mathbf{l}_j^T \cdot \mathbf{W} \cdot \mathbf{r}_j = t_j \bmod q$
- $\mathbf{A} \cdot (\text{vec}(\mathbf{Y}_i))_{i \in [k_{1in}]} = \mathbf{b} \bmod q$
- $\|\mathbf{W} \cdot \text{diag}(\mathbf{s})\|_2 \leq \beta_{\mathbf{w}}$
- $\|\text{diag}(\mathbf{s})\|_{\text{op},2} \leq \varrho$

Although the witness  $\mathbf{W}$  does not appear inside an explicit commitment verification, it is committed: since  $\mathbf{F}_0 \in \mathfrak{A}$  is a vSIS commitment key, the constraint  $\mathbf{F}_0 \mathbf{W} = \mathbf{H}_0 \mathbf{Y}_0 \bmod q$  binds  $\mathbf{H}_0 \mathbf{Y}_0$  as a commitment of  $\mathbf{W}$ . Such a notation is necessary since the first commitment layer binds columns of the witness  $\mathbf{W}$  individually under the same commitment key  $\mathbf{F}_0$ , allowing for short random linear combinations of the columns of  $\mathbf{W}$ .

We say that  $\Xi_{\text{COM}}^{\text{lin-rel}}$  is **relaxed** if  $\varrho > 1$ . We take  $\varrho = 1$  if omitted and in which case we can assume without loss of generality that  $\mathbf{s} = \mathbf{1}$  is the all-one vector.

*Functional-sumcheck relation.* Let  $\text{COM} = \text{COM}_{\text{par}_{\text{com}}, \beta_{\mathbf{y}}, \beta_{\mathbf{x}}} = (\text{Setup}, \text{Com}, \text{Verify})$  be the commitment scheme from Fig. 1. We define the functional-sumcheck relation  $\Xi_{\text{COM}}^{\text{sum}}$  as follows:

- Parameters:  $\text{par} = (\text{ck} = \mathfrak{A}, k_{\text{sc}}, n, m_{\mathbf{w}}, r, \text{par}_{\text{com}}, m_{\mathbf{y}}, \beta_{\mathbf{w}}, \beta_{\mathbf{x}}, \beta_{\mathbf{y}}, \delta_{\text{sc}})$ ,  $m_{\mathbf{w}} \cdot r = 2^\nu$  is a power of 2
- Statement:  $\text{stmt} = (\text{com}, \mathbf{F} \in \mathcal{R}_q^{n \times m_{\mathbf{w}}}, \mathbf{H} \in \mathcal{R}_q^{n \times m_{\mathbf{y}}}, (f_{\text{sc}}^{(i)})_{i \in [k_{\text{sc}}]})$  where
  - $\mathbf{F} \in \mathfrak{A}$  is a vSIS commitment key.
  - For  $i \in [k_{\text{sc}}]$ , each  $f_{\text{sc}}^{(i)} \in (\mathcal{R}_q[\mathbf{y}])[x_0, x_1]$  is a polynomial in variables  $x_0, x_1$  over the polynomial ring  $\mathcal{R}_q[\mathbf{y}]$ .
  - $\mathcal{R}_q[\mathbf{y}]$  is the polynomial ring over  $\mathcal{R}_q$  in variables  $\mathbf{y} := (y_0, \dots, y_{\nu-1})$ .
  - For  $x_0, x_1 \in \mathcal{R}_q[\mathbf{y}]^{\leq 1}$ , it holds that  $f_{\text{sc}}^{(i)}(x_0, x_1) \in \mathcal{R}_q[\mathbf{y}]^{\leq \delta_{\text{sc}}}$  for all  $i \in [k_{\text{sc}}]$ .
- Witness:  $\text{wit} = (\mathbf{W} \in \mathcal{R}^{m_{\mathbf{w}} \times r}, \mathbf{Y} \in \mathcal{R}^{m_{\mathbf{y}} \times r}, \mathbf{x} \in \mathcal{R}^{m_{\mathbf{x}}[\text{par}_{\text{com}}]})$
- Constraints:
  - $\mathbf{F}\mathbf{W} = \mathbf{H}\mathbf{Y} \bmod q$
  - $\text{COM.Verify}_{\text{par}_{\text{com}}, \beta_{\mathbf{y}}, \beta_{\mathbf{x}}}(\text{ck}, \text{vec}(\mathbf{Y}), \text{com}, \mathbf{x}) = 1$
  - $i \in [k_{\text{sc}}]: \sum_{\mathbf{z} \in \{0,1\}^\nu} f_{\text{sc}}^{(i)} \left( \text{MLE}[\text{vec}(\mathbf{W})], \text{MLE}[\overline{\text{vec}(\mathbf{W})}] \right) (\mathbf{z}) = 0 \bmod q$
  - $\|\mathbf{W}\|_2 \leq \beta_{\mathbf{w}}, \|\text{vec}(\mathbf{Y})\|_2 \leq \beta_{\mathbf{y}}, \|\mathbf{x}\|_2 \leq \beta_{\mathbf{x}}$

Similarly to the committed-linear relation, the witness  $\mathbf{W}$  is committed via the constraint  $\mathbf{F}\mathbf{W} = \mathbf{H}\mathbf{Y} \bmod q$ .

*SIS relation.* We rely on (v)SIS-assumptions of varying dimension and norm bounds, chosen with the same hardness. Below, we define the corresponding (v)SIS-family relation  $\Xi^{\text{sis}}$ .

- Parameters:  $\text{par} = (\text{ck} = \mathfrak{A}, \{\text{par}_i\}_{i \in [T]})$  where
  - $\mathfrak{A} = (\mathbf{A}_{n, 2^\mu})_{n \leq N, \mu \leq M}$  for  $N, M \in \mathbb{N}$ ,
  - $\text{par}_i = (n_i, m_{\mathbf{w}, i}, \beta_{\mathbf{w}, i})$  for  $m_{\mathbf{w}, i} = 2^{\mu_i}$  for  $\mu_i \in \mathbb{N}$ .
- Statement:  $\text{stmt} = \varepsilon$
- Witness:  $\text{wit} = (\mathbf{w} \in \mathcal{R}^{m_{\mathbf{w}, i}})$  for some  $i \in [T]$ .
- Constraints:  $\exists i \in [T]$  s.t.
  - $\mathbf{A}_{n_i, m_{\mathbf{w}, i}} \mathbf{w} = \mathbf{0} \bmod q$ ,
  - $0 \neq \|\mathbf{w}\|_2 \leq \beta_{\mathbf{w}, i}$ .

**Table 2.** Individual parameters appearing in relations.

| Symbol                                        | Type                  | Description                                                                          |
|-----------------------------------------------|-----------------------|--------------------------------------------------------------------------------------|
| $k_{\text{lin}}$                              | $\mathbb{N}$          | Number of committed-linear blocks                                                    |
| $k_{\text{lr}}$                               | $\mathbb{N}$          | Number of left-right inner-product constraints                                       |
| $k_{\text{sc}}$                               | $\mathbb{N}$          | Number of sumcheck polynomials                                                       |
| $n$                                           | $\mathbb{N}$          | Number of rows of global linear constraint                                           |
| $m_{\mathbf{w}}$                              | $\mathbb{N}$          | Number of rows of main witness $\mathbf{W}$                                          |
| $r$                                           | $\mathbb{N}$          | Number of columns of main witness $\mathbf{W}$                                       |
| $m_{\mathbf{y}} / m_{\mathbf{y},i}$           | $\mathbb{N}$          | Dimension of committed vector $\mathbf{Y} / \mathbf{Y}_i$                            |
| $m_{\mathbf{x}}[\text{par}_{\text{com}}]$     | $\mathbb{N}$          | Derived auxiliary information dimension: $\sum_{i \in [d-1]} n_i \cdot \ell_i$       |
| $\beta_{\mathbf{w}}$                          | $\mathbb{R}_{>0}$     | Norm bound on the main witness $\mathbf{W}$                                          |
| $\beta_{\mathbf{x}} / \beta_{\mathbf{x},i}$   | $\mathbb{R}_{>0}$     | Norm bound on commitment auxiliary information $\mathbf{x} / \mathbf{x}_i$           |
| $\beta_{\mathbf{x}}[\text{par}_{\text{com}}]$ | $\mathbb{R}_{>0}$     | Derived auxiliary norm: $\sqrt{\sum_{i \in [d-1]} \text{dcmp}_{n_i, \ell_i}(q/2)^2}$ |
| $\beta_{\mathbf{y}} / \beta_{\mathbf{y},i}$   | $\mathbb{R}_{>0}$     | Norm bound on committed right-hand sides $\mathbf{Y} / \mathbf{Y}_i$ (vectorised)    |
| $n_i$                                         | $\mathbb{N}$          | Output dimension at layer $i$ of the commitment / SIS rank                           |
| $\delta_{\text{sc}}$                          | $\mathbb{N}$          | Degree bound for sumcheck polynomials                                                |
| $\varrho$                                     | $\mathbb{R}_{\geq 1}$ | Relaxation factor; $\varrho = 1$ means non-relaxed                                   |
| $T$                                           | $\mathbb{N}$          | Number of SIS-parameter tuples in $\Xi^{\text{sis}}$                                 |

*Commitment breaks to SIS break.* Let  $\text{COM} = \text{COM}_{\text{par}_{\text{com}}} = (\text{Setup}, \text{Com}, \text{Verify})$  be the commitment scheme from Fig. 1. A commitment binding break consists of a commitment  $\text{com}$  and of two openings  $(\mathbf{w}_b \in \mathcal{R}^{m_{\mathbf{w}}}, \mathbf{x}_b \in \mathcal{R}^{m_{\mathbf{x}}[\text{par}_{\text{com}}]} )_{b \in [2]}$  such that  $\mathbf{w}_0 \neq \mathbf{w}_1$ , and yet  $\text{COM.Verify}_{\text{par}_{\text{com}}}(\text{ck}, \mathbf{w}_b, \text{com}, \mathbf{x}_b) = 1$  for  $b \in [2]$ . The following result follows immediately from the definition of  $\text{COM.Verify}_{\text{par}_{\text{com}}}$ .

**Lemma 4 (Binding break to SIS break).** *Let  $\text{COM}_{\text{par}_{\text{com}}, \beta} = (\text{Setup}, \text{Com}_{\text{par}_{\text{com}}}, \text{Verify}_{\text{par}_{\text{com}}, \beta})$  be the commitment scheme from Fig. 1 where  $\text{par}_{\text{com}} = (d, \mathbf{l}, \mathbf{n})$ . Let  $\text{par}_{\text{break}}[\text{par}_{\text{com}}, m_{\mathbf{y}}, \beta] = \{(n_0, m_{\mathbf{y}}, 2\beta_{\mathbf{y}})\} \cup \{(n_{i+1}, \ell_i \cdot n_i, 2\beta_{\mathbf{x},i})\}_{i \in [d-1]}$  where  $\beta = (\beta_{\mathbf{y}}, \beta_{\mathbf{x},0}, \dots, \beta_{\mathbf{x},d-2})$ . Consider a SIS relation  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}}, m_{\mathbf{y}}, \beta]$ . Given a binding break  $(\mathbf{w}_b \in \mathcal{R}^{m_{\mathbf{w}}}, \mathbf{x}_b \in \mathcal{R}^{m_{\mathbf{x}}[\text{par}_{\text{com}}]} )_{b \in [2]}$  for  $\text{com}$ , one can efficiently compute a witness  $\mathbf{z}$  for  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  with parameters  $(n_0, m_{\mathbf{y}}, 2\beta_{\mathbf{y}})$  or  $(n_{i+1}, \ell_i \cdot n_i, 2\beta_{\mathbf{x},i})$  for some  $i \in [d-1]$ . As in Section 3.4, we will typically write  $\text{par}_{\text{break}}[\text{par}_{\text{com}}, m_{\mathbf{y}}, \beta_{\mathbf{y}}, \beta_{\mathbf{x}}]$  instead of  $\text{par}_{\text{break}}[\text{par}_{\text{com}}, m_{\mathbf{y}}, \beta]$  if  $\beta_0 = \beta_{\mathbf{y}}$  and  $\beta_1 = \dots = \beta_{d-1} = \beta_{\mathbf{x}}$ .*

*Parameter reference.* Table 2 collects parameters used in the relations above, and Table 3 lists the parameter sets.

## 5 Committed Random Projection

Random projection controls the norm growth during extraction. The extractor for folding (Section 6) outputs a witness with a “slack” factor  $\varrho$ : the extracted  $\mathbf{W}$  satisfies  $\|\mathbf{W} \cdot \text{diag}(\mathbf{s})\|_2 \leq \beta'_{\mathbf{w}}$  for some  $\mathbf{s}$  with  $\|\mathbf{s}\|_2 \leq \varrho$ . In recursive composition, this slack would accumulate and blow up the norm bound. Applying random projection before folding ensures that the projected witness satisfies a strict norm bound (without slack) with high probability. We give two projection RoKs, inspired by [KLNO25]. A *coarse* variant  $\Pi^{\text{proj-c}}$  (Fig. 2) applies a random projection matrix to full ring elements; it is simple and efficient but requires the witness dimension to be large enough for the projection to compress. A *fine* variant  $\Pi^{\text{proj-f}}$  (Fig. 3) instead projects the coefficients of the witness ring elements, handling

**Table 3.** Parameter sets.

| Parameter Set                                                                                                                                                                                                      | Description                                                                                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{par}_{\text{com}} = (d, \mathbf{l}, \mathbf{n})$                                                                                                                                                            | Commitment parameters: depth $d$ , gadget-decomposition lengths $\mathbf{l} = (\ell_0, \dots, \ell_{d-2})$ , output dimensions $\mathbf{n} = (n_0, \dots, n_{d-1})$ . Together with the verification norms $\boldsymbol{\beta} := (\beta_0, \dots, \beta_{d-1})$ , this defines the configuration for the commitment scheme $\text{Com}_{\text{par}_{\text{com}}, \boldsymbol{\beta}}$ . |
| $\text{par}_{\text{lin}} = \left( \begin{array}{c} k_{\text{lin}}, k_{\text{lr}}, n, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, \\ (\text{par}_{\text{lin}, i})_{i \in [k_{\text{lin}}]}, \varrho \end{array} \right)$ | Parameter set for the committed-linear relation $\Xi_{\text{COM}}^{\text{lin}}$ .                                                                                                                                                                                                                                                                                                        |
| $\text{par}_{\text{lin}, i} = (n_i, \text{par}_{\text{com}, i}, m_{y, i}, \beta_{x, i}, \beta_{y, i})$                                                                                                             | Per-block parameters for the $i$ -th committed-linear block: output dimension, commitment config, committed-vector dimension, auxiliary norm bound, committed-vector norm bound.                                                                                                                                                                                                         |
| $\text{par}_{\text{sc}} = \left( \begin{array}{c} k_{\text{sc}}, n, m_{\mathbf{w}}, r, \text{par}_{\text{com}}, \\ m_y, \beta_{\mathbf{w}}, \beta_{\mathbf{x}}, \beta_y, \delta_{\text{sc}} \end{array} \right)$   | Parameter set for the functional-sumcheck relation $\Xi_{\text{COM}}^{\text{sum}}$ .                                                                                                                                                                                                                                                                                                     |
| $\text{par}_{\text{sis}} = \{\text{par}_i\}_{i \in [T]}$ where<br>$\text{par}_i = (n_i, m_{\mathbf{w}, i}, \beta_{\mathbf{w}, i})$                                                                                 | SIS-family relation parameters: a set of (rank, dimension, norm-bound) tuples.                                                                                                                                                                                                                                                                                                           |
| $\text{par}_{\text{break}}[\text{par}_{\text{com}}, m_y, \boldsymbol{\beta}]$ or<br>$\text{par}_{\text{break}}[\text{par}_{\text{com}}, m_y, \beta_y, \beta_x]$                                                    | SIS parameters arising from a commitment binding break (Lemma 4), with $\boldsymbol{\beta} = (\beta_y, \beta_{x, 0}, \dots, \beta_{x, d-2})$ : $\{(n_0, m_y, 2\beta_y)\} \cup \{(n_{i+1}, \ell_i \cdot n_i, 2\beta_{x, i})\}_{i \in [d-1]}$ . Simplified variant uses the collective bound $\beta_x^2 = \sum_{i \in [d-1]} \beta_{x, i}^2$ .                                             |

smaller witness dimensions. Both are self-reductions for the committed linear relation  $\Xi_{\text{COM}}^{\text{lin}}$ , adding fresh commitments to the projected witnesses.

### 5.1 Johnson–Lindenstrauss Lemma

We first recall the following lemma, which is a ring variant of the Johnson–Lindenstrauss lemma taken from [BS23] and adapted to our setting.

**Lemma 5.** *Let  $\chi$  be a distribution over  $\{-1, 0, 1\}$  with  $\Pr[\chi = 0] = \frac{1}{2}$ ,  $\Pr[\chi = 1] = \Pr[\chi = -1] = \frac{1}{4}$ ,  $q \in \mathbb{N}$  and  $\kappa_{\text{rp}} \in (0, 1)$ . There exist parameters  $n_{\text{rp}} = n_{\text{rp}}(\kappa_{\text{rp}})$ ,  $\alpha_{\text{rp}} = \alpha_{\text{rp}}(\kappa_{\text{rp}})$ ,  $\beta_{\text{rp}} = \beta_{\text{rp}}(\kappa_{\text{rp}})$  and  $b = b(\kappa_{\text{rp}})$  such that for any  $m_{\text{rp}} \in \mathbb{N}$  and any non-zero  $\mathbf{w} \in \mathbb{Z}^{m_{\text{rp}}}$ , we have*

$$\Pr[\|\mathbf{J}\mathbf{w}\|_2 / \|\mathbf{w}\|_2 \notin [\alpha_{\text{rp}}, \beta_{\text{rp}}] \mid \mathbf{J} \leftarrow_{\$} \chi^{n_{\text{rp}} \times m_{\text{rp}}}] \leq \kappa_{\text{rp}}.$$

and for  $0 < \theta \leq q/b$  and  $\|\mathbf{w}\|_2 \geq \theta$  such that  $\mathbf{w} \in [\pm q/2]^{m_{\text{rp}}}$  it holds that

$$\Pr[\|\mathbf{J}\mathbf{w} \bmod q\|_2 \leq \alpha_{\text{rp}}\theta \mid \mathbf{J} \leftarrow_{\$} \chi^{n_{\text{rp}} \times m_{\text{rp}}}] \leq \kappa_{\text{rp}}.$$

Concretely, for  $\kappa_{\text{rp}} = 2^{-128}$ , we take  $n_{\text{rp}} = 256$ ,  $\alpha_{\text{rp}} = \sqrt{30}$ ,  $\beta_{\text{rp}} = \sqrt{337}$  and  $b = 125$  [BS23, Lemma 4.1 and 4.2].

Moreover, for  $n_{\text{rp}} = \Theta(\log(1/\kappa_{\text{rp}}))$  we conjecture the asymptotic scaling

$$\alpha_{\text{rp}}(\kappa_{\text{rp}}) = \Theta(\sqrt{\log(1/\kappa_{\text{rp}})}) \quad \beta_{\text{rp}}(\kappa_{\text{rp}}) = \Theta(\sqrt{\log(1/\kappa_{\text{rp}})}) \quad b(\kappa_{\text{rp}}) = \Theta(\sqrt{\log(1/\kappa_{\text{rp}})}),$$

stated formally as Conjecture 1.

We extend Lemma 5 to cover structured projections.

**Lemma 6.** Let  $\mathbf{W} \in \mathbb{Z}^{m_u \times r}$  be non-zero and  $q \in \mathbb{N}$ . Let  $\mathbf{J} \leftarrow \$ \chi^{n_{\text{rp}} \times m_{\text{rp}}}$  be a random matrix as in Lemma 5 and define  $\mathbf{V} := (\mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{J}) \cdot \mathbf{W} \in \mathbb{Z}^{m_u \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$  with columns  $\mathbf{v}_0, \dots, \mathbf{v}_{r-1}$ . Then, it holds that

$$\Pr[\|\mathbf{V}\|_2 / \|\mathbf{W}\|_2 \notin [\alpha_{\text{rp}}, \beta_{\text{rp}}]] \leq \kappa_{\text{rp}} \cdot r m_w / m_{\text{rp}},$$

and for  $0 < \theta \leq q/b$  and  $\|\mathbf{W}\|_2 \geq \theta$  such that  $\mathbf{W} \in [\pm q/2]^{m_u \times r}$  it holds that

$$\Pr[\|\mathbf{V} \bmod q\|_2 \leq \alpha_{\text{rp}} \theta] \leq \kappa_{\text{rp}} \cdot m_w / m_{\text{rp}},$$

where  $\alpha_{\text{rp}}, \beta_{\text{rp}}, \kappa_{\text{rp}}, b$  are as in Lemma 5.

*Proof.* Since we define norm on matrices as the largest Euclidean norm of its columns, it suffices to prove the statement for each column of  $\mathbf{W}$  and  $\mathbf{V}$  separately and then apply the union bound.

By definition of the Euclidean norm on matrices, we have that

$$\|\mathbf{W}\|_2 = \max_{j \in [r]} \|\mathbf{w}_j\|_2, \quad \|\mathbf{V}\|_2 = \max_{j \in [r]} \|\mathbf{v}_j\|_2,$$

where  $\mathbf{w}_j$  and  $\mathbf{v}_j$  are the  $j$ -th columns of  $\mathbf{W}$  and  $\mathbf{V}$ , respectively. Let us fix an arbitrary column  $j \in [r]$ . We split the vector  $\mathbf{w}_j$  into  $m_w/m_{\text{rp}}$  chunks of height  $m_{\text{rp}}$  each:  $\mathbf{w}_{j,1}, \dots, \mathbf{w}_{j,m_u/m_{\text{rp}}}$ . Similarly, we split  $\mathbf{v}_j$  into  $m_w/m_{\text{rp}}$  chunks of height  $n_{\text{rp}}$  each:  $\mathbf{v}_{j,1}, \dots, \mathbf{v}_{j,m_u/m_{\text{rp}}}$ .

By construction of  $\mathbf{V}$ , it holds that  $\mathbf{v}_{j,i} = \mathbf{J} \mathbf{w}_{j,i}$  for all  $i \in [m_w/m_{\text{rp}}]$ . By the first part of Lemma 5, for each  $i \in [m_w/m_{\text{rp}}]$ , it holds that

$$\Pr\left[\frac{\|\mathbf{v}_{j,i}\|_2}{\|\mathbf{w}_{j,i}\|_2} \notin [\alpha_{\text{rp}}, \beta_{\text{rp}}]\right] \leq \kappa_{\text{rp}}.$$

Assuming that  $\frac{\|\mathbf{v}_{j,i}\|_2}{\|\mathbf{w}_{j,i}\|_2} \in [\alpha_{\text{rp}}, \beta_{\text{rp}}]$ , we observe that the Euclidean norm behaves under concatenation as follows:

$$\|\mathbf{v}_j\|_2^2 = \sum_{i \in [m_w/m_{\text{rp}}]} \|\mathbf{v}_{j,i}\|_2^2 \in \left[ \alpha_{\text{rp}}^2 \sum_{i \in [m_w/m_{\text{rp}}]} \|\mathbf{w}_{j,i}\|_2^2, \beta_{\text{rp}}^2 \sum_{i \in [m_w/m_{\text{rp}}]} \|\mathbf{w}_{j,i}\|_2^2 \right] = [\alpha_{\text{rp}}^2 \|\mathbf{w}_j\|_2^2, \beta_{\text{rp}}^2 \|\mathbf{w}_j\|_2^2].$$

It follows that, conditioned on the per-chunk events above holding for every  $i \in [m_w/m_{\text{rp}}]$ ,

$$\frac{\|\mathbf{v}_j\|_2}{\|\mathbf{w}_j\|_2} \in [\alpha_{\text{rp}}, \beta_{\text{rp}}], \quad \text{equivalently} \quad \|\mathbf{v}_j\|_2 \in [\alpha_{\text{rp}} \|\mathbf{w}_j\|_2, \beta_{\text{rp}} \|\mathbf{w}_j\|_2].$$

Since  $j$  was arbitrary, a union bound over all  $(i, j) \in [m_w/m_{\text{rp}}] \times [r]$  shows that, except with probability at most  $\kappa_{\text{rp}} \cdot r m_w / m_{\text{rp}}$ , the per-chunk events hold simultaneously for *every* column  $j \in [r]$ . On this event, taking the maximum over  $j \in [r]$  and using  $\alpha_{\text{rp}}, \beta_{\text{rp}} > 0$  gives

$$\|\mathbf{V}\|_2 = \max_{j \in [r]} \|\mathbf{v}_j\|_2 \in \left[ \alpha_{\text{rp}} \max_{j \in [r]} \|\mathbf{w}_j\|_2, \beta_{\text{rp}} \max_{j \in [r]} \|\mathbf{w}_j\|_2 \right] = [\alpha_{\text{rp}} \|\mathbf{W}\|_2, \beta_{\text{rp}} \|\mathbf{W}\|_2],$$

hence  $\|\mathbf{V}\|_2 / \|\mathbf{W}\|_2 \in [\alpha_{\text{rp}}, \beta_{\text{rp}}]$ , which proves the first claim.

We now establish the second claim. Suppose  $0 < \theta \leq q/b$  and  $\|\mathbf{W}\|_2 \geq \theta$ . Choose a column  $j^* \in [r]$  such that  $\|\mathbf{w}_{j^*}\|_2 = \|\mathbf{W}\|_2$ . This column is fixed before  $\mathbf{J}$  is sampled. Split  $\mathbf{w}_{j^*}$  into chunks  $\mathbf{w}_{j^*,1}, \dots, \mathbf{w}_{j^*,m_u/m_{\text{rp}}}$  of height  $m_{\text{rp}}$ . Since  $\mathbf{W} \in [\pm q/2]^{m_u \times r}$ , every chunk also lies in  $[\pm q/2]^{m_{\text{rp}}}$ .

For every chunk  $\mathbf{w}_{j^*,i}$ , set

$$\theta_i := \theta \cdot \frac{\|\mathbf{w}_{j^*,i}\|_2}{\|\mathbf{w}_{j^*}\|_2}.$$

For every nonzero chunk,  $\theta_i > 0$ ,  $\theta_i \leq \theta \leq q/b$ , and  $\|\mathbf{w}_{j^*,i}\|_2 \geq \theta_i$ . Hence, by the second part of Lemma 5,

$$\Pr[\|\mathbf{J}\mathbf{w}_{j^*,i} \bmod q\|_2 \leq \alpha_{\text{rp}}\theta_i] \leq \kappa_{\text{rp}}.$$

A union bound over the at most  $m_{\mathbf{w}}/m_{\text{rp}}$  nonzero chunks shows that, except with probability at most  $\kappa_{\text{rp}} \cdot m_{\mathbf{w}}/m_{\text{rp}}$ , none of these bad events occurs, i.e.,

$$\|\mathbf{J}\mathbf{w}_{j^*,i} \bmod q\|_2 > \alpha_{\text{rp}}\theta_i$$

for every nonzero chunk. On this event, the  $j^*$ -th column of  $\mathbf{V} \bmod q$  satisfies

$$\begin{aligned} \|\mathbf{v}_{j^*} \bmod q\|_2^2 &= \sum_{i \in [m_{\mathbf{w}}/m_{\text{rp}}]} \|\mathbf{J}\mathbf{w}_{j^*,i} \bmod q\|_2^2 \\ &> \alpha_{\text{rp}}^2 \sum_{i \in [m_{\mathbf{w}}/m_{\text{rp}}]} \theta_i^2 = \alpha_{\text{rp}}^2 \theta^2 \frac{\sum_{i \in [m_{\mathbf{w}}/m_{\text{rp}}]} \|\mathbf{w}_{j^*,i}\|_2^2}{\|\mathbf{w}_{j^*}\|_2^2} = \alpha_{\text{rp}}^2 \theta^2. \end{aligned}$$

Since the matrix norm is the maximum column norm,  $\|\mathbf{V} \bmod q\|_2 \geq \|\mathbf{v}_{j^*} \bmod q\|_2 > \alpha_{\text{rp}}\theta$  on the same event. Therefore,

$$\Pr[\|\mathbf{V} \bmod q\|_2 \leq \alpha_{\text{rp}}\theta] \leq \kappa_{\text{rp}} \cdot m_{\mathbf{w}}/m_{\text{rp}},$$

which proves the second claim.  $\square$

From this point onwards, we fix parameters  $n_{\text{rp}}, m_{\text{rp}}, \alpha_{\text{rp}}, \beta_{\text{rp}}, b$ , as in Lemma 5.

## 5.2 Committed Coarse Random Projection

The committed coarse random projection RoK  $\Pi_{\ell, m_{\mathbf{x}}, \beta_{\mathbf{x}}}^{\text{proj-c}}$  is given in Fig. 2. On a technical level, it compresses the witness  $\mathbf{W}$  into a shorter witness with closely related norm by using a random projection matrix  $\mathbf{J}$ . To keep the specification of the projection succinct, a structured block-diagonal  $\mathbf{I} \otimes \mathbf{J}$  matrix is used. The projection  $\mathbf{V}$  has dimension  $m_{\mathbf{w}} \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}}$ . After gadget decomposition as  $\mathbf{Y}_{k_{\text{lin}}}$ , it is committed within  $\text{com}_{k_{\text{lin}}}$ . The relation is updated with  $\text{com}_{k_{\text{lin}}}$  and consistency checks that  $\mathbf{Y}_{k_{\text{lin}}}$  does contain the random projection of  $\mathbf{W}$ . In terms of relations,  $\Pi^{\text{proj-c}}$  is therefore a self-reduction  $\Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$  that *appends a single committed-linear block*. Concretely, an instance with  $k_{\text{lin}}$  blocks is mapped to one with  $k_{\text{lin}} + 1$  blocks (Lemma 7): the new block  $\text{par}_{\text{lin}, k_{\text{lin}}}$  carries  $\text{com}_{k_{\text{lin}}}$  and its consistency constraints, while the original  $k_{\text{lin}}$  blocks are passed through unchanged. The bounded statement growth introduced by projection is reversed by the subsequent folding step  $\Pi^{\text{fold-split}}$  (Section 6).

The key feature of  $\Pi^{\text{proj-c}}$  is that it asserts strict norm-bounds on  $\mathbf{W}$ : The  $\mathbf{Y}_i$  in  $\Xi_{\text{COM}}^{\text{lin}}$  have small norm and no slack (i.e.  $\varrho = 1$ ) by definition of  $\Xi_{\text{COM}}^{\text{lin}}$ . Thus, by properties of random projections, it follows that  $\mathbf{W}$  must have no slack and its norm relative to  $\mathbf{Y}$  is controlled by  $\frac{n_{\text{rp}}}{m_{\text{rp}}}$ .

**Lemma 7.** *Let  $\ell$  be a gadget decomposition parameter and  $\text{par}_{\text{com}}$  be parameters for COM so that  $\beta_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}_{\text{com}}]$ . Let  $m_{\text{rp}} | m_{\mathbf{w}}$ . The RoK  $\Pi_{\ell, \text{par}_{\text{com}}}^{\text{proj-c}}$  (Fig. 2) satisfies the following properties:*

| $P(\text{stmt}, \text{wit})$                                                                                                                                                                                                     | $V(\text{stmt})$                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| $\text{parse stmt} = ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}, \mathbf{b})$                                                   |                                                                   |
| $\text{parse wit} = (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{\text{lin}}]})$                                                                                                                                         |                                                                   |
| $\parallel (\text{stmt}, \text{wit}) \in \Xi_{\text{COM}}^{\text{lin}} [k_{\text{lin}}, k_{\text{lr}}, n, m_w, r, \beta_w, (n_i, \text{par}_{\text{com}, i}, m_{y,i}, \beta_{x,i}, \beta_{y,i})_{i \in [k_{\text{lin}}]}]$       |                                                                   |
| $\mathbf{V} := (\mathbf{I}_{m_w/m_{\text{rp}}} \otimes \mathbf{J}) \cdot \mathbf{W} \in \mathcal{R}^{m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                    | $\mathbf{J} \leftarrow \chi^{n_{\text{rp}} \times m_{\text{rp}}}$ |
| $\mathbf{Y}_{k_{\text{lin}}} := \mathbf{G}_\ell^{-1}(\mathbf{V}) \in \mathcal{R}^{\ell \cdot m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                            |                                                                   |
| $(\text{com}_{k_{\text{lin}}}, \mathbf{x}_{k_{\text{lin}}}) := \text{Com}_{\text{par}_{\text{com}}}(\text{ck}, \text{vec}(\mathbf{Y}_{k_{\text{lin}}}))$                                                                         | $\xrightarrow{\text{com}_{k_{\text{lin}}}}$                       |
| $\text{wit}' := (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{\text{lin}}+1]})$                                                                                                                                           |                                                                   |
| $\parallel (\mathbf{I}_{m_w/m_{\text{rp}}} \otimes \mathbf{J}) \cdot \mathbf{W} = \mathbf{G}_\ell \cdot \mathbf{Y}_{k_{\text{lin}}} \pmod q$                                                                                     |                                                                   |
| $\mathbf{F}_{k_{\text{lin}}} := \mathbf{I}_{m_w/m_{\text{rp}}} \otimes \mathbf{J}, \mathbf{H}_{k_{\text{lin}}} := \mathbf{G}_\ell$                                                                                               |                                                                   |
| $n_{k_{\text{lin}}} := m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}}, \text{par}_{\text{com}, k_{\text{lin}}} = \text{par}_{\text{com}}, m_{y, k_{\text{lin}}} := \ell \cdot n_{k_{\text{lin}}}$                                 |                                                                   |
| $\beta_{x, k_{\text{lin}}} := \beta_x, \beta_{y, k_{\text{lin}}} := \text{dcmp}_\ell(\beta_{\text{rp}} \cdot \beta_w)$                                                                                                           |                                                                   |
| $\text{stmt}' := ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}+1]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}, \mathbf{b})$                                                     |                                                                   |
| $\parallel (\text{stmt}', \text{wit}') \in \Xi_{\text{COM}}^{\text{lin}} [k_{\text{lin}}+1, k_{\text{lr}}, n, m_w, r, \beta_w, (n_i, \text{par}_{\text{com}, i}, m_{y,i}, \beta_{x,i}, \beta_{y,i})_{i \in [k_{\text{lin}}+1]}]$ |                                                                   |
| $\text{return } (\text{stmt}', \text{wit}')$                                                                                                                                                                                     | $\text{return stmt}'$                                             |

**Fig. 2.**  $\Pi_{\ell, \text{par}_{\text{com}}}^{\text{proj-c}}$  : The commit-project self-RoK for  $\Xi_{\text{COM}}^{\text{lin}}$ .

- It is  $\epsilon$ -complete, with  $\epsilon = \kappa_{\text{rp}} \cdot \varphi r m_w / m_{\text{rp}}$ , for  $\Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$  with parameter change  $k_{\text{lin}} \mapsto k_{\text{lin}} + 1$  where

$$\text{par}_{\text{lin}, k_{\text{lin}}} = \left( m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}}, \text{par}_{\text{com}}, \ell \cdot m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}}, \beta_x, \text{dcmp}_\ell(\beta_{\text{rp}} \cdot \beta_w) \right).$$

- It is  $\kappa$ -knowledge-sound, with  $\kappa = \kappa_{\text{rp}} \cdot \varphi m_w / m_{\text{rp}}$ , for

$$\begin{aligned} & \Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}] \vee \Xi_{\text{COM}}^{\text{lin}} [k_{\text{lin}}, k_{\text{lr}}, n, m_w, r, \text{cmp}(\beta'_{y, k_{\text{lin}}}) / \alpha_{\text{rp}}, (n_i, \text{par}_{\text{com}, i}, m_{y,i}, \beta'_{x,i}, \beta'_{y,i})_{i \in [k_{\text{lin}}]}] \\ & \leftarrow \Xi_{\text{COM}}^{\text{lin-rel}} [k_{\text{lin}} + 1, k_{\text{lr}}, n, m_w, r, \beta'_w, (n_i, \text{par}_{\text{com}, i}, m_{y,i}, \beta'_{x,i}, \beta'_{y,i})_{i \in [k_{\text{lin}}+1]}, \varrho'], \end{aligned}$$

where  $\text{cmp}(\beta'_{y, k_{\text{lin}}}) / \alpha_{\text{rp}} < q/b$  and  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{\text{lin}}}, m_{y, k_{\text{lin}}}, \beta'_{y, k_{\text{lin}}}, \beta'_{x, k_{\text{lin}}}] \cup \{(n_0, m_w, 2\beta'_w \varrho')\}$

- It has the following efficiency measures:

- Prover runtime:  $O(m_w r n_{\text{rp}})$   $\mathcal{R}_q$  operations and  $\text{com}_{k_{\text{lin}}}$  computation,
- Verifier runtime:  $O(n_{\text{rp}} m_{\text{rp}})$  samplings from  $\chi$ ,
- Prover-to-verifier communication:  $|\text{com}_{k_{\text{lin}}}|$ .

*Proof.*

Completeness:

Let  $\mathbf{W}$  be the witness held by an honest prover and  $\beta_w$  be the norm bound from the parameters. Then, by Lemma 6 (applied on each coefficient), except with probability at most  $\epsilon := \kappa_{\text{rp}} \cdot \varphi r m_w / m_{\text{rp}}$ , we have that  $\|\mathbf{V}\|_2 \leq \beta_{\text{rp}} \cdot \beta_w$ . Therefore, the norm of the committed witness  $\mathbf{Y}_{k_{\text{lin}}}$  is at most  $\text{dcmp}(\beta_{\text{rp}} \cdot \beta_w)$ .

Knowledge-soundness:

Let  $\mathcal{A}$  be a prover so that the verifier accepts with probability at least  $\epsilon$ . We construct an extractor

Ext that extracts a witness  $\mathbf{W}^*$  from  $\mathcal{A}$ . The extractor runs the prover  $\mathcal{A}$  to obtain witness-statement pair  $(\text{stmt}', \text{wit}')$ . If the verifier rejects, the extractor aborts. If the verifier accepts, the extractor checks if  $\mathbf{W}'$  from the witness  $(\mathbf{W}', \mathbf{s}')$  satisfies the input constraints and outputs the witness if so. If not, the extractor rewinds  $\mathcal{A}$  and repeats the process to obtain a second witness-statement pair  $(\text{stmt}'', \text{wit}'')$ .

Now, let  $(\mathbf{W}', (\mathbf{Y}'_i, \mathbf{x}'_i)_{i \in [k_{\text{lin}}+1]}, \mathbf{s}') := \text{wit}'$ , and likewise for  $\text{wit}''$ . Here the extracted witness carries  $k_{\text{lin}} + 1$  blocks, whereas the output relation  $\Xi_{\text{COM}}^{\text{lin}}$  has only  $k_{\text{lin}}$ : the first  $k_{\text{lin}}$  blocks  $(\mathbf{Y}'_i, \mathbf{x}'_i)_{i \in [k_{\text{lin}}]}$  carry over to the output unchanged, while block  $k_{\text{lin}}$  (the projection block introduced by this RoK) is dropped on output. If it holds that  $(\mathbf{W}', \mathbf{s}') \neq (\mathbf{W}'', \mathbf{s}'')$ , then the extractor outputs a non-zero column  $\mathbf{W}'\mathbf{s}'' - \mathbf{W}''\mathbf{s}'$  which satisfies  $\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}]$  branch for extraction, where  $\text{par}_{\text{sis}} \ni (n_0, m_{\mathbf{w}}, 2\beta'_{\mathbf{w}}\rho')$ . If it holds  $\mathbf{Y}'_{k_{\text{lin}}} \neq \mathbf{Y}''_{k_{\text{lin}}}$ , extractor outputs  $(\text{vec}(\mathbf{Y}'_{k_{\text{lin}}}), \mathbf{x}'_{k_{\text{lin}}}), (\text{vec}(\mathbf{Y}''_{k_{\text{lin}}}), \mathbf{x}''_{k_{\text{lin}}})$  which satisfies  $\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for the extraction, where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{\text{lin}}}, m_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{x}, k_{\text{lin}}}]$ , which is a valid extraction due to Lemma 4.

Let us now assume that  $(\mathbf{W}', \mathbf{s}') = (\mathbf{W}'', \mathbf{s}'') =: (\mathbf{W}^*, \mathbf{s}^*)$ . Let  $\mathbf{V}^* := \mathbf{G}\mathbf{Y}^*_{k_{\text{lin}}}$  and  $\mathbf{J}$  be the random matrix sampled by  $\mathcal{A}$  in the second execution.

Since  $\mathbf{J}$  is now independent from  $\mathbf{W}^*$ , by Lemma 6 (exploiting that  $\text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}} < q/b$ ), it holds that

$$\|\mathbf{W}^*\|_2 \leq \|\mathbf{V}^*\|_2/\alpha_{\text{rp}} \leq \text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}},$$

unless with the failure probability  $\kappa_1 := \kappa_{\text{rp}} \cdot \varphi m_{\mathbf{w}}/m_{\text{rp}}$  by Lemma 5 and the union bound, which concludes the extractability proof as the extractor outputs  $\mathbf{W}^*$  for relation  $\Xi_{\text{COM}}^{\text{lin}}$ .

Extractor runtime:

To find an accepting transcript with probability at least  $\epsilon$ , the extractor needs to rewind  $\mathcal{A}$  at most  $1/\epsilon$  times in expectation. Since the extractor only rewinds if the initial uniform challenge was successful, and the challenges are sampled uniformly at random, it follows by a standard argument that  $\epsilon \cdot \frac{1}{\epsilon}$  rewinds are required in expectation. Hence, the extractor is expected polynomial-time.

Efficiency:

For the prover, the cost is dominated by computing  $\mathbf{V}$ . The cost of computing  $\mathbf{V}$  is  $O(m_{\mathbf{w}}rn_{\text{rp}})$  ring operations. Further, the prover needs to compute one commitment for  $\mathbf{Y}_{k_{\text{lin}}}$ . For the verifier, the cost is dominated by sampling  $\mathbf{J}$ , which costs  $O(n_{\text{rp}}m_{\text{rp}})$  samplings from  $\chi$ . The communication cost is dominated by sending the commitment  $\text{com}_{k_{\text{lin}}}$ .  $\square$

### 5.3 Committed Fine Random Projection

The committed fine random projection RoK  $\Pi_{n_{\text{bat}}, \ell, \ell', \text{par}_{\text{com}}, \text{par}'_{\text{com}}}^{\text{proj-f}}$  is given in Fig. 3. For the projection to compress, we need  $m_{\text{rp}} > n_{\text{rp}}$ , which fails once  $m_{\text{rp}}$  drops to  $n_{\text{rp}}$ . However, there are two dimensions in consideration: The dimension  $m_{\mathbf{w}} = \dim_{\mathcal{R}}(\mathbf{w})$  of  $\mathbf{w}$  as a vector of  $\mathcal{R}$ -elements (which is how  $\Pi^{\text{proj-c}}$  operates), and the dimension  $\dim_{\mathbb{Z}}(\mathbf{w}) = \varphi \cdot m_{\mathbf{w}}$  as a vector of  $\mathbb{Z}$ -elements. In the latter case, we can compress as long as  $\varphi m_{\text{rp}} > n_{\text{rp}}$ , which is what  $\Pi^{\text{proj-f}}$  does.

To realise the above, the random projection  $\mathbf{J}$  and  $\mathbf{W}$  are multiplied on the coefficient level, i.e., we use  $\text{trace}(\mathbf{V}) = \mathbf{J} \cdot \text{cf}(\mathbf{W}) = \text{trace}(\text{cf}_{\mathbb{V}}(\mathbf{J}) \cdot \mathbf{W})$ . This is more sophisticated and expensive than in  $\Pi^{\text{proj-c}}$ , hence we switch from coarse  $\Pi^{\text{proj-c}}$  to fine  $\Pi^{\text{proj-f}}$  as late as possible.<sup>11</sup>

<sup>11</sup> Lifting  $\mathbf{J}$  to  $\text{cf}_{\mathbb{V}}(\mathbf{J})$  and using it is computationally more expensive than mere ring element addition and subtraction required to compute  $\mathbf{J}\mathbf{W}$  in  $\Pi^{\text{proj-c}}$ .

On a technical level,  $\Pi^{\text{proj-f}}$  projects over the coefficient embedding, using the trace and the trace-dual basis. The projected witness  $\mathbf{V}_{\text{tr}}$  (over  $\mathbb{Z}$ ) is re-embedded into  $\mathcal{R}$ , and committed as  $\text{com}_{k_{\text{lin}}}$ . Note that constraints in  $\Xi_{\text{COM}}^{\text{lin}}$  are over  $\mathcal{R}$  (not  $\mathbb{Z}$ ). Therefore, we cannot directly express the consistency of  $\mathbf{V}_{\text{tr}}$  via constraints. Instead, constraints are converted to equations over  $\mathcal{R}$ , where the verifier checks the trace. Moreover, to check the consistency of the witness and its projection, the prover batches the linear constraints using random row-tensor matrices  $\mathbf{Z}^{(0)}, \mathbf{Z}^{(1)}$  and  $\mathbf{Z}^{(2)}$  (sampled by the verifier) over  $\mathbb{Z}_q$ , by embedding the batching challenges via trace-dual embedding. Due to batching, the vector  $\mathbf{r}$  (sent in the last move) consists of only  $n_{\text{bat}}$  elements. Lastly, the batch-verify equations are encoded into an “ $\mathbf{A} \cdot \text{vec}(\mathbf{Y}) = \mathbf{b}$ ” and two “ $\mathbf{F}_i \mathbf{W} = \mathbf{Y}_i$ ” constraints. In terms of relations,  $\Pi^{\text{proj-f}}$  is likewise a self-reduction  $\Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$ , but it grows the statement by *two* committed-linear blocks and  $n_{\text{bat}}$  extra global linear constraints ( $k_{\text{lin}} \mapsto k_{\text{lin}} + 2$ ,  $n \mapsto n + n_{\text{bat}}$ ; Lemma 8): one new block carries  $\text{com}_{k_{\text{lin}}}$  (a commitment to the re-embedded projection), the other carries  $\text{com}_{k_{\text{lin}}+1}$  (absorbing the batched trace-consistency checks).

**Lemma 8.** *Let  $n_{\text{bat}} \in \mathbb{N}$  be a batching parameter,  $\ell, \ell' \in \mathbb{N}$  be gadget decomposition parameters and  $\text{par}_{\text{com}}, \text{par}'_{\text{com}}$  be parameters for COM so that  $\beta_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}_{\text{com}}]$  and  $\beta'_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}'_{\text{com}}]$ . Let  $m_{\text{rp}} | \varphi m_{\mathbf{w}}$ . The RoK  $\Pi^{\text{proj-f}}_{n_{\text{bat}}, \ell, \ell', \text{par}_{\text{com}}, \text{par}'_{\text{com}}}$  (Fig. 3) satisfies the properties:*

- It is  $\epsilon$ -complete, with  $\epsilon = (\varphi m_{\mathbf{w}}/m_{\text{rp}})\kappa_{\text{rp}}$ , for  $\Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$  with parameter changes  $k_{\text{lin}} \mapsto k_{\text{lin}} + 2$ ,  $n \mapsto n + n_{\text{bat}}$ , where

$$\begin{aligned} \text{par}_{\text{lin}, k_{\text{lin}}} &:= \left( 0, \text{par}_{\text{com}}, \ell m_{\mathbf{w}} \frac{n_{\text{rp}}}{m_{\text{rp}}}, \beta_{\mathbf{x}}, \text{dcmp}_{\ell}(\beta_{\text{rp}} \cdot \beta_{\mathbf{w}}) \right), \\ \text{par}_{\text{lin}, k_{\text{lin}}+1} &:= (n_{\text{bat}}, \text{par}'_{\text{com}}, \ell' \cdot n_{\text{bat}}, \beta'_{\mathbf{x}}, \text{dcmp}_{\ell'}(\sqrt{\varphi n_{\text{bat}}} q/2)). \end{aligned}$$

- It is knowledge-sound with  $\kappa = ((\log(\varphi m_{\mathbf{w}}/m_{\text{rp}}))/q)^{n_{\text{bat}}} + (\varphi m_{\mathbf{w}}/m_{\text{rp}})\kappa_{\text{rp}}$ , for

$$\begin{aligned} &\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}] \vee \Xi_{\text{COM}}^{\text{lin}}[k_{\text{lin}}, k_{\text{lr}}, n, m_{\mathbf{w}}, r, \text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}}, (n_i, \text{par}_{\text{com}, i}, m_{\mathbf{y}, i}, \beta'_{\mathbf{x}, i}, \beta'_{\mathbf{y}, i})_{i \in [k_{\text{lin}}]}] \\ &\leftarrow \Xi_{\text{COM}}^{\text{lin-rel}}[k_{\text{lin}} + 2, k_{\text{lr}}, n + n_{\text{bat}}, m_{\mathbf{w}}, r, \beta'_{\mathbf{w}}, (n_i, \text{par}_{\text{com}, i}, m_{\mathbf{y}, i}, \beta'_{\mathbf{x}, i}, \beta'_{\mathbf{y}, i})_{i \in [k_{\text{lin}}+2]}, \varrho'] . \end{aligned}$$

where  $\text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}} < q/b$  and

$$\begin{aligned} \text{par}_{\text{sis}} &\supseteq \{(n_0, m_{\mathbf{w}}, 2\beta'_{\mathbf{w}}\varrho')\} \cup \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{\text{lin}}}, m_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{x}, k_{\text{lin}}}] \\ &\cup \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{\text{lin}}+1}, m_{\mathbf{y}, k_{\text{lin}}+1}, \beta'_{\mathbf{y}, k_{\text{lin}}+1}, \beta'_{\mathbf{x}, k_{\text{lin}}+1}] \end{aligned}$$

- It has the following efficiency measures:

- Prover runtime:  $O(m_{\mathbf{w}} r n_{\text{rp}} \varphi)$   $\mathbb{Z}$  operations,  $O(n_{\text{bat}} m_{\mathbf{w}} r)$   $\mathcal{R}_q$  operations and the cost of computing  $\text{com}_{k_{\text{lin}}}$  and  $\text{com}_{k_{\text{lin}}+1}$ .
- Verifier runtime:  $O(n_{\text{rp}} m_{\text{rp}})$  samplings from  $\chi$ ,  $O(n_{\text{bat}} \log(\varphi m_{\mathbf{w}}/m_{\text{rp}}))$  samplings from  $\mathbb{Z}_q$  and  $O(n_{\text{bat}})$   $\mathcal{R}_q$  operations.
- Prover-to-verifier comm.:  $|\text{com}_{k_{\text{lin}}}| + |\text{com}_{k_{\text{lin}}+1}| + O(n_{\text{bat}} \cdot \log |\mathcal{R}_q|)$ .

*Proof.*

Completeness:

For an honest prover holding witness  $\mathbf{W}$  with norm at most  $\beta_{\mathbf{w}}$ , by Lemma 6, except with probability at most  $(\varphi m_{\mathbf{w}}/m_{\text{rp}})\kappa_{\text{rp}}$ , we have that  $\|\mathbf{V}_{\text{emb}}\|_2 \leq \beta_{\text{rp}} \cdot \beta_{\mathbf{w}}$ . Therefore, the norm of the committed witness  $\mathbf{Y}_{k_{\text{lin}}}$  is at most  $\text{dcmp}(\beta_{\text{rp}} \cdot \beta_{\mathbf{w}})$ .

| $\mathcal{P}(\text{stmt}, \text{wit})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | $\mathcal{V}(\text{stmt})$                                                                               |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| $\text{parse stmt} = ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}, \mathbf{b})$<br>$\text{parse wit} = (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{\text{lin}}]})$<br>$\parallel (\text{stmt}, \text{wit}) \in \Xi_{\text{COM}}^{\text{lin}} [k_{\text{lin}}, k_{\text{lr}}, n, m_w, r, \beta_w, (n_i, \text{par}_{\text{com}, i}, m_{y, i}, \beta_{x, i}, \beta_{y, i})_{i \in [k_{\text{lin}}]}]$ |                                                                                                          |
| $\mathbf{V} := (\mathbf{I}_{\varphi m_w / m_{\text{rp}}} \otimes \text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{J}^T)^T) \cdot \mathbf{W} \in \mathcal{R}^{\varphi m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                                                                                                                                                                                                                                          | $\mathbf{J} \leftarrow \chi^{n_{\text{rp}} \times m_{\text{rp}}}$                                        |
| $\mathbf{V}_{\text{tr}} := \text{trace}(\mathbf{V}) \in \mathbb{Z}^{\varphi m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                          |
| $\mathbf{V}_{\text{emb}} := \text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{V}_{\text{tr}}) \in \mathcal{R}^{m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                                                                                                                                                                                                                                                                                                 |                                                                                                          |
| $\mathbf{Y}_{k_{\text{lin}}} := \mathbf{G}_{\ell}^{-1}(\mathbf{V}_{\text{emb}}) \in \mathcal{R}^{\ell m_w \cdot \frac{n_{\text{rp}}}{m_{\text{rp}}} \times r}$                                                                                                                                                                                                                                                                                                                                              | $\mathbf{Z}^{(0)} \leftarrow \mathbb{Z}_q^{n_{\text{bat}} \times 2^{\otimes \log \varphi}}$              |
| $\mathbf{F}_k = \mathbf{0}, \mathbf{H}_k = \mathbf{0} \parallel \text{ignored}$                                                                                                                                                                                                                                                                                                                                                                                                                             | $\mathbf{Z}^{(1)} \leftarrow \mathbb{Z}_q^{n_{\text{bat}} \times 2^{\otimes \log(m_w / m_{\text{rp}})}}$ |
| $(\text{com}_{k_{\text{lin}}}, \mathbf{x}_{k_{\text{lin}}}) := \text{Com}_{\text{par}_{\text{com}}}(\text{ck}, \text{vec}(\mathbf{Y}_{k_{\text{lin}}}))$                                                                                                                                                                                                                                                                                                                                                    | $\mathbf{Z} := \mathbf{Z}^{(1)} \bullet \mathbf{Z}^{(0)}$                                                |
| $\mathbf{V}_{\text{bat}} := \mathbf{Z} \cdot \mathbf{V} \bmod q \in \mathcal{R}_q^{n_{\text{bat}} \times r}$                                                                                                                                                                                                                                                                                                                                                                                                | $\mathbf{Z}^{(2)} \leftarrow \mathbb{Z}_q^{n_{\text{bat}} \times 2^{\otimes \log r}}$                    |
| $\mathbf{Y}_{k_{\text{lin}}+1} := \mathbf{G}_{\ell'}^{-1}(\mathbf{V}_{\text{bat}}) \in \mathcal{R}^{\ell' \cdot n_{\text{bat}} \times r}$                                                                                                                                                                                                                                                                                                                                                                   |                                                                                                          |
| $(\text{com}_{k_{\text{lin}}+1}, \mathbf{x}_{k_{\text{lin}}+1}) := \text{Com}_{\text{par}'_{\text{com}}}(\text{ck}, \text{vec}(\mathbf{Y}_{k_{\text{lin}}+1}))$                                                                                                                                                                                                                                                                                                                                             |                                                                                                          |
| $\mathbf{F}_{k_{\text{lin}}+1} = \mathbf{Z}(\mathbf{I}_{\varphi m_w / m_{\text{rp}}} \otimes \text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{J}^T)^T), \mathbf{H}_{k_{\text{lin}}+1} = \mathbf{G}_{\ell'}$                                                                                                                                                                                                                                                                                                            |                                                                                                          |
| $\hat{z}_i^{(0)} := \text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{z}_i^{(0)}) \in \mathcal{R} \quad \forall i \in [n_{\text{bat}}]$                                                                                                                                                                                                                                                                                                                                                                                 | $\forall i \in [n_{\text{bat}}]:$                                                                        |
| $\mathbf{r} := (\mathbf{V}_{\text{bat}} \mathbf{z}_i^{(2)} - \mathbf{z}_i^{(1)T} \hat{z}_i^{(0)} \mathbf{V}_{\text{emb}} \mathbf{z}_i^{(2)})_{i \in [n_{\text{bat}}]}$                                                                                                                                                                                                                                                                                                                                      | $\text{trace}(r_i) \stackrel{?}{=} 0$                                                                    |
| $\mathbf{A}' := \text{diag}(\mathbf{A}, (\mathbf{z}_i^{(2)} \otimes \mathbf{z}_i \otimes \mathbf{g} \parallel \mathbf{z}_i^{(2)} \otimes \mathbf{z}_i^{(1)} \otimes \hat{z}_i^{(0)} \otimes \mathbf{g})_{i \in [n_{\text{bat}}]}^T)$                                                                                                                                                                                                                                                                        |                                                                                                          |
| $\mathbf{b}' := [\mathbf{b} \parallel \mathbf{r}]$                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                          |
| $\text{wit}' := (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{\text{lin}}+2]})$                                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                          |
| $(n_{k_{\text{lin}}}, \text{par}_{\text{com}, k_{\text{lin}}}, m_{y, k_{\text{lin}}}, \beta_{x, k_{\text{lin}}}, \beta_{y, k_{\text{lin}}}) := (0, \text{par}_{\text{com}}, \ell m \frac{n_{\text{rp}}}{m_{\text{rp}}}, \beta_x[\text{par}_{\text{com}}], \text{dcmp}_{\ell}(\beta_{\text{rp}} \cdot \beta_w))$                                                                                                                                                                                             |                                                                                                          |
| $(n_{k_{\text{lin}}+1}, \text{par}_{\text{com}, k_{\text{lin}}+1}, m_{y, k_{\text{lin}}+1}, \beta_{x, k_{\text{lin}}+1}, \beta_{y, k_{\text{lin}}+1}) := (n_{\text{bat}}, \text{par}'_{\text{com}}, \ell' \cdot n_{\text{bat}}, \beta_x[\text{par}'_{\text{com}}], \text{dcmp}_{\ell'}(\sqrt{\varphi n_{\text{bat}}} q / 2))$                                                                                                                                                                               |                                                                                                          |
| $\text{stmt}' := ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}+2]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}', \mathbf{b}')$                                                                                                                                                                                                                                                                                                                              |                                                                                                          |
| $\parallel (\text{stmt}', \text{wit}') \in \Xi_{\text{COM}}^{\text{lin}} [k_{\text{lin}} + 2, k_{\text{lr}}, n + n_{\text{bat}}, m_w, r, \beta_w, (n_i, \text{par}_{\text{com}, i}, m_{y, i}, \beta_{x, i}, \beta_{y, i})_{i \in [k_{\text{lin}}+2]}]$                                                                                                                                                                                                                                                      |                                                                                                          |
| $\text{return}(\text{stmt}', \text{wit}')$                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | $\text{return stmt}'$                                                                                    |

**Fig. 3.**  $\Pi_{n_{\text{bat}}, \ell, \ell', \text{par}_{\text{com}}, \text{par}'_{\text{com}}}^{\text{proj-f}}$ : The commit-project self-RoK for  $\Xi_{\text{COM}}^{\text{lin}}$ .

For the batch constraints, we have  $\mathbf{V}_{\text{bat}}$  with norm at most  $\sqrt{\varphi n_{\text{bat}}} q / 2$ . Therefore, the committed witness  $\mathbf{Y}_{k_{\text{lin}}+1}$  has norm at most  $\text{dcmp}(\sqrt{\varphi n_{\text{bat}}} q / 2)$ .

To argue that the batch constraints are satisfied, we first note that:

$$\text{trace}(\text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{J}^T)^T \mathbf{w}_{i,j}) = \mathbf{J} \text{cf}(\mathbf{w}_{i,j}) \quad \forall (i, j) \in [m_w / m_{\text{rp}}, r],$$

For each  $i \in [n_{\text{bat}}]$ , we observe that the following holds:

$$\text{trace}(\mathbf{z}_i^{(1)T} \otimes \mathbf{z}_i^{0T} \text{cf}_{\sqrt{\cdot}}^{-1}(\mathbf{J}^T)^T \cdot \mathbf{W} \mathbf{z}_i^{(2)}) = \mathbf{z}_i^{(1)T} \otimes \mathbf{z}_i^{0T} \mathbf{V}_{\text{tr}} \mathbf{z}_i^{(2)},$$

which is immediate since  $\text{trace}$  is  $\mathbb{Z}_q$  linear. Since  $\mathbf{z}_i^{0T} \mathbf{V}_{\text{tr}} = \text{trace}(\hat{z}_i^{(0)} \mathbf{V}_{\text{emb}})$ , it also holds that:

$$\text{trace}(\mathbf{z}_i^{(1)T} \otimes \mathbf{z}_i^{0T} \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T \cdot \mathbf{W} \mathbf{z}_i^{(2)}) = \text{trace}(\mathbf{z}_i^{(1)T} \otimes \hat{z}_i^{(0)} \mathbf{V}_{\text{emb}} \mathbf{z}_i^{(2)}),$$

which concludes that  $\text{trace}(r_i) = 0$  and also certifies the correctness of the new  $\mathbf{A}'$  constraints.

Knowledge-soundness:

The analysis of the extractor is conveniently split into two steps.

1. First, we consider the prover  $\mathcal{A}_{\text{sz}}$  for the sub-protocol that receives  $\mathbf{Z}$  and  $\mathbf{Z}^{(2)}$  from the verifier and sends  $\text{com}_{k_{11n}+1}$  and  $\mathbf{r}$ . We construct an extractor  $\text{Ext}_{\text{sz}}$  for the input intermediate relation,  $\Xi_{\text{COM}}^{\text{lin}}$  with an additional constraint that  $\mathbf{V}_{\text{tr}} = \text{trace}(\mathbf{I}_{\varphi m_w/m_{\text{rp}}} \otimes \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T \cdot \mathbf{W})$ . First, the extractor executes  $\mathcal{A}_{\text{sz}}$  to obtain an accepting transcript. If the verifier rejects, the extractor aborts. If the verifier accepts, the extractor checks if the witness  $\mathbf{W}'$  from the witness  $(\mathbf{W}', \mathbf{s}')$  satisfies the input constraints and the additional trace constraint. If so, the extractor outputs the witness. If not, the extractor rewinds  $\mathcal{A}_{\text{sz}}$  and repeats the process to obtain a second accepting transcript. Now, let  $(\mathbf{W}', \mathbf{s}')$  and  $(\mathbf{W}'', \mathbf{s}'')$  be the witnesses extracted from the two transcripts, respectively. If it holds that  $(\mathbf{W}', \mathbf{s}') \neq (\mathbf{W}'', \mathbf{s}'')$ , then the extractor outputs a non-zero column  $\mathbf{W}' \mathbf{s}'' - \mathbf{W}'' \mathbf{s}'$  which satisfies  $\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}]$  branch for extraction, where  $\text{par}_{\text{sis}} \ni (n_0, m_w, 2\beta'_w \varrho')$ . If it holds that  $\mathbf{Y}'_{k_{11n}+1} \neq \mathbf{Y}''_{k_{11n}+1}$ , then the extractor outputs  $(\text{vec}(\mathbf{Y}'_{k_{11n}+1}), \mathbf{x}'_{k_{11n}+1}), (\text{vec}(\mathbf{Y}''_{k_{11n}+1}), \mathbf{x}''_{k_{11n}+1})$  which satisfies  $\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for the extraction, where

$$\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{11n}+1}, m_{y, k_{11n}+1}, \beta'_{y, k_{11n}+1}, \beta'_{x, k_{11n}+1}],$$

which is a valid extraction due to Lemma 4. Similarly, if it holds that  $\mathbf{Y}'_{k_{11n}} \neq \mathbf{Y}''_{k_{11n}}$ , then the extractor outputs  $(\text{vec}(\mathbf{Y}'_{k_{11n}}), \mathbf{x}'_{k_{11n}}), (\text{vec}(\mathbf{Y}''_{k_{11n}}), \mathbf{x}''_{k_{11n}})$  which satisfies  $\Xi_{\text{sis}}^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for the extraction, where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{11n}}, m_{y, k_{11n}}, \beta'_{y, k_{11n}}, \beta'_{x, k_{11n}}]$ , which is a valid extraction due to Lemma 4.

Let us now assume that  $(\mathbf{W}', \mathbf{s}') = (\mathbf{W}'', \mathbf{s}'') = (\mathbf{W}^*, \mathbf{s}^*)$ . Let  $\mathbf{V}_{\text{bat}}^* := \mathbf{G} \mathbf{Y}_{k_{11n}+1}^*$ ,  $\mathbf{V}_{\text{emb}}^* := \mathbf{G} \mathbf{Y}_{k_{11n}}^*$ ,  $\mathbf{V}_{\text{tr}}^* := \text{cf}(\mathbf{V}_{\text{emb}}^*)$ , and  $\mathbf{Z}^*, \mathbf{Z}^{*(2)}$  be the random matrices sampled in the second execution. Since,

$$\mathbf{V}_{\text{bat}}^* = \mathbf{Z}(\mathbf{I}_{\varphi m_w/m_{\text{rp}}} \otimes \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T) \cdot \mathbf{W}^* \bmod q,$$

then also by  $\mathbf{A}'$  constraints it holds that for each  $i \in [n_{\text{bat}}]$ :

$$\text{trace}(\mathbf{z}_i^T (\mathbf{I}_{\varphi m_w/m_{\text{rp}}} \otimes \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T) \cdot \mathbf{W}^* \cdot \mathbf{z}_i^{(2)}) = \text{trace}(\mathbf{z}_i^{(1)} \otimes \hat{z}_i^{(0)} \mathbf{V}_{\text{emb}}^* \mathbf{z}_i^{(2)}) \bmod q,$$

which implies that  $\mathbf{V}_{\text{emb}}^* = \text{cf}^{-1} \mathbf{V}_{\text{tr}}^*$  satisfies the constraint:

$$\text{trace}(\mathbf{z}_i^T (\mathbf{I}_{\varphi m_w/m_{\text{rp}}} \otimes \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T) \cdot \mathbf{W}^* \cdot \mathbf{z}_i^{(2)}) = \mathbf{z}_i^T \mathbf{V}_{\text{tr}}^* \mathbf{z}_i^{(2)} \bmod q.$$

Now, since  $\mathbf{Z}^*, \mathbf{Z}^{*(2)}$  are uniformly random row-tensor matrices, and any nonzero constraint violation has a nonzero entry in some  $\mathbb{Z}_q$ -coordinate, by a standard coordinate-wise Schwartz-Zippel argument (i.e. observing that batching with row-tensor vector  $(\mathbf{z}_i^{(2)})^T \otimes \mathbf{z}_i^T$  of dimension  $\varphi m_w/m_{\text{rp}}$  is equivalent to evaluating a multilinear polynomial of total degree  $\log(\varphi m_w/m_{\text{rp}})$ ), except with probability at most

$$\kappa_{1, \text{sz}} := (\log(\varphi m_w/m_{\text{rp}})/q)^{n_{\text{bat}}},$$

it holds that:

$$\text{trace}((\mathbf{I}_{\varphi m_w/m_{\text{rp}}} \otimes \text{cf}_{\sqrt{}}^{-1}(\mathbf{J}^T)^T) \cdot \mathbf{W}^*) = \mathbf{V}_{\text{tr}}^* \bmod q.$$

2. Now, we consider the prover  $\mathcal{A}_{\text{rp}}$  for the sub-protocol that receives  $\mathbf{J}$  from the verifier and sends  $\text{com}_{k_{\text{lin}}}, \mathbf{Z}, \mathbf{Z}^{(2)}$ . We construct an extractor  $\text{Ext}_{\text{rp}}$  for the input relation,  $\Xi_{\text{COM}}^{\text{lin}}$ . Assume that the extractor has access to the (correct) output of the previous extractor  $\text{Ext}_{\text{sz}}$ .

The extractor runs  $\mathcal{A}_{\text{rp}}$  to obtain an accepting transcript. If the verifier rejects, the extractor aborts. If the verifier accepts, the extractor checks if the witness  $\mathbf{W}'$  from the witness  $(\mathbf{W}', \mathbf{s}')$  satisfies the input constraints. If so, the extractor outputs the witness. If not, the extractor rewinds  $\mathcal{A}_{\text{rp}}$  and repeats the process to obtain a second accepting transcript. Now, let  $(\mathbf{W}', \mathbf{s}')$  and  $(\mathbf{W}'', \mathbf{s}'')$  be the witnesses extracted from the two transcripts, respectively. If it holds that  $(\mathbf{W}', \mathbf{s}') \neq (\mathbf{W}'', \mathbf{s}'')$ , then the extractor outputs a non-zero column  $\mathbf{W}'\mathbf{s}'' - \mathbf{W}''\mathbf{s}'$  which satisfies the  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$ , where  $\text{par}_{\text{sis}} \ni (n_0, m_{\mathbf{w}}, 2\beta'_{\mathbf{w}}\varrho')$ . If it holds that  $\mathbf{Y}'_{k_{\text{lin}}} \neq \mathbf{Y}''_{k_{\text{lin}}}$ , then the extractor outputs  $(\text{vec}(\mathbf{Y}'_{k_{\text{lin}}}), \mathbf{x}'_{k_{\text{lin}}}), (\text{vec}(\mathbf{Y}''_{k_{\text{lin}}}), \mathbf{x}''_{k_{\text{lin}}})$  which satisfies  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for extraction, where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, k_{\text{lin}}}, m_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{y}, k_{\text{lin}}}, \beta'_{\mathbf{x}, k_{\text{lin}}}]$ , which is a valid extraction due to Lemma 4. Let us now assume that  $(\mathbf{W}', \mathbf{s}') = (\mathbf{W}'', \mathbf{s}'') =: (\mathbf{W}^*, \mathbf{s}^*)$ . Let  $\mathbf{V}_{\text{emb}}^* := \mathbf{G}\mathbf{Y}_{k_{\text{lin}}}^*$ ,  $\mathbf{V}_{\text{tr}}^* := \text{cf}(\mathbf{V}_{\text{emb}}^*)$ , and  $\mathbf{J}^*$  be the random matrix sampled in the second execution. Since,

$$\text{trace}((\mathbf{I}_{\varphi m_{\mathbf{w}}/m_{\text{rp}}} \otimes \text{cf}_{\mathbf{V}}^{-1}(\mathbf{J}^{*\text{T}})^{\text{T}}) \cdot \mathbf{W}^*) = \mathbf{V}_{\text{tr}}^* \bmod q,$$

we can now apply Lemma 6 (exploiting that  $\text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}} < q/b$ ) to conclude that

$$\|\mathbf{W}^*\|_2 \leq \|\mathbf{V}_{\text{emb}}^*\|_2/\alpha_{\text{rp}} \leq \text{cmp}(\beta'_{\mathbf{y}, k_{\text{lin}}})/\alpha_{\text{rp}},$$

unless with the failure probability  $\kappa_{1, \text{rp}} := (\varphi m_{\mathbf{w}}/m_{\text{rp}}) \cdot \kappa_{\text{rp}}$  by Lemma 6, which concludes the extractability proof as the extractor outputs  $\mathbf{W}^*$  for relation  $\Xi_{\text{COM}}^{\text{lin-rel}}$ .

In total, the failure probability is at most  $\kappa_1 := \kappa_{1, \text{sz}} + \kappa_{1, \text{rp}}$  by the union bound.

Extractor runtime:

The extractor  $\text{Ext}$  runs the two extractors  $\text{Ext}_{\text{sz}}$  and  $\text{Ext}_{\text{rp}}$  sequentially. Each of these extractors runs in expected polynomial-time by a similar argument as in Lemma 7.

Efficiency:

The prover's cost is dominated by computing  $\mathbf{V}_{\text{tr}}$  (no need to compute  $\mathbf{V}$  explicitly), which costs  $O(m_{\mathbf{w}} r n_{\text{rp}} \varphi) \mathbb{Z}$  operations and by computing  $\mathbf{V}_{\text{bat}}$ , which costs  $O(n_{\text{bat}} m_{\mathbf{w}} r) \mathcal{R}_q$  operations. Further, the prover needs to compute two commitments – one for  $\mathbf{Y}_{k_{\text{lin}}}$  and one for  $\mathbf{Y}_{k_{\text{lin}}+1}$ .

For the verifier, the cost is sampling  $\mathbf{J}$ , which is  $O(m_{\text{rp}} n_{\text{rp}})$  samplings from  $\chi$ , sampling  $\mathbf{Z}$  and  $\mathbf{Z}^{(2)}$ , which is  $O(n_{\text{bat}} \log(\varphi r m_{\mathbf{w}}/m_{\text{rp}}))$  samplings from  $\mathbb{Z}_q$  and  $O(n_{\text{bat}})$  ring operations for computing each  $\text{trace}(r_i)$ . The communication cost is dominated by sending the commitments  $\text{com}_{k_{\text{lin}}}, \text{com}_{k_{\text{lin}}+1}$  and  $O(n_{\text{bat}}) |\mathcal{R}_q|$  for sending  $\mathbf{r}$ .  $\square$

**Committed Fine Random Projection with “Lift-and-Batch”** The strategy in Fig. 3 can be further optimised asymptotically, though this barely matters for practical parameters. The technique, called “lift-and-batch”, was introduced in [KLNO25] and exploits favourable ring structure to gradually lift multiple (partial) trace-equalities to ring equalities by going through intermediate subrings. For suitable parameter choices and rings, this yields a protocol  $\Pi^{\text{proj-fl}}$  with prover communication  $|\text{com}_{k_{\text{lin}}}| + |\text{com}_{k_{\text{lin}}+1}| + O(\log |\mathcal{R}_q|)$ . The changes are similar to [KLNO25], and we provide a protocol description in Appendix B.

## 6 Committed Folding

In this section, we present the committed folding RoK  $\Pi^{\text{fold-split}}$  for  $\Xi_{\text{COM}}^{\text{lin}}$ . Starting from a committed-linear instance of width  $r$ , it folds the witness columns with a random challenge  $\mathbf{c} \in \mathcal{C}^r$  and then repacks the folded data into a new witness matrix of width  $r'$ . The output commitment is made to the derived output linear block (via  $\text{vec}(\mathbf{Y})$ ), and all resulting claims are expressed as sumcheck constraints. Thus,  $\Pi^{\text{fold-split}}$  reduces the witness size while preserving the committed-linear structure needed by the next round.

On a technical level, the protocol consists of four main steps. First, the prover sends a commitment to  $\mathbf{T} = \mathbf{F}_{k_{\text{lin}}} \mathbf{W}$ , i.e. the right-constraint block needed to encode all witness constraints as linear relations  $\mathbf{F}_i \mathbf{W} = \mathbf{H}_i \mathbf{Y}_i \bmod q$ . Second, the verifier samples a folding challenge  $\mathbf{c}$ , and the prover computes the folded witness  $\mathbf{W}\mathbf{c}$ , its decomposition  $\tilde{\mathbf{w}}$ , and the packed vector  $\hat{\mathbf{w}}$ . Third, the prover reshapes  $\hat{\mathbf{w}}$  to width  $r'$ , derives the output linear block  $\mathbf{F}\mathbf{U} = \mathbf{G}_{\ell''} \mathbf{Y} \bmod q$ , and commits to  $\text{vec}(\mathbf{Y})$ . All folded consistency constraints are then expressed as a single sumcheck relation by linearity (e.g.,  $\mathbf{F}_i(\mathbf{W}\mathbf{c}) = (\mathbf{F}_i \mathbf{W})\mathbf{c}$ ). Fourth, the prover also computes the self-inner product  $v = \langle \hat{\mathbf{w}}, \hat{\mathbf{w}} \rangle$  of the folded witness  $\hat{\mathbf{w}}$ . This establishes a norm claim and bound (via  $\text{trace}(v) = \|\hat{\mathbf{w}}\|_{\sigma,2}^2$ ). Finally, all claims are turned into sumcheck-based constraints by standard machinery, here denoted as `sumcheckify`. We provide an explicit description of this step in Section 6.1 and a concrete example of one constraint in Section 6.2. Crucially, commitment verification for all  $(\text{com}_i)_{i \in [k_{\text{lin}}]}$  is part of the constraints, so that validity of the resulting claim in  $\Xi_{\text{COM}}^{\text{sum}}$  implies knowledge of valid (short) openings.

This treatment of folding is similar to the one in [BS23, KLNO24b, KLNO25, BC25b]. The extractor, based on coordinate-wise rewinding [FMN24], exploits the linearity of the folding operation.

**Lemma 9.** *Let  $\ell, \ell', \ell'' \in \mathbb{N}$  be gadget decomposition parameters, let  $r, r'$  be powers-of-2 split widths (old and new, respectively), and let  $\text{par}_{\text{com}}, \text{par}'_{\text{com}}$  be parameters for COM so that  $\beta_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}_{\text{com}}]$ ,  $\beta_{\mathbf{y}} = \beta_{\mathbf{y}}[\text{par}_{\text{com}}]$  and  $\beta'_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}'_{\text{com}}]$ . Suppose that  $\tilde{\beta}_{\mathbf{w}}^2 \geq \text{dcmp}_{\ell'}(r\gamma_{\mathcal{C}}\beta_{\mathbf{w}})^2 + \beta_{\mathbf{x}}^2 + \beta_{\mathbf{y}}^2 + \sum_{i \in [k_{\text{lin}}]} \beta_{\mathbf{y},i}^2 + \beta_{\mathbf{x},i}^2$ . Let  $k_i$  be the number of linear constraints necessary to express  $\text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}_i), \text{com}_i, \mathbf{x}_i)$  for each  $i \in [k_{\text{lin}} + 1]$  and let  $k_{\text{sc}} = \sum_{i \in [k_{\text{lin}} + 1]} k_i + \sum_{i \in [k_{\text{lin}} + 1]} n_i + n + k_{1\mathbf{r}} + 1$ . The protocol  $\Pi_{r', \ell, \ell', \ell'', \text{par}_{\text{com}}, \text{par}'_{\text{com}}, \tilde{\beta}_{\mathbf{w}}}^{\text{fold-split}}$  (Fig. 4) satisfies the following properties:*

- It is perfectly complete for

$$\begin{aligned} & \Xi_{\text{COM}}^{\text{lin}}[k_{1\text{in}}, k_{1\mathbf{r}}, n, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, (n_i, \text{par}_{\text{com},i}, m_{\mathbf{y},i}, \beta_{\mathbf{x},i}, \beta_{\mathbf{y},i})_{i \in [k_{1\text{in}}]}] \\ & \rightarrow \Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n, \hat{m}_{\mathbf{w}}/r', r', \text{par}'_{\text{com}}, \ell''n, \tilde{\beta}_{\mathbf{w}}, \beta'_{\mathbf{x}}, \text{dcmp}_{\ell''}(q/2), 2] \end{aligned}$$

where  $\hat{m}_{\mathbf{w}} = \text{minpowtwo}(\ell' m_{\mathbf{w}} + \sum_{i \in [k_{1\text{lin}} + 1]} (m_{\mathbf{y},i} \cdot r + m_{\mathbf{x}}[\text{par}_{\text{com},i}]))$ , where  $\text{par}_{\text{com}, k_{1\text{lin}}} = \text{par}_{\text{com}}$  and  $m_{\mathbf{y}, k_{1\text{lin}}} = \ell'' n$ .

- It is  $\kappa$ -knowledge-sound, with  $\kappa = r/|\mathcal{C}|$ , for

$$\begin{aligned} & \Xi^{\text{sis}}[\text{par}_{\text{sis}}] \vee \Xi_{\text{COM}}^{\text{lin-rel}} \left[ k_{1\text{in}}, k_{1\mathbf{r}}, n, m_{\mathbf{w}}, r, 2\text{cmp}_{\ell'}(\tilde{\beta}_{\mathbf{w}}'), (n_i, \text{par}_{\text{com},i}, m_{\mathbf{y},i}, \tilde{\beta}_{\mathbf{w}}', \tilde{\beta}_{\mathbf{w}}')_{i \in [k_{1\text{in}}]}, 2\gamma_{\mathcal{C}} \right] \\ & \leftarrow \Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n, \hat{m}_{\mathbf{w}}/r', r', \text{par}'_{\text{com}}, \ell''n, \beta'_{\mathbf{w}}, \beta'_{\mathbf{x}}, \text{dcmp}_{\ell''}(q/2), 2], \end{aligned}$$

where

$$\text{par}_{\text{sis}} \supseteq \bigcup_{i \in [k_{1\text{lin}} + 1]} \text{par}_{\text{break}}[\text{par}_{\text{com},i}, m_{\mathbf{y},i}, \tilde{\beta}_{\mathbf{w}}', \tilde{\beta}_{\mathbf{w}}']$$

and  $\tilde{\beta}_{\mathbf{w}}'^2 = \tilde{\beta}_{\mathbf{w}}^2 \cdot \hat{\mathbf{f}} \cdot \text{rad}(\mathbf{f})/\mathbf{f}$  if  $r' \cdot \beta_{\mathbf{w}}'^2 \hat{\mathbf{f}} < q/2$ .

| $P(\text{stmt}, \text{wit})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | $V(\text{stmt})$                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| $\text{parse stmt} = ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{1\text{in}}]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{1\text{r}}]}, \mathbf{A}, \mathbf{b})$<br>$\text{parse wit} = (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{1\text{in}}]})$<br>$\parallel (\text{stmt}, \text{wit}) \in \Xi_{\text{COM}}^{\text{lin}}[k_{1\text{in}}, k_{1\text{r}}, n, m_w, r, \beta_w, (n_i, \text{par}_{\text{com}, i}, m_y, i, \beta_x, i, \beta_y, i)_{i \in [k_{1\text{in}}]}]$<br>$\mathbf{F}_{k_{1\text{in}}} := \text{matrix-from-rows}((\mathbf{l}_j)_{j \in [k_{1\text{r}}]}), \quad n_{k_{1\text{in}}} := k_{1\text{r}}$<br>$\mathbf{T} := \mathbf{F}_{k_{1\text{in}}} \mathbf{W} \bmod q$<br>$\mathbf{Y}_{k_{1\text{in}}} := \mathbf{G}_\ell^{-1}(\mathbf{T}) \in \mathcal{R}^{\ell k_{1\text{r}} \times r} \quad \parallel \mathbf{F}_{k_{1\text{in}}} \mathbf{W} = \mathbf{G} \mathbf{Y}_{k_{1\text{in}}} \bmod q$<br>$(\text{com}_{k_{1\text{in}}}, \mathbf{x}_{k_{1\text{in}}}) := \text{Comp}_{\text{par}_{\text{com}}}(\text{ck}, \text{vec}(\mathbf{Y}_{k_{1\text{in}}}))$ |                                                                      |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\xrightarrow{\text{com}_{k_{1\text{in}}}}$                          |
| $\tilde{\mathbf{w}} := \mathbf{G}_{\ell'}^{-1}(\mathbf{W}\mathbf{c})$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | $\mathbf{c} \leftarrow \mathcal{C}^r$                                |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\xleftarrow{\mathbf{c}}$                                            |
| $\hat{\mathbf{w}} := \text{pack}(\tilde{\mathbf{w}}, (\text{vec}(\mathbf{Y}_i), \mathbf{x}_i) : i \in [k_{1\text{in}} + 1])$<br>$v \leftarrow \langle \hat{\mathbf{w}}, \hat{\mathbf{w}} \rangle$<br>$\mathbf{U} := \text{reshape}_{r'}(\hat{\mathbf{w}}) \in \mathcal{R}^{\hat{m}_w / r' \times r'}$<br>$\mathbf{F} := \mathbf{A}_{n, \hat{m}_w / r'} \quad \parallel \text{from ck}$<br>$\mathbf{Y} := \mathbf{G}_{\ell''}^{-1}(\mathbf{F}\mathbf{U} \bmod q) \in \mathcal{R}^{\ell'' n \times r'}$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                      |
| $(\text{com}, \mathbf{x}) \leftarrow \text{Comp}_{\text{par}'_{\text{com}}}(\text{ck}, \text{vec}(\mathbf{Y}))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\text{trace}(v) \stackrel{?}{\leq} \hat{f} \cdot \tilde{\beta}_w^2$ |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | $\xrightarrow{\text{com}, v}$                                        |
| $f_{\text{sc}} \leftarrow \text{sumcheckify} \left( \begin{array}{l} (\tilde{\mathbf{w}} \mid \mathbf{Y}_i, \mathbf{x}_i : i \in [k_{1\text{in}}] \mid \mathbf{Y}_{k_{1\text{in}}}, \mathbf{x}_{k_{1\text{in}}}) : \\ \mathbf{F}_i \cdot \mathbf{G}_{\ell'} \cdot \tilde{\mathbf{w}} = \mathbf{H}_i \cdot \mathbf{Y}_i \cdot \mathbf{c} \bmod q \quad \forall i \in [k_{1\text{in}}] \\ \mathbf{F}_{k_{1\text{in}}} \cdot \mathbf{G}_{\ell'} \cdot \tilde{\mathbf{w}} = \mathbf{G}_\ell \cdot \mathbf{Y}_{k_{1\text{in}}} \cdot \mathbf{c} \bmod q \\ \text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}_i), \text{com}_i, \mathbf{x}_i) = 1 \quad \forall i \in [k_{1\text{in}} + 1] \\ (\mathbf{G} \cdot \mathbf{Y}_{k_{1\text{in}}})[j, :] \cdot \mathbf{r}_j = t_j \bmod q \quad \forall j \in [k_{1\text{r}}] \\ \mathbf{A} \cdot (\text{vec}(\mathbf{Y}_i))_{i \in [k_{1\text{in}}]} = \mathbf{b} \bmod q \\ \langle \hat{\mathbf{w}}, \hat{\mathbf{w}} \rangle = v \bmod q \end{array} \right)$                                                                                                        |                                                                      |
| $\text{wit}' := (\mathbf{U}, \mathbf{Y}, \mathbf{x})$<br>$k_i := \#\text{constraints}(\text{par}_{\text{com}, i})$<br>$k_{\text{sc}} := \sum_{i \in [k_{1\text{in}} + 1]} k_i + \sum_{i \in [k_{1\text{in}} + 1]} n_i + n + k_{1\text{r}} + 1$<br>$\text{stmt}' := (\text{com}, \mathbf{F}, \mathbf{G}_{\ell''}, f_{\text{sc}})$<br>$\parallel (\text{stmt}', \text{wit}') \in \Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n, \hat{m}_w / r', r', \text{par}'_{\text{com}}, \ell'' n, \tilde{\beta}_w, \beta_x, \text{dcmp}_{\ell''}(q/2), 2]$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                                                                      |
| $\text{return}(\text{stmt}', \text{wit}')$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | $\text{return stmt}'$                                                |

**Fig. 4.**  $\Pi_{r', \ell, \ell', \ell'', \text{par}_{\text{com}}, \text{par}'_{\text{com}}, \tilde{\beta}_w}^{\text{fold-split}}$ : The commit-fold RoK from  $\Xi_{\text{COM}}^{\text{lin}}$  to  $\Xi_{\text{COM}}^{\text{sum}}$ .

– It has the following efficiency measures:

- Prover runtime:  $O(\ell m_w r)$   $\mathcal{R}_q$  operations and the  $\text{com}$  computation.
- Verifier runtime:  $O(1)$   $\mathcal{R}_q$  operations and the cost of sampling  $r$  challenges.
- Prover-to-verifier communication:  $|\text{com}_{k_{1\text{in}}}| + |\text{com}| + O(\log |\mathcal{R}_q|)$ .

*Proof.*

Completeness:

Let

$$\text{stmt} = ((\text{com}_i, \mathbf{F}_i, \mathbf{H}_i)_{i \in [k_{\text{lin}}]}, (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [k_{\text{lr}}]}, \mathbf{A}, \mathbf{b}), \quad \text{wit} = (\mathbf{W}, (\mathbf{Y}_i, \mathbf{x}_i)_{i \in [k_{\text{lin}}]})$$

be such that  $(\text{stmt}, \text{wit}) \in \Xi_{\text{COM}}^{\text{lin}}$ . The protocol outputs

$$\text{stmt}' = (\text{com}, \mathbf{F}, \mathbf{G}_{\ell''}, f_{\text{sc}}), \quad \text{wit}' = (\mathbf{U}, \mathbf{Y}, \mathbf{x}),$$

and we show

$$(\text{stmt}', \text{wit}') \in \Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n, \hat{m}_{\mathbf{w}}/r', r', \text{par}'_{\text{com}}, \ell''n, \tilde{\beta}_{\mathbf{w}}, \beta_{\mathbf{x}}, \text{dcmp}_{\ell''}(q/2), 2].$$

First,  $\mathbf{F}_{k_{\text{lin}}}$  is defined from the left-constraint vectors (more precisely, by  $\mathbf{l}$  constraints from the linear constraints from the input relation  $\Xi_{\text{COM}}^{\text{lin}}$ ) and

$$\mathbf{T} = \mathbf{F}_{k_{\text{lin}}} \mathbf{W} \bmod q, \quad \mathbf{Y}_{k_{\text{lin}}} = \mathbf{G}_{\ell}^{-1}(\mathbf{T}),$$

hence  $\mathbf{F}_{k_{\text{lin}}} \mathbf{W} = \mathbf{G}_{\ell} \mathbf{Y}_{k_{\text{lin}}} \bmod q$ . Let  $\tilde{\mathbf{w}} = \mathbf{G}_{\ell'}^{-1}(\mathbf{W}\mathbf{c})$  and  $\hat{\mathbf{w}} = \text{pack}(\tilde{\mathbf{w}}, (\text{vec}(\mathbf{Y}_i), \mathbf{x}_i) : i \in [k_{\text{lin}} + 1])$ .

By assumption on  $\tilde{\beta}_{\mathbf{w}}$ ,

$$\|\hat{\mathbf{w}}\|_2^2 \leq \text{dcmp}_{\ell'}(r\gamma_{\mathcal{C}}\beta_{\mathbf{w}})^2 + \sum_{i \in [k_{\text{lin}} + 1]} \beta_{\mathbf{y}, i}^2 + \beta_{\mathbf{x}, i}^2 \leq \tilde{\beta}_{\mathbf{w}}^2.$$

Next, the protocol computes the output linear block:

$$\mathbf{U} = \text{reshape}_{r'}(\hat{\mathbf{w}}), \quad \mathbf{F} = \mathbf{A}_{n, \hat{m}_{\mathbf{w}}/r'}, \quad \mathbf{Y} = \mathbf{G}_{\ell''}^{-1}(\mathbf{F}\mathbf{U} \bmod q),$$

so

$$\mathbf{F}\mathbf{U} = \mathbf{G}_{\ell''} \mathbf{Y} \bmod q.$$

By our matrix norm convention (maximum column norm), this also gives

$$\|\mathbf{U}\|_2 \leq \|\text{vec}(\mathbf{U})\|_2 = \|\hat{\mathbf{w}}\|_2.$$

Further,  $(\text{com}, \mathbf{x})$  is computed by  $\text{Com}_{\text{par}'_{\text{com}}}$  on the output committed message, so the output commitment verification constraint holds by perfect correctness of  $\text{COM}$ .

Let  $v = \langle \hat{\mathbf{w}}, \hat{\mathbf{w}} \rangle$ . As in the standard cyclotomic trace identity,

$$\text{trace}(v) = \|\hat{\mathbf{w}}\|_{\sigma, 2}^2 \leq \hat{f} \tilde{\beta}_{\mathbf{w}}^2,$$

therefore the verifier norm check on  $v$  is satisfied.

It remains to check  $f_{\text{sc}}$ . Every constraint used by  $\text{sumcheckify}$  is true by construction:

- $\mathbf{F}_i \mathbf{G}_{\ell'} \tilde{\mathbf{w}} = \mathbf{H}_i \mathbf{Y}_i \mathbf{c} \bmod q$  for all  $i \in [k_{\text{lin}}]$ , from the input linear constraints and linearity of folding;
- $\mathbf{F}_{k_{\text{lin}}} \mathbf{G}_{\ell'} \tilde{\mathbf{w}} = \mathbf{G}_{\ell} \mathbf{Y}_{k_{\text{lin}}} \mathbf{c} \bmod q$ , by definition of  $\mathbf{Y}_{k_{\text{lin}}}$ ;
- $\text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}_i), \text{com}_i, \mathbf{x}_i) = 1$  for all  $i \in [k_{\text{lin}} + 1]$ , from input validity and construction of  $(\text{com}_{k_{\text{lin}}}, \mathbf{x}_{k_{\text{lin}}})$ ;
- $(\mathbf{G} \mathbf{Y}_{k_{\text{lin}}})[j, :] \cdot \mathbf{r}_j = t_j \bmod q$  for all  $j \in [k_{\text{lr}}]$ , from the right-constraints in the input relation;
- $\mathbf{A} \cdot (\text{vec}(\mathbf{Y}_i))_{i \in [k_{\text{lin}}]} = \mathbf{b} \bmod q$ , from the global linear constraints in the input relation;

–  $\langle \widehat{\mathbf{w}}, \overline{\widehat{\mathbf{w}}} \rangle = v \bmod q$ , by definition of  $v$ .

Hence all encoded sumcheck claims are valid, and  $\Pi^{\text{fold-split}}$  is perfectly complete.

Knowledge-soundness:

Let  $\mathcal{A}$  convince the verifier in  $\Pi^{\text{fold-split}}$  with probability at least  $\epsilon$ . By the coordinate-wise rewinding extractor [FMN24, Lemma 7.1], we obtain  $r + 1$  accepting transcripts for challenges  $\mathbf{c}^{(0)}, \dots, \mathbf{c}^{(r)}$ , where for every  $j \in [r]$ ,  $\mathbf{c}^{(j)}$  and  $\mathbf{c}^{(r)}$  differ only in coordinate  $j$ . Let

$$\text{wit}^{*(j)} = (\mathbf{U}^{*(j)}, \mathbf{Y}^{*(j)}, \mathbf{x}^{*(j)})$$

be the extracted witness for transcript  $j$ . Since  $\text{wit}^{*(j)}$  is an accepting source witness with bound parameter  $\beta'_w$ , we have

$$\|\mathbf{U}^{*(j)}\|_2 \leq \beta'_w.$$

Write

$$\widehat{\mathbf{w}}^{*(j)} = \text{vec}(\mathbf{U}^{*(j)}).$$

Then

$$\|\widehat{\mathbf{w}}^{*(j)}\|_2 \leq \sqrt{r'} \|\mathbf{U}^{*(j)}\|_2 \leq \sqrt{r'} \beta'_w.$$

We then parse  $\widehat{\mathbf{w}}^{*(j)}$  according to the fixed packing layout into  $\widetilde{\mathbf{w}}^{*(j)}$  and  $(\mathbf{Y}_i^{*(j)}, \mathbf{x}_i^{*(j)})_{i \in [k_{\text{in}}+1]}$ .

1. Norm bound. For each  $j$ , the sumcheck constraints give

$$\langle \widehat{\mathbf{w}}^{*(j)}, \overline{\widehat{\mathbf{w}}^{*(j)}} \rangle = v^{*(j)} \bmod q.$$

Here, this is the same  $v^{*(j)}$  that appears in the transcript and whose trace is checked by the verifier. Together with the verifier check on  $\text{trace}(v^{*(j)})$  and the side condition from Lemma 9, namely  $r' \cdot \beta_w'^2 \hat{f} < q/2$ , we have  $\|\widehat{\mathbf{w}}^{*(j)}\|_{\sigma,2}^2 < q/2$ , so no wrap-around modulo  $q$  is possible; hence the equality lifts to integers and yields

$$\|\widehat{\mathbf{w}}^{*(j)}\|_{\sigma,2}^2 \leq \hat{f} \tilde{\beta}_w'^2 \implies \|\widehat{\mathbf{w}}^{*(j)}\|_2^2 \leq \tilde{\beta}_w'^2 = \tilde{\beta}_w'^2 \cdot \frac{\hat{f} \text{rad}(\hat{f})}{\hat{f}}.$$

2. SIS from commitment breaks. If there exists  $i \in [k_{\text{in}} + 1]$  and  $j \in [r]$  with  $\mathbf{Y}_i^{*(j)} \neq \mathbf{Y}_i^{*(r)}$ , then both openings verify for the same  $\text{com}_i$  (by accepted transcripts), so we output

$$(\text{vec}(\mathbf{Y}_i^{*(j)}), \mathbf{x}_i^{*(j)}), (\text{vec}(\mathbf{Y}_i^{*(r)}), \mathbf{x}_i^{*(r)})$$

which satisfies  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for extraction, where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com},i}, m_{\mathbf{y},i}, \tilde{\beta}_w', \tilde{\beta}_w']$ , which is a valid extraction due to Lemma 4. Henceforth assume  $\mathbf{Y}_i^{*(j)} = \mathbf{Y}_i^{*(r)} =: \mathbf{Y}_i^*$  and  $\mathbf{x}_i^{*(j)} = \mathbf{x}_i^{*(r)} =: \mathbf{x}_i^*$  for all  $i, j$ .

3. Unfolding by coordinate differences. For each  $j$ , define

$$\mathbf{v}^{*(j)} := \mathbf{G}_{\ell'} \widetilde{\mathbf{w}}^{*(j)}.$$

From the sumcheck constraints, for all  $i \in [k_{\text{in}} + 1]$ ,

$$\mathbf{F}_i \mathbf{v}^{*(j)} = \mathbf{H}_i \mathbf{Y}_i^* \mathbf{c}^{(j)} \bmod q, \quad \mathbf{F}_i \mathbf{v}^{*(r)} = \mathbf{H}_i \mathbf{Y}_i^* \mathbf{c}^{(r)} \bmod q.$$

Subtracting and using  $s_j := c_j^{(j)} - c_j^{(r)} \neq 0$ , let

$$\mathbf{u}^{(j)} := \frac{\mathbf{v}^{*(j)} - \mathbf{v}^{*(r)}}{s_j}.$$

If  $\mathbf{y}_{i,j}^*$  denotes column  $j$  of  $\mathbf{H}_i \mathbf{Y}_i^*$ , then

$$\mathbf{F}_i \mathbf{u}^{(j)} = \mathbf{y}_{i,j}^* \bmod q \quad \forall i \in [k_{\text{lin}} + 1], j \in [r].$$

Therefore, with

$$\mathbf{U} := (\mathbf{u}^{(j)})_{j \in [r]}, \quad \mathbf{s} := (s_j)_{j \in [r]},$$

we get

$$\mathbf{F}_i \mathbf{U} = \mathbf{H}_i \mathbf{Y}_i^* \bmod q \quad \forall i \in [k_{\text{lin}} + 1].$$

Moreover,

$$\|\mathbf{U} \text{diag}(\mathbf{s})\|_2 \leq 2 \text{cmp}_{\ell'}(\tilde{\beta}_{\mathbf{w}}'), \quad \|\text{diag}(\mathbf{s})\|_{\text{op},2} \leq 2\gamma_{\mathcal{C}}.$$

4. Remaining constraints. Let  $\mathbf{T}^* := \mathbf{G} \mathbf{Y}_{k_{\text{lin}}}^*$ . The sumcheck constraints give

$$\mathbf{T}^*[j, :] \cdot \mathbf{r}_j = t_j \bmod q \quad \forall j \in [k_{\text{lr}}].$$

Since  $\mathbf{F}_{k_{\text{lin}}}$  is built from rows  $(\mathbf{l}_j)$  and  $\mathbf{F}_{k_{\text{lin}}} \mathbf{U} = \mathbf{T}^*$ , we conclude

$$\mathbf{l}_j^T \mathbf{U} \mathbf{r}_j = t_j \bmod q \quad \forall j \in [k_{\text{lr}}].$$

The constraints

$$\mathbf{A} \cdot (\text{vec}(\mathbf{Y}_i^*))_{i \in [k_{\text{lin}}]} = \mathbf{b} \bmod q$$

and  $\text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}_i^*), \text{com}_i, \mathbf{x}_i^*) = 1$  for  $i \in [k_{\text{lin}}]$  follow directly from accepted sumcheck claims.

Hence the extractor outputs

$$((\mathbf{U}, \mathbf{s}), (\mathbf{Y}_i^*, \mathbf{x}_i^*)_{i \in [k_{\text{lin}}]})$$

satisfying the  $\Xi_{\text{COM}}^{\text{lin-rel}}$  branch with the parameters from the lemma statement, unless it already output a SIS witness via a commitment break. By [FMN24, Lemma 7.1], success probability is at least  $\epsilon - \frac{r}{|\mathcal{C}|}$ .

Extractor runtime:

Again by [FMN24, Lemma 7.1], the extractor uses  $O(r)$  prover invocations.

Efficiency:

The dominant prover costs are computing  $\mathbf{T} = \mathbf{F}_{k_{\text{lin}}} \mathbf{W}$ , the decomposition/folding maps, and the two commitments  $\text{com}_{k_{\text{lin}}}$ ,  $\text{com}$ . The verifier cost is dominated by sampling  $\mathbf{c} \in \mathcal{C}^r$  and checking  $\text{trace}(v)$ . Communication is dominated by sending  $\text{com}_{k_{\text{lin}}}$ ,  $\text{com}$ , and  $v$ .  $\square$

*Remark 3.* We note the protocol  $\Pi^{\text{fold-split}}$  can be improved when  $\mathcal{R}$  is a power-of-two cyclotomic ring. In the modified variant, instead of inspecting  $\text{trace}(v) \leq \hat{f} \cdot \tilde{\beta}_{\mathbf{w}}^2$ , the verifier checks that  $\text{ct}(v) \leq \tilde{\beta}_{\mathbf{w}}^2$ , since the square of the 2-norm in the coefficient embedding is equal to the constant term of the inner

|                                                                                                                                                                                                          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\text{sumcheckify} : \mathcal{R}^m \times \mathcal{R}^m \times (\mathcal{R}_q^{2m+3})^{k_{\text{sc}}} \rightarrow ((\mathcal{R}_q[y_0, \dots, y_{\mu-1}]^{\leq 2})[x_0, x_1]^{\leq 1})^{k_{\text{sc}}}$ |
| <b>parse input</b> $:= (\mathbf{v}, \mathbf{w}, (a_{i,1}, b_{i,1}, \mathbf{a}_{i,0}, \mathbf{b}_{i,0}, c_i)_{i \in [k_{\text{sc}}]})$                                                                    |
| <b>for</b> $i \in [k_{\text{sc}}]$ :                                                                                                                                                                     |
| $f_{\text{sc}}^{(i)}(x_0, x_1) := (a_{i,1} \cdot x_0 + \text{MLE}[\mathbf{a}_{i,0}]) \cdot (b_{i,1} \cdot x_1 + \text{MLE}[\mathbf{b}_{i,0}]) - \frac{c_i}{m}$                                           |
| <b>return</b> $(f_{\text{sc}}^{(0)}, \dots, f_{\text{sc}}^{(k_{\text{sc}}-1)})$                                                                                                                          |

**Fig. 5.** The sumcheckify algorithm. Here,  $m = 2^\mu$  is the length of the vectors  $\mathbf{v}$  and  $\mathbf{w}$ .

product in that case. The knowledge-soundness proof follows similarly, with the extractor relying on the relaxed bound  $r' \cdot \beta_{\mathbf{w}}'^2 \leq q/2$ , instead of  $r' \cdot \beta_{\mathbf{w}}'^2 \hat{f} \leq q/2$ . The factor of  $\hat{f}$  is avoided together with the translations between the coefficient and canonical embeddings. Further, the  $\beta_{\mathbf{w}}'$  parameter in the extractability statement is simplified to  $\tilde{\beta}_{\mathbf{w}}' = \tilde{\beta}_{\mathbf{w}}$ .

### 6.1 Inner Product Relations $\rightarrow$ Sumcheck Claims

In this subsection we provide an efficient deterministic algorithm, called **sumcheckify**, which takes as input two vectors  $\mathbf{v}, \mathbf{w} \in \mathcal{R}^m$ , together with  $k \geq 1$  inner product relations, and outputs a list of  $k$  bilinear polynomials  $f_{\text{sc}}^{(i)} \in (\mathcal{R}_q[y_0, \dots, y_{\log m-1}]^{\leq 2})[x_0, x_1]^{\leq 1}$  for  $i \in [k_{\text{sc}}]$ <sup>12</sup>. Informally, the **sumcheckify** algorithm will translate inner product statements into sumcheck-type relations, for which we can use the sumcheck protocol to prove them.

For simplicity, assume  $m = 2^\mu$  is a power-of-two – it will always be the case in our protocols, as both vectors  $\mathbf{v}$  and  $\mathbf{w}$  will be an output of the packing function (cf. Lemma 3). We represent the  $i$ -th inner product constraint, for  $i \in [k_{\text{sc}}]$ , with a tuple  $(a_{i,1}, b_{i,1}, \mathbf{a}_{i,0}, \mathbf{b}_{i,0}, c_i)$ , which corresponds to:

$$\langle a_{i,1} \cdot \mathbf{v} + \mathbf{a}_{i,0}, b_{i,1} \cdot \mathbf{w} + \mathbf{b}_{i,0} \rangle = c_i \bmod q \quad (4)$$

where  $\mathbf{a}_{i,0}, \mathbf{b}_{i,0} \in \mathcal{R}_q^m, a_{i,1}, b_{i,1}, c_i \in \mathcal{R}_q$ . The most important property of the output polynomials  $f_{\text{sc}}^{(i)}$  is that vectors  $\mathbf{v}, \mathbf{w} \in \mathcal{R}^m$  satisfy the relation in (4) if and only if

$$\sum_{\mathbf{z} \in \{0,1\}^\mu} F^{(i)}(\mathbf{z}) = 0 \bmod q, \quad \text{where} \quad F^{(i)} := f_{\text{sc}}^{(i)}(\text{MLE}[\mathbf{v}], \text{MLE}[\mathbf{w}]).$$

We provide a formal definition of **sumcheckify** in Figure 5 and show its correctness below.

**Lemma 10.** *Given input  $:= (\mathbf{v}, \mathbf{w}, (a_{i,1}, b_{i,1}, \mathbf{a}_{i,0}, \mathbf{b}_{i,0}, c_i)_{i \in [k_{\text{sc}}]}) \in \mathcal{R}^m \times \mathcal{R}^m \times (\mathcal{R}_q^{2m+3})^{k_{\text{sc}}}$ , the list of polynomials  $(f_{\text{sc}}^{(i)})_{i \in [k_{\text{sc}}]} \leftarrow \text{sumcheckify}(\text{input})$  satisfies*

$$\sum_{\mathbf{z} \in \{0,1\}^\mu} (f_{\text{sc}}^{(i)}(\text{MLE}[\mathbf{v}], \text{MLE}[\mathbf{w}])(\mathbf{z})) = 0 \bmod q \iff \langle a_{i,1} \cdot \mathbf{v} + \mathbf{a}_{i,0}, b_{i,1} \cdot \mathbf{w} + \mathbf{b}_{i,0} \rangle = c_i \bmod q$$

for all  $i \in [k_{\text{sc}}]$ .

<sup>12</sup> In other words, coefficients of the bilinear polynomial  $f^{(i)}$  are multilinear polynomials in  $\mathcal{R}_q[y_0, \dots, y_{\log m-1}]^{\leq 2}$ .

*Proof.* Fix  $i \in [k_{\text{sc}}]$  and consider the  $i$ -th constraint, as described in (4). Using the fact that for  $j \in [m]$ , the  $j$ -th coefficient of the vector  $\mathbf{v}$  is equal to  $\text{MLE}[\mathbf{v}](\text{bin}(j))$  (and similarly for  $\mathbf{w}, \mathbf{a}_{i,0}, \mathbf{b}_{i,0}$ ), we note that the inner product is equivalent to:

$$\sum_{\mathbf{z} \in \{0,1\}^\mu} (a_{i,1} \cdot \text{MLE}[\mathbf{v}](\mathbf{z}) + \text{MLE}[\mathbf{a}_{i,0}](\mathbf{z})) \cdot (b_{i,1} \cdot \text{MLE}[\mathbf{w}](\mathbf{z}) + \text{MLE}[\mathbf{b}_{i,0}](\mathbf{z})) = c_i \bmod q.$$

Hence, by defining

$$f_{\text{sc}}^{(i)}(x_0, x_1) := (a_{i,1} \cdot x_0 + \text{MLE}[\mathbf{a}_{i,0}]) (b_{i,1} \cdot x_1 + \text{MLE}[\mathbf{b}_{i,0}]) - \frac{c_i}{m},$$

the equation above can be written equivalently as

$$\sum_{\mathbf{z} \in \{0,1\}^\mu} (f_{\text{sc}}^{(i)}(\text{MLE}[\mathbf{v}], \text{MLE}[\mathbf{w}])(\mathbf{z})) = 0 \bmod q.$$

Moreover, coefficients of  $f_{\text{sc}}^{(i)}$  indeed lie in  $\mathcal{R}_q[y_0, \dots, y_{\mu-1}]^{\leq 1}$  by definition of a multilinear extension, which concludes the proof.  $\square$

*Connection to committed folding RoK.* Observe that all constraints passed to `sumcheckify` in Figure 4 take the form of inner product relations as in (4), instantiated with  $\mathbf{w} := \widehat{\mathbf{w}}$  and  $\mathbf{v} := \widehat{\mathbf{w}}$  (and also  $a_{i,1} = 0$  for every constraint except the final one). Hence, we can make use of the fact that

$$\text{MLE}[\widehat{\mathbf{w}}] = \overline{\text{MLE}[\widehat{\mathbf{w}}]}$$

so evaluating the multilinear extension of  $\mathbf{v}$  reduces to evaluating the multilinear extension of  $\mathbf{w}$ .

Importantly, the `sumcheckify` procedure is oblivious to the cost of evaluating  $\text{MLE}[\mathbf{a}_{i,0}]$  and  $\text{MLE}[\mathbf{b}_{i,0}]$ , since it does not exploit any structural properties of the inner product vectors  $\mathbf{a}_{i,0}, \mathbf{b}_{i,0}$ . In contrast, the constraints in Figure 4 are either sparse with small Hamming weight, because only specific components of  $\widehat{\mathbf{w}}$  participate in each inner product, or exhibit strong tensor structure (e.g., those derived from vSIS commitment matrices). In Section 8.2, we demonstrate how to leverage these properties to evaluate the `sumcheckify` constraints efficiently, thereby achieving succinct verification.

## 6.2 Example of `sumcheckify` in $\Pi^{\text{fold-split}}$

To illustrate the use of `sumcheckify` in the committed folding protocol  $\Pi^{\text{fold-split}}$ , we consider the constraint

$$\mathbf{F}_0 \mathbf{G}_\ell \widetilde{\mathbf{w}} = \mathbf{G}_{\ell'} \mathbf{Y}_0 \mathbf{c} \bmod q,$$

which is one of the constraints in Fig. 4 that the prover needs to prove after folding. Here,  $\widetilde{\mathbf{w}}$  is the (decomposed) folded witness vector,  $\mathbf{F}_0$  is a vSIS commitment key,  $\mathbf{G}_\ell$  and  $\mathbf{G}_{\ell'}$  are gadget matrices,  $\mathbf{Y}_0$  is the committed output linear block, and  $\mathbf{c}$  is the folding challenge. This constraint can be rewritten as an inner product relation of the form in (4) by considering the  $j$ -th row of the equation, for  $j \in [n]$ :

$$\langle \mathbf{f}_j \otimes \mathbf{g}_\ell, \widetilde{\mathbf{w}} \rangle = \langle \mathbf{y}_j, \mathbf{c} \otimes \mathbf{g}_{\ell'} \rangle \bmod q.$$

Here  $\mathbf{f}_j$  is the  $j$ -th row of the commitment key  $\mathbf{F}_0$ , and  $\mathbf{y}_j$  is the vectorisation of the  $j$ -th gadget-decomposition block of  $\mathbf{Y}_0$  (the rows that  $\mathbf{G}_{\ell'}$  recombines into the  $j$ -th coordinate). The identity

is simply the  $j$ -th coordinate of the matrix equation above: since the gadget matrix factors as  $\mathbf{G}_\ell = \mathbf{I} \otimes \mathbf{g}_\ell^\top$ , the  $j$ -th row of  $\mathbf{F}_0 \mathbf{G}_\ell$  equals  $\mathbf{f}_j \otimes \mathbf{g}_\ell$ , and likewise the  $j$ -th coordinate of  $\mathbf{G}_{\ell'} \mathbf{Y}_0 \mathbf{c}$  equals  $\langle \mathbf{y}_j, \mathbf{c} \otimes \mathbf{g}_{\ell'} \rangle$ .

We note that  $\tilde{\mathbf{w}}$  and  $\mathbf{y}_j$  are committed as part of  $\hat{\mathbf{w}}$  (via the pack algorithm, cf. Lemma 3). Therefore, they are indexed by unique prefixes  $\mathbf{p}_w$  and  $\mathbf{p}_j$  such that

$$\forall \mathbf{z} \in \{0,1\}^{\log \tilde{m}_w} \text{MLE}[\hat{\mathbf{w}}](\mathbf{p}_w \| \mathbf{z}) = \text{MLE}[\tilde{\mathbf{w}}](\mathbf{z})$$

and

$$\forall \mathbf{z} \in \{0,1\}^{\log m_{y,0}/n} \text{MLE}[\hat{\mathbf{w}}](\mathbf{p}_j \| \mathbf{z}) = \text{MLE}[\mathbf{y}_j](\mathbf{z}),$$

where  $\tilde{m}_w$  is the length of  $\tilde{\mathbf{w}}$ ,  $m_{y,0}/n$  is the length of  $\mathbf{y}_j$ , and  $\hat{m}_w$  is the length of  $\hat{\mathbf{w}}$ . Therefore, the inner product relation above can be rewritten as a sumcheck relation with the MLE of  $\hat{\mathbf{w}}$  as the main input, by defining the following constraint:

$$\sum_{\mathbf{z} \in \{0,1\}^{\log \tilde{m}_w}} \text{MLE}[\mathbf{g}_\ell](\mathbf{z}_0) \cdot \text{MLE}[\mathbf{f}_j](\mathbf{z}_1) \cdot \text{eq}(\mathbf{p}_w, \mathbf{z}_2) \cdot \text{MLE}[\hat{\mathbf{w}}](\mathbf{z}) - \text{eq}(\mathbf{p}_j, \mathbf{z}_3) \cdot \text{MLE}[\hat{\mathbf{w}}](\mathbf{z}) \cdot \text{MLE}[\mathbf{c}](\mathbf{z}_4) \cdot \text{MLE}[\mathbf{g}_{\ell'}](\mathbf{z}_5) = 0 \bmod q$$

where  $\mathbf{z}_0, \mathbf{z}_1, \mathbf{z}_2, \mathbf{z}_3, \mathbf{z}_4, \mathbf{z}_5$  are (potentially overlapping) substrings of  $\mathbf{z}$ .

**Efficiency of the sumcheck protocol.** Executing the sumcheck protocol on this relation is linear for the prover. For the verifier, after performing the sumcheck protocol, it receives an evaluation claim over  $\text{MLE}[\hat{\mathbf{w}}]$  at a random point  $\mathbf{r}$ . The verifier can then compute  $\text{MLE}[\mathbf{g}_\ell](\mathbf{r}_0)$ ,  $\text{MLE}[\mathbf{f}_j](\mathbf{r}_1)$ ,  $\text{eq}(\mathbf{p}_w, \mathbf{r}_2)$ ,  $\text{eq}(\mathbf{p}_j, \mathbf{r}_3)$ ,  $\text{MLE}[\mathbf{c}](\mathbf{r}_4)$ , and  $\text{MLE}[\mathbf{g}_{\ell'}](\mathbf{r}_5)$ . Verifier does it efficiently since

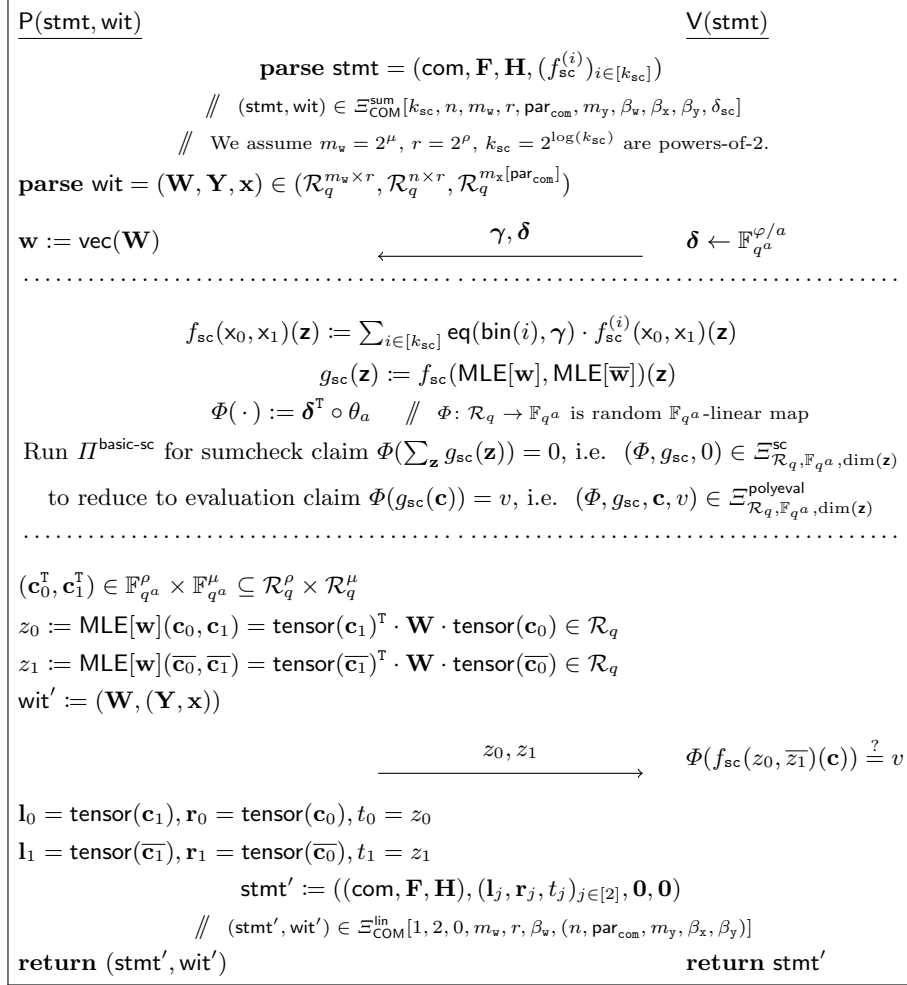
- (i)  $\mathbf{g}_\ell, \mathbf{g}_{\ell'}$  and  $\mathbf{c}$  are low-dimensional (of lengths  $\ell, \ell'$  and  $r$  respectively), so their multilinear extensions are evaluated directly in  $O(\ell), O(\ell')$  and  $O(r)$   $\mathcal{R}_q$ -operations,
- (ii)  $\text{eq}(\mathbf{p}_w, \mathbf{r}_2)$  and  $\text{eq}(\mathbf{p}_j, \mathbf{r}_3)$  are efficient due to the structure stated in Section 3.2,
- (iii)  $\mathbf{f}_j$  is a row of a vSIS commitment key, which is a row-tensor product of length-2 vectors, i.e.  $\mathbf{f}_j = \mathbf{f}_{j,0} \otimes \mathbf{f}_{j,1} \otimes \dots \otimes \mathbf{f}_{j,\mu_1-1}$ , so  $\text{MLE}[\mathbf{f}_j](\mathbf{r}_1)$  can be computed as  $\prod_{k \in [\mu_1]} (f_{j,k,0} + (f_{j,k,1} - f_{j,k,0}) \cdot r_{1,k})$

A more formal treatment of the verifier's and prover's efficiency is provided in Section 8.2.

## 7 Linearisation with Sumcheck

*Basic sumcheck.* We recall the standard sumcheck protocol in Appendix C with a small modification: we include a linear map  $\Phi$  into the claim. In a nutshell, we show that sumcheck reduces a summation claim  $\Phi(\sum_{\mathbf{z}} g(\mathbf{z})) = y$  to an evaluation claim  $\Phi(g(\mathbf{c})) = v$ . We write such statements as  $(\Phi, g, v)$  and  $(\Phi, g, \mathbf{c}, v)$ , respectively. We define languages  $\Xi^{\text{sc}}$  and  $\Xi^{\text{polyeval}}$  for the above sumcheck and poly evaluation claims, parameterised by the (domain of) the linear map  $\Phi$ , variables, challenge, and summation domains. We concretely instantiate  $\Phi$  as a map with two steps. First, it applies an isomorphism  $\theta_a: \mathcal{R}_q \rightarrow (\mathbb{F}_{q^a})^{\varphi/a}$ . Then, it applies a random  $\mathbb{F}_{q^a}$ -linear map  $\delta^T \cdot \_$  to batch the  $(\mathbb{F}_{q^a})^{\varphi/a}$  slots together.

To define  $\theta_a$ , we use the NTT isomorphism  $\text{NTT}: \mathcal{R}_q \rightarrow \mathbb{F}_{q^e}^{\varphi/e}$  defined in Section 3.1. For any  $a \mid e$ , we view the field  $\mathbb{F}_{q^e}$  as an  $\mathbb{F}_{q^a}$ -vector space of dimension  $e/a$ . This induces an isomorphism of  $\mathbb{F}_{q^a}$ -vector spaces  $\mathcal{R}_q \cong (\mathbb{F}_{q^a})^{\varphi/a}$ , i.e. an  $\mathbb{F}_{q^a}$ -linear map (that is *not* a ring isomorphism unless  $a = e$ ). We denote this isomorphism by  $\theta_a: \mathcal{R}_q \rightarrow (\mathbb{F}_{q^a})^{\varphi/a}$ ; If  $a$  equals the residue degree of  $q$



**Fig. 6.**  $\Pi_{\theta_a}^{lin}$ : The linearisation RoK for  $\Xi_{COM}^{sum}$ .

in  $\mathcal{R}$ , then  $\theta_a$  coincides with an NTT transformation. We combine this representation with batch verification in the  $\Pi^{lin}$  protocol below. That is,  $\Phi: \mathcal{R}_q = \mathbb{F}_{q^a}^{\varphi/a} \rightarrow \mathbb{F}_{q^a}$  batches claims  $(\mathbb{F}_{q^a})^{\varphi/a} \rightarrow \mathbb{F}_{q^a}$ , and  $\Phi$  is  $\mathbb{F}_{q^a}$ -linear. Thus, only a polynomial element over  $\mathbb{F}_{q^a}$  (instead of  $\mathcal{R}_q$ ) is sent per round, improving communication and computation. We use  $\mathbb{F}_{q^a} \subseteq \mathcal{R}_q$  as challenge space.

*Remark 4 (Sumcheck as reduction of knowledge).* We treat sumcheck as a reduction of knowledge, although the witness for the statements is trivial. Moreover, we consider sumcheck a special type of “delayed-statement” reduction of knowledge. Namely, the verifier reduces a summation claim  $(\Phi, g, y)$  to an evaluation claim  $(\Phi, g, \mathbf{c}, v)$ , while it does not need to know anything about  $g$  (except degree and number of variables). This ensures that the honest verifier in our protocol executes sumcheck (as  $g$  is not known to it). During knowledge extraction, the extractor knows  $g$ .

*Linearisation protocol.* We describe the protocol  $\Pi^{lin}$  in Fig. 6. The protocol keeps the commitment block (com, **F**, **H**) from the input statement and linearises the sumcheck claims into a minimal committed-linear output statement. Concretely, all claims  $f_{sc}^{(i)}$  are batched into one polynomial

$f_{\text{sc}}$  using a random linear combination (via  $\text{eq}(\text{bin}(i), \gamma)$ ), and a random  $\mathbb{F}_{q^a}$ -linear map  $\Phi: \mathcal{R}_q = \mathbb{F}_{q^a}^{\varphi/a} \rightarrow \mathbb{F}_{q^a}$  batches the slots. The protocol then runs basic sumcheck, obtains the evaluation claim  $\Phi(g_{\text{sc}}(\mathbf{c})) = v$ , and defers verification of this claim to two bilinear constraints over  $\mathbf{W}$  encoded in the output relation  $\Xi_{\text{COM}}^{\text{lin}}$ .

**Lemma 11.** *Let  $\theta_a$  be the  $\mathbb{F}_{q^a}$ -linear isomorphism  $\mathcal{R}_q \rightarrow (\mathbb{F}_{q^a})^{\varphi/a}$  for some  $a \mid e$ . Assume that the input matrix  $\mathbf{F}_0 \in \mathfrak{A}$  is a vSIS commitment key. The protocol  $\Pi_{\theta_a}^{\text{lin}}$  (Fig. 6) satisfies the properties:*

- It is a perfectly complete reduction of knowledge

$$\Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n, m_{\mathbf{w}}, r, \text{par}_{\text{com}}, m_{\mathbf{y}}, \beta_{\mathbf{w}}, \beta_{\mathbf{x}}, \beta_{\mathbf{y}}, \delta_{\text{sc}}] \rightarrow \Xi_{\text{COM}}^{\text{lin}}[1, 2, 0, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, (\text{par}_{\text{lin}, i})_{i \in [1]}]$$

where  $\text{par}_{\text{lin}, 0} := (n, \text{par}_{\text{com}}, m_{\mathbf{y}}, \beta_{\mathbf{x}}, \beta_{\mathbf{y}})$ .

- It is  $\kappa$ -knowledge-sound, with  $\kappa = \frac{\log(k_{\text{sc}}) + (\mu + \rho)\delta_{\text{sc}} + 1}{q^a}$ , for

$$\begin{aligned} & \Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n_0, m_{\mathbf{w}}, r, \text{par}_{\text{com}, 0}, m_{\mathbf{y}, 0}, \beta'_{\mathbf{w}}, \beta'_{\mathbf{x}, 0}, \beta'_{\mathbf{y}, 0}, \delta_{\text{sc}}] \cup \Xi^{\text{sis}}[\text{par}_{\text{sis}}] \\ & \leftarrow \Xi_{\text{COM}}^{\text{lin}}[1, 2, 0, m_{\mathbf{w}}, r, \beta'_{\mathbf{w}}, (n_0, \text{par}_{\text{com}, 0}, m_{\mathbf{y}, 0}, \beta'_{\mathbf{x}, 0}, \beta'_{\mathbf{y}, 0})] \end{aligned}$$

where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com}, 0}, m_{\mathbf{y}, 0}, \beta'_{\mathbf{y}, 0}, \beta'_{\mathbf{x}, 0}] \cup \{(n_0, m_{\mathbf{w}}, 2\beta'_{\mathbf{w}})\}$

- It has the following efficiency measures:

- Prover runtime:  $O(m)$   $\mathcal{R}_q$  operations, cost of computing  $\text{com}$ , two  $\text{MLE}[\text{vec}(\mathbf{W})]$  evaluations, and batching of  $f_{\text{sc}}^{(i)}$  plus the  $\mu$ -variate sumcheck for  $g_{\text{sc}}$  of individual degree  $\leq \delta_{\text{sc}}$ .
- Verifier runtime: Sampling  $O(\log(k_{\text{sc}}) + \mu + \rho + \varphi/e)$  elements from  $\mathbb{F}_{q^e}$ , cost of running the  $(\mu + \rho)$ -variate sumcheck, evaluating  $f_{\text{sc}}$  once, where  $\mu = \log(m_{\mathbf{w}})$  and  $\rho = \log(r)$ .
- Prover-to-verifier comm.:  $O(\log |\mathcal{R}_q|) + O(\delta_{\text{sc}}(\mu + \rho) \log(q^a))$ .

*Proof.*

Completeness:

Let

$$\text{stmt} = (\text{com}, \mathbf{F}, \mathbf{H}, (f_{\text{sc}}^{(i)})_{i \in [k_{\text{sc}}]}), \quad \text{wit} = (\mathbf{W}, \mathbf{Y}, \mathbf{x}),$$

and let  $(\text{stmt}', \text{wit}')$  be the output of  $\Pi^{\text{lin}}$  with

$$\text{stmt}' = ((\text{com}, \mathbf{F}, \mathbf{H}), (\mathbf{l}_j, \mathbf{r}_j, t_j)_{j \in [2]}, \mathbf{0}, \mathbf{0}), \quad \text{wit}' = (\mathbf{W}, (\mathbf{Y}, \mathbf{x})).$$

We show  $(\text{stmt}', \text{wit}') \in \Xi_{\text{COM}}^{\text{lin}}[1, 2, 0, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, (n, \text{par}_{\text{com}}, m_{\mathbf{y}}, \beta_{\mathbf{x}}, \beta_{\mathbf{y}})]$ . Since  $(\text{stmt}, \text{wit}) \in \Xi_{\text{COM}}^{\text{sum}}$ , we already have

$$\mathbf{FW} = \mathbf{HY} \bmod q, \quad \text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}), \text{com}, \mathbf{x}) = 1, \quad \|\mathbf{W}\|_2 \leq \beta_{\mathbf{w}}, \quad \|\text{vec}(\mathbf{Y})\|_2 \leq \beta_{\mathbf{y}}, \quad \|\mathbf{x}\|_2 \leq \beta_{\mathbf{x}}.$$

These are exactly the committed-linear constraints for the single linear block. The global linear constraint is trivial because  $\mathbf{A} = \mathbf{0}$  and  $\mathbf{b} = \mathbf{0}$ .

Let  $\mathbf{w} = \text{vec}(\mathbf{W})$ . By definition of multilinear extension and of  $\mathbf{l}_j, \mathbf{r}_j, t_j$ , we have

$$t_0 = z_0 = \text{MLE}[\mathbf{w}](\mathbf{c}_0, \mathbf{c}_1) = \text{tensor}(\mathbf{c}_1)^{\text{T}} \mathbf{W} \text{tensor}(\mathbf{c}_0) = \mathbf{l}_0^{\text{T}} \mathbf{W} \mathbf{r}_0,$$

and similarly

$$t_1 = z_1 = \text{MLE}[\mathbf{w}](\overline{\mathbf{c}}_0, \overline{\mathbf{c}}_1) = \mathbf{l}_1^{\text{T}} \mathbf{W} \mathbf{r}_1.$$

Hence the linear constraints in  $\Xi_{\text{COM}}^{\text{lin}}$  hold.

Finally, because the input summation claims are true, the batched claim  $\Phi(\sum_{\mathbf{z}} g_{\text{sc}}(\mathbf{z})) = 0$  is true, and perfect completeness of  $\Pi^{\text{basic-sc}}$  gives  $\Phi(f_{\text{sc}}(z_0, \bar{z}_1)(\mathbf{c})) = \Phi(g_{\text{sc}}(\mathbf{c})) = v$ , i.e. the verifier accepts. Therefore,  $\Pi^{\text{lin}}$  is perfectly complete.

Knowledge-soundness:

Let  $\mathcal{A}$  be a prover convincing the verifier with probability at least  $\epsilon$ . We describe an extractor  $\text{Ext}$ . First,  $\text{Ext}$  runs  $\mathcal{A}$  once and obtains an accepting transcript with witness

$$\text{wit}' = (\mathbf{W}', (\mathbf{Y}', \mathbf{x}')) \in \Xi_{\text{COM}}^{\text{lin}}[1, 2, 0, m_{\mathbf{w}}, r, \beta'_{\mathbf{w}}, (n_0, \text{par}_{\text{com},0}, m_{\mathbf{y},0}, \beta'_{\mathbf{x},0}, \beta'_{\mathbf{y},0})].$$

If  $(\mathbf{W}', \mathbf{Y}', \mathbf{x}')$  satisfies all input summation claims, then  $\text{Ext}$  outputs this witness for

$$\Xi_{\text{COM}}^{\text{sum}}[k_{\text{sc}}, n_0, m_{\mathbf{w}}, r, \text{par}_{\text{com},0}, m_{\mathbf{y},0}, \beta'_{\mathbf{w}}, \beta'_{\mathbf{x},0}, \beta'_{\mathbf{y},0}, \delta_{\text{sc}}].$$

Otherwise, define

$$y_i^* := \sum_{\mathbf{z} \in \{0,1\}^\mu} f_{\text{sc}}^{(i)} \left( \text{MLE}[\text{vec}(\mathbf{W}')] , \text{MLE}[\overline{\text{vec}(\mathbf{W}')}] \right) (\mathbf{z}) \in \mathcal{R}_q$$

and note that some  $y_i^* \neq 0$ . The extractor rewinds  $\mathcal{A}$  with fixed prover randomness and obtains a second accepting transcript for fresh verifier challenges, with witness  $\text{wit}'' = (\mathbf{W}'', (\mathbf{Y}'', \mathbf{x}''))$ .

If  $\mathbf{Y}'' \neq \mathbf{Y}'$ , then both openings verify for the same commitment  $\text{com}$ , so  $\text{Ext}$  outputs

$$(\text{vec}(\mathbf{Y}'), \mathbf{x}'), (\text{vec}(\mathbf{Y}''), \mathbf{x}'')$$

which satisfies  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  relation for extraction, where  $\text{par}_{\text{sis}} \supseteq \text{par}_{\text{break}}[\text{par}_{\text{com},0}, m_{\mathbf{y},0}, \beta'_{\mathbf{y},0}, \beta'_{\mathbf{x},0}]$ , which is a valid extraction due to Lemma 4.

Assume now  $\mathbf{Y}'' = \mathbf{Y}'$ . Since both transcripts satisfy the linear block of  $\Xi_{\text{COM}}^{\text{lin}}$  with the same  $(\mathbf{F}, \mathbf{H})$ ,

$$\mathbf{F}(\mathbf{W}'' - \mathbf{W}') = \mathbf{0} \bmod q.$$

If  $\mathbf{W}'' \neq \mathbf{W}'$ , pick a non-zero column  $\mathbf{u}$  of  $\mathbf{W}'' - \mathbf{W}'$ . Then  $\mathbf{A}_{n_0, m_{\mathbf{u}}} \mathbf{u} = \mathbf{0} \bmod q$  (for  $\mathbf{F} = \mathbf{A}_{n_0, m_{\mathbf{u}}}$ ), and

$$\|\mathbf{u}\|_2 \leq \|\mathbf{W}'' - \mathbf{W}'\|_2 \leq 2\beta'_{\mathbf{w}},$$

so  $\text{Ext}$  outputs a witness for  $\Xi^{\text{sis}}[\text{par}_{\text{sis}}]$  branch for extraction, where  $\text{par}_{\text{sis}} \ni (n_0, m_{\mathbf{w}}, 2\beta'_{\mathbf{w}})$ .

Let  $\nu = \log(m_{\mathbf{w}}r)$ . Suppose that the  $y_i^* := \sum_{\mathbf{z} \in \{0,1\}^\nu} f_{\text{sc}}^{(i)}(\text{MLE}[\mathbf{w}'], \text{MLE}[\overline{\mathbf{w}'}])(\mathbf{z}) \in \mathcal{R}_q$  are the honest sums (derived from  $\mathbf{w}'$  obtained above). Note that some  $y_{i^*}^*$  is non-zero, or extraction would have succeeded. Relative to the fixed known witness  $\mathbf{W}'$ , the challenges satisfy that

- except with probability  $\log(k_{\text{sc}})/q^a$ :  $y^* = \sum_{i \in [k_{\text{sc}}]} \text{eq}(\text{bin}(i), \gamma'') \cdot y_i^* \neq 0$ ;
- except with probability  $1/q^a$ :  $\Phi(y^*) \neq 0$ , and  $(\Phi, g_{\text{sc}}, 0) \notin \Xi_{\mathcal{R}_q, \mathbb{F}_{q^a}, \dim(\mathbf{z})}^{\text{sc}}$ ;
- except with probability  $\kappa_{\text{sc}} \leq (\mu + \rho)\delta_{\text{sc}}/q^a$ , sumcheck reduces to a false claim  $(\Phi, g_{\text{sc}}, \mathbf{c}'', v^*) \notin \Xi_{\mathcal{R}_q, \mathbb{F}_{q^a}, \dim(\mathbf{z})}^{\text{polyeval}}$ .

By the union bound, this bad case has probability at most

$$\kappa = \frac{\log(k_{\text{sc}}) + (\mu + \rho)\delta_{\text{sc}} + 1}{q^a}.$$

Therefore, with probability at least  $\epsilon - \kappa$ , the extractor  $\text{Ext}$  outputs a witness for one of the branches in the lemma statement.

#### Extractor runtime:

The extractor performs one accepting run and one forked rewind, so by standard rewinding analysis it runs in expected polynomial time.

#### Efficiency:

The efficiency claims follow directly from the protocol description.  $\square$

*Efficiency of  $\Pi^{\text{lin}}$ .* The exact efficiency of  $\Pi^{\text{lin}}$  depends heavily on the sumcheck claims  $(f_{\text{sc}}^{(i)})_{i \in [k_{\text{sc}}]}$  in the input statement. Naively, the cost of computing the batched  $f_{\text{sc}}$  can be as high as  $O(m_w \cdot k_{\text{sc}})$ , which would be prohibitive for our goal of achieving succinct verification. However, in our application of  $\Pi^{\text{lin}}$  in the composition, the sumcheck claims are well-structured: they either correspond to structured linear constraints (e.g. evaluations of MLEs for  $\text{tensor}(\cdot)$  or rows of the vSIS-based commitment) or are sparse, i.e. concern only small subsets of the witness (e.g. the evaluation of the MLE of the folding challenge). Section 8.2, as a part of the composition-specific analysis, summarises the efficiency of  $\Pi^{\text{lin}}$  when used in composition with the other RoKs defined in this work.

## 8 Composition and Asymptotics

As in prior works [KLNO24b, KLNO25, KLOT25], we can compose the RoKs  $\Pi^{\text{proj-c}}$ ,  $\Pi^{\text{proj-f}}$ ,  $\Pi^{\text{proj-fl}}$ ,  $\Pi^{\text{fold-split}}$  and  $\Pi^{\text{lin}}$  presented in this work into a succinct argument. In contrast to these prior works, which focus on obtaining a succinct argument for (a less expressive variant of)  $\Xi_{\text{COM}}^{\text{lin}}$ , we have the flexibility to choose  $\Xi_{\text{COM}}^{\text{lin}}$  or  $\Xi_{\text{COM}}^{\text{sum}}$  as our “starting” relation. While (a succinct argument for)  $\Xi_{\text{COM}}^{\text{lin}}$  suffices for e.g. constructing polynomial commitment schemes (PCS),  $\Xi_{\text{COM}}^{\text{sum}}$  is more expressive and can capture complex relations such as Rank-1 Constraint Systems (R1CS). Our composition is also much simpler than those in the aforementioned works: it consists of a chain of RoKs alternating between two groups<sup>13</sup>:

$$\Pi^{\text{fold-split}} \circ \Pi^{\text{proj}} : \Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{sum}} \quad \text{and} \quad \Pi^{\text{lin}} : \Xi_{\text{COM}}^{\text{sum}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$$

where  $\Pi^{\text{proj}} \in \{\Pi^{\text{proj-c}}, \Pi^{\text{proj-f}}, \Pi^{\text{proj-fl}}\}$ . As usual, the goal of the composition is to gradually reduce the size of the witness until it becomes so small that it can be sent in plain.

### 8.1 Simplifying Assumptions and Summary of RoKs

As we focus only on asymptotics in this section, we make several simplifying assumptions that ease analysis without affecting the asymptotic behaviour of the composition.

*vSIS parameters.* To assess the hardness of vSIS, we model it as an SIS-type problem whose associated Euclidean kernel lattice has dimension  $N := \varphi \cdot n$ , where  $\varphi$  is the cyclotomic degree and  $n$  is the module rank. Under this identification, a random  $n$ -tuple over  $\mathcal{R}_q$  defines a random  $\mathbb{Z}_q$ -linear constraint, and solving vSIS amounts to finding a short nonzero kernel vector, as in SIS [MR09].

<sup>13</sup>  $\Pi' \circ \Pi$  means apply  $\Pi$  first then  $\Pi'$ , as in function composition.

We use the standard heuristic that the best known attacks are generic lattice-reduction attacks (e.g., BKZ), whose cost is mainly governed by  $N$  and by the ratio between the target norm bound  $\beta$  and the natural volume scale, which for (module-)SIS behaves like  $q^{1/n}$  [CN11]. We assume  $q = \text{poly}(\lambda)$  and  $\beta = \text{poly}(\lambda)$ . Then  $\log q = \Theta(\log \lambda)$  and  $\log \beta = \Theta(\log \lambda)$ , so the dependence on  $q^{1/n}$  saturates once  $n$  exceeds a constant (more precisely, around  $n \approx \log q / \log \beta = \Theta(1)$ ). Beyond this point, increasing  $n$  yields little gain while it increases the dimension  $N = \varphi n$ . Consequently, we fix  $n = O(1)$  and scale security by  $\varphi$ , choosing  $\varphi = \Theta(\lambda)$  so that  $N = \Theta(\lambda)$ . Under these choices, we treat  $\text{vSIS}_{\mathcal{R}, \varphi, n, m, q, \beta}$  as heuristically as hard as SIS in dimension  $N$  against generic lattice-reduction attacks. Similarly, for recursive commitments we will assume that all ranks are  $O(1)$ .

*Commitment parameters.* We assume that the recursive commitment  $\text{COM} = \text{COM}_{m_x, \beta_x}$  has depth 1. As such, we can assume that  $m_x = 0$  and  $\beta_x[\text{par}_{\text{com}}] = 0$  throughout and omit them from the parameters of relations and RoKs. We will also omit all  $\text{aux} = \mathbf{x}$  which would otherwise have appeared. We assume that the commitment key of  $\text{COM}$  has module rank  $n$ , so  $\#\text{constraints}(\text{par}_{\text{com}}) = n \in O(1)$ . This choice of  $\text{COM}$  does not affect the asymptotics as we assume that the rank of the commitment is  $O(1)$ , and the only effect of the commitment in the composition is to add  $O(1)$  linear constraints to the relations. With the choice of  $\text{COM}$  fixed, we will drop it from the subscripts of all relations.

*Random projection parameters.* We assume that  $n_{\text{rp}} = O(\lambda)$  and  $m_{\text{rp}} = \rho \cdot n_{\text{rp}}$  for some  $\rho = \rho(\lambda)$  that we will specify later. This means  $n_{\text{rp}}/m_{\text{rp}} = \rho$  and  $n_{\text{rp}} \cdot m_{\text{rp}} = \lambda^2 \cdot \rho$ . We will also consider  $\mathbf{W} \in \mathcal{R}^{m_v \times \rho}$  with  $\rho$  columns throughout our composition.

*Decomposition degree.* We will also fix a large enough norm bound  $\beta \in \Theta(\sqrt{m_w} \lambda \log q)$  throughout and assume that the decomposition degree  $\ell$  for gadget decomposition  $\mathbf{G}_\ell^{-1}$  is adaptively chosen so that the result has norm at most  $\beta$ . For an input vector  $\mathbf{x} \in \mathcal{R}^{m_v}$  of norm at most  $\alpha \leq q/2$ , it suffices to take  $\ell \geq -2 \ln \alpha / W_{-1} \left( -\frac{\varphi m_v \ln \alpha}{2\beta^2} \right)$  where  $W_{-1}$  is the secondary branch of the Lambert  $W$  function. Note that for any  $u > 0$ , we have  $-W_{-1}(-e^{-u-1}) \geq 1 + \sqrt{2u} + \frac{2u}{3} > \frac{2u}{3}$  due to [Cha13, Theorem 1]. Substituting  $u = \ln \frac{2\beta^2}{e\varphi m_v \ln \alpha}$ <sup>14</sup> and taking  $\beta \geq 2\sqrt{\varphi m_w} \ln q > 2\sqrt{\varphi m_w} \ln \alpha$ , it suffices to set  $\ell \geq 3$ , which is constant. We will therefore assume throughout that  $\ell \leq O(1)$  and also omit it from all parameters.

*Batching parameters.* For  $\Pi^{\text{proj-f}}$ , we pick  $n_{\text{bat}} = O(\lambda / \log \lambda)$ . For  $\Pi^{\text{proj-fl}}$ , we assume that  $\mathcal{R}$  admits a 2-smooth tower structure, i.e.  $c = \log \varphi$ . We let  $n_{\text{bat}, c} = 1$ ,  $n_{\text{bat}, c-1} = O(1)$ ,  $n_{\text{bat}, i} = n_{\text{bat}, c-1}^{c-i}$  for all  $i \in [c]$ , so that  $\sum_{i \in [c]} n_{\text{bat}, i} \in O(\varphi) = O(\lambda)$ . We further assume that  $n_{\text{bat}, 0} \in O(\lambda / \log \lambda)$ . For batching of sumchecks, we pick the parameter  $a$  for the map  $\theta$  in  $\Pi^{\text{lin}}$  to be  $a = O(\lambda / \log \lambda)$ , so that the knowledge error for batching is negligible.

*Dropping parameters.* With the above simplifications, and the observation that  $\Pi^{\text{proj-c}}$ ,  $\Pi^{\text{proj-f}}$ ,  $\Pi^{\text{lin}}$  almost<sup>15</sup> all produce  $\Xi^{\text{lin}}$  relations with  $m_{\mathbf{y}, i} = O(n_i)$  and  $\beta_{\mathbf{y}, i} = \beta$ , the parameters

$$(k_{1\text{in}}, k_{1\text{r}}, n, m_{\mathbf{w}}, r, \beta_{\mathbf{w}}, (n_i, m_{\mathbf{x}, i}, m_{\mathbf{y}, i}, \beta_{\mathbf{x}, i}, \beta_{\mathbf{y}, i})_{i \in [k_{1\text{in}}]})$$

of  $\Xi^{\text{lin}}$  can be simplified to

$$(k_{1\text{in}}, k_{1\text{r}}, n, m_{\mathbf{w}}, \beta_{\mathbf{w}}, (n_i)_{i \in [k_{1\text{in}}]}).$$

<sup>14</sup>  $e$  here is Euler's number.

<sup>15</sup> The only exception is that the parameters of an output relation of  $\Pi^{\text{proj-f}}$  contain a tuple  $(n_{k_{1\text{in}}}, m_{\mathbf{y}, k_{1\text{in}}})$  where  $n_{k_{1\text{in}}} = 0$  but  $m_{\mathbf{y}, k_{1\text{in}}} = O(m/\rho)$ .

**Table 4.** Summary of all RoKs presented in this work with simplifications discussed in Section 8.1. We omit  $k_{1\text{in}}, k_{1\text{r}}$  factors in  $O(\cdot)$  expressions since they stay constant in the compositions that we consider. The prover time and verifier time are measured in number of  $\mathcal{R}_q$  operations (or equivalent). The communication cost is measured in number of  $\mathcal{R}_q$  elements (or equivalent).

|    | Parameters                                                                                                                                                                                                                                                                                     | Prover                               | Verifier                                            | Comm                         |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|-----------------------------------------------------|------------------------------|
| 7  | $\Pi^{\text{proj-c}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{lin}}$<br>$k_{1\text{in}} \mapsto k_{1\text{in}} + 1$<br>$n_{k_{1\text{in}}} = m_w/\rho$                                                                                                                                        | $O(m_w \rho \lambda)$                | $O(\rho \lambda / \log \lambda)$                    | $O(1)$                       |
| 8  | $\Pi^{\text{proj-f}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{lin}}$<br>$k_{1\text{in}} \mapsto k_{1\text{in}} + 2$<br>$n \mapsto n + n_{\text{bat}}$<br>$n_{k_{1\text{in}}} = m_w/\rho$<br>$n_{k_{1\text{in}}+1} = n_{\text{bat}}$                                                           | $O(m_w \rho \lambda)$                | $O((\rho \lambda + \log(m_w \rho))/\log \lambda)$   | $O(\lambda / \log \lambda)$  |
| 12 | $\Pi^{\text{proj-fl}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{lin}}$<br>$k_{1\text{in}} \mapsto k_{1\text{in}} + 2$<br>$n \mapsto n + 1$<br>$n_{k_{1\text{in}}} = m_w/\rho$<br>$n_{k_{1\text{in}}+1} = 1$                                                                                    | $O(m_w \rho \lambda^2 \log \lambda)$ | $O((\rho \lambda + \log(m_w \rho))/\log \lambda)$   | $O(\log \lambda)$            |
| 9  | $\Pi^{\text{fold-split}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{sum}}$<br>$k_{\text{sc}} = O(n + \sum_{i \in [k_{1\text{in}}]} n_i)$<br>$m_w \mapsto O(m_w/\rho + \sum_{i \in [k_{1\text{in}}]} n_i \rho)$<br>$\beta_w \mapsto O(\beta)$<br>$n = 0$<br>$n_0 = O(1)$<br>$k_{1\text{in}} = 1$ | $O(m_w \rho)$                        | $O(\rho / \log \lambda)$                            | $O(1)$                       |
| 11 | $\Pi^{\text{lin}} : \Xi^{\text{sum}} \rightarrow \Xi^{\text{lin}}$<br>$k_{1\text{r}} = 2$                                                                                                                                                                                                      | $O(m_w \rho + \rho \lambda^2)$       | $O(\rho \lambda^2 + \log k_{\text{sc}} + \log m_w)$ | $O(\log m_w / \log \lambda)$ |

Similarly, with the observation that  $\Pi^{\text{fold-split}}$  always outputs a  $\Xi^{\text{sum}}$  relation with  $\delta_{\text{sc}} = 2$ , the parameters  $(k_{\text{sc}}, m_w, m_x, \beta_w, \beta_x, \delta_{\text{sc}})$  for  $\Xi^{\text{sum}}$  can be simplified to  $(k_{\text{sc}}, m_w, \beta_w)$ .

*RoKs Summary* We summarise the effect of each RoK of interest in Table 4.<sup>16</sup> To reflect intuition that  $\lambda \ll m_w \in \text{poly}(\lambda)$ , when reporting asymptotics, we treat  $\log m_w$  as being dominated by  $\lambda$ , e.g.  $O(\lambda + \log m_w) = O(\lambda)$ , and  $\log \lambda$  as being dominated by  $\log m_w$ , e.g.  $O(\log \lambda / \log m_w) = O(1)$ . We are concerned about mainly two chains of RoKs:

**Coarse**  $\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-c}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{lin}}$  and

**Fine**  $\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-fl}} : \Xi^{\text{lin}} \rightarrow \Xi^{\text{lin}}$ .

We analyse the costs of these chains assuming that the starting relation has  $k_{1\text{in}}, k_{1\text{r}}, n, n_i \in O(1)$  for all  $i \in [k_{1\text{in}}]$ . We also assume that we have  $m_w \geq \Omega(\lambda)$  for the coarse chain, since  $\Pi^{\text{proj-c}}$  does not work for  $m_w$  too small. We see that both chains have the following effect on the parameters:

$$\begin{aligned} k_{1\text{in}} &\mapsto 1, & k_{1\text{r}} &\mapsto 2, & n &\mapsto 0, \\ m_w &\mapsto O(m_w/\rho), & \beta_w &\mapsto O(\beta), & (n_i)_{i \in [k_{1\text{in}}]} &\mapsto (O(1)). \end{aligned}$$

The coarse chain costs in total  $O(m_w \rho \lambda)$  prover time,  $O(\rho \lambda^2 + \log m_w)$  verifier time and  $O(\log m_w / \log \lambda)$  proof size. The fine chain costs in total  $O(m_w \rho \lambda^2 \log \lambda)$  prover time,  $O(\rho \lambda^2 + \log m_w)$  verifier time and  $O(\log \lambda + \log m_w / \log \lambda)$  proof size. All of the above is measured in number of  $\mathcal{R}_q$  operations/elements (or equivalent).

<sup>16</sup> The parameter  $n_{k_{1\text{in}}}$  in  $\Pi^{\text{proj-f}}$  and  $\Pi^{\text{proj-fl}}$  is an overestimate. Accurately,  $n_{k_{1\text{in}}} = 0$  and  $m_{y, k_{1\text{in}}} = O(m_w/\rho)$ .

## 8.2 Efficiency of $\Pi^{\text{lin}}$

The exact efficiency of  $\Pi^{\text{lin}}$  highly depends on the sumcheck claims  $(f_{\text{sc}}^{(i)})_{i \in [k_{\text{sc}}]}$  given in the input statement. The following Corollary 1 summarises the efficiency of  $\Pi^{\text{lin}}$  when used in the above composition.

**Corollary 1.** *In the above RoK composition towards the PCS construction, each instance of  $\Pi^{\text{lin}}$  has the following performance guarantees:*

- *Prover complexity:*  $O(m_{\mathbf{w}} + n_{\text{rp}} \cdot m_{\text{rp}}) \mathcal{R}_q$  operations,
- *Verifier complexity:*  $O(r + n_{\text{rp}} \cdot m_{\text{rp}} + \log k_{\text{sc}}) \mathcal{R}_q$  operations and  $O(\mu) \mathbb{F}_{q^e}$  operations.
- *Prover-to-verifier communication:*  $O(\log m_{\mathbf{w}} / \log \lambda) \mathcal{R}_q$  elements.

*Proof.* In the RoK composition towards the PCS construction, we have  $n, k_{1\text{in}}, k_{1\text{r}}, \delta_{\text{sc}} = O(1)$ .

*Prover efficiency.* As stated in Lemma 11, the prover complexity of  $\Pi^{\text{lin}}$  consists of  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations, the cost of computing  $\text{com}$ , two  $\text{MLE}[\widehat{\mathbf{w}}]$  evaluations, batching of  $f_{\text{sc}}^{(i)}$ , and the  $(\mu + \rho)$ -variate sumcheck for  $g_{\text{sc}}$  of individual degree  $\leq \delta_{\text{sc}}$ . Except for the batching of  $(f_{\text{sc}}^{(i)})_{i \in [\mu]}$ , all other costs are bounded by  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations due to the efficiency of the MLE evaluation and the sumcheck protocol analysis in Lemma 13, assuming that  $f_{\text{sc}} = \sum_{i \in [k_{\text{sc}}]} \text{eq}(\text{bin}(i), \gamma) \cdot f_{\text{sc}}^{(i)}$  is computed explicitly (as a list of coefficients) by the prover. The computation of  $\text{com}$  is upper bounded by  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations since the commitment key of  $\text{COM}$  has module rank  $n = O(1)$ .

We have  $k_{\text{sc}}$  polynomials to batch. Except for the projection constraints, the number of constraints is  $O(1)$ , and thus the cost of batching these constraints is  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations.

For the projection constraints, we inspect the constraints for “coarse” projections which are of the form

$$(\mathbf{I}_{m_{\mathbf{w}}/m_{\text{rp}}} \otimes \mathbf{J}) \mathbf{G}_{\ell'} \widetilde{\mathbf{w}} = \mathbf{G}_{\ell} \mathbf{V} \mathbf{c} \bmod q$$

with an implicit lift to  $\mathcal{R}_q$ .

Assume, without loss of generality, that those constraints correspond to the first  $k'_{\text{sc}} \leq k_{\text{sc}}$  polynomials  $f_{\text{sc}}^{(i)}$  for  $i \in [k'_{\text{sc}}]$ . The cost of batching those  $(f_{\text{sc}}^{(i)})_{i \in [k'_{\text{sc}}]}$  is equivalent to the cost of computing coefficients of the following polynomial in  $\mathcal{R}_q[\mathbf{y}]$ :

$$\text{eq}(\mathbf{0}, \gamma_0) \left( \begin{aligned} & \text{MLE}[\text{tensor}(\gamma_1) \cdot (\mathbf{I}_{m_{\mathbf{w}}/m_{\text{rp}}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^{\text{T}})] \cdot \text{eq}(\mathbf{p}_w) \\ & - \text{MLE}[\mathbf{c} \otimes \text{tensor}(\gamma_1) \otimes \mathbf{g}_{\ell}] \cdot \text{eq}(\mathbf{p}_v) \end{aligned} \right)$$

where  $\gamma := (\gamma_0, \gamma_1)^{\text{T}}$  for  $\gamma_0 \in \mathcal{R}_q^{\log k_{\text{sc}} - \log k'_{\text{sc}}}$  and  $\gamma_1 \in \mathcal{R}_q^{\log k'_{\text{sc}}}$  and  $\mathbf{p}_w$  and  $\mathbf{p}_v$  are the selectors for the  $\widetilde{\mathbf{w}}$  and  $\text{vec}(\mathbf{V})$  within  $\widehat{\mathbf{w}}$ . The cost of computing all  $\text{eq}$  terms is  $O(m_{\mathbf{w}}) \mathcal{R}_q$  operations. The cost of computing  $\text{tensor}(\gamma_1)(\mathbf{I}_{m_{\mathbf{w}}/m_{\text{rp}}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^{\text{T}})$  is  $O(m_{\mathbf{w}} + n_{\text{rp}} \cdot m_{\text{rp}}) \mathcal{R}_q$  operations by the mixed-product property of the Kronecker product. The cost of computing  $\mathbf{c} \otimes \text{tensor}(\gamma_1) \otimes \mathbf{g}_{\ell}$  is  $O(m_{\mathbf{w}}) \mathcal{R}_q$ .

Analogously, the cost of batching constraints for “fine” projections is also  $O(m_{\mathbf{w}} + n_{\text{rp}} \cdot m_{\text{rp}} / \varphi) \mathcal{R}_q$  operations.

*Verifier efficiency.* Engaging in the sumcheck protocol for the batched polynomial  $f_{\text{sc}}$  takes  $O(\mu) \mathbb{F}_{q^e}$  operations. Additionally, observe that the verifier efficiency in  $\Pi^{\text{lin}}$  depends on the efficiency of verifying the sumcheck claims, i.e., that

$$f_{\text{sc}}(z_0, \overline{z_1})(\mathbf{s}) = \sum_{i \in [k_{\text{sc}}]} \text{eq}(\text{bin}(i), \gamma) \cdot f_{\text{sc}}^{(i)}(z_0, \overline{z_1})(\mathbf{s}) \stackrel{?}{=} v \bmod q.$$

Each of the  $k_{\text{sc}}$  polynomials  $f_{\text{sc}}^{(i)}$  represents a linear constraint on the original witness  $\mathbf{w}$  (and its conjugate) and the “public” vectors. The evaluation of the MLE for the witness (and its conjugate) at  $\mathbf{s}$  is claimed by the prover as  $z_0$  and  $\bar{z}_1$  (and is verified in further rounds of the protocol). For the MLEs of public inputs, we draw the following conclusions:

- (a) Commitment keys  $\mathbf{F} \in \mathfrak{A}$  are row-tensors: every row  $\mathbf{f}$  of such a key factors as a tensor product  $\mathbf{f} = \mathbf{f}_0 \otimes \mathbf{f}_1 \otimes \cdots \otimes \mathbf{f}_{\mu-1}$  of  $\mu$  length-2 vectors. This perfectly aligns with the structure of the multilinear extension, whose evaluation factors over the variables in the same way:  $\text{MLE}[\mathbf{f}](\mathbf{s}) = \prod_{k \in [\mu]} (f_{k,0} + (f_{k,1} - f_{k,0}) \cdot s_k)$ , as in the example of Section 6.2. Since each of the  $\mu$  factors costs  $O(1) \mathcal{R}_q$  operations, the evaluation of the MLE at  $\mathbf{s}$  is done in  $O(\mu) \mathcal{R}_q$  time.
- (b) For the rows obtained from the “left-right” constraints (i.e.,  $\mathbf{l}_i^T \cdot \mathbf{W} \cdot \mathbf{r}_i = t_i \bmod q$  for  $i \in [k_{\text{lr}}]$ ), they are also row-tensors by construction, as they correspond to the evaluation of  $\mathbf{v}$  (the evaluation point) using a row-tensor basis of the polynomial ring. This alignment with the MLE structure means the evaluation at  $\mathbf{s}$  can be done in  $O(\mu) \mathcal{R}_q$  time.
- (c) The same argument applies for the constraints corresponding to the  $\mathbf{A}$  matrix in the  $\Xi_{\text{COM}}^{\text{lin}}$  relation, which is also a row-tensor; the evaluation of the MLE at  $\mathbf{s}$  can be done in  $O(\mu) \mathcal{R}_q$  time.
- (d) For the challenge vector  $\mathbf{c}$ , the linear constraints, although unstructured, concern only a small part of the witness. In other words, the variables used in the linear constraints form only a small subset of the variables in the multilinear extension, allowing the evaluation of the MLE at  $\mathbf{c}$  to be done in  $O(r) \mathcal{R}_q$  time.
- (e) Finally, the largest chunk of constraints arises from the projection constraints. For “coarse” projections, the constraints correspond to the expression:

$$(\mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^T) \cdot \tilde{\mathbf{w}} = \mathbf{G}_{\ell} \mathbf{V} \mathbf{c} \bmod q$$

or equivalently, when vectorised,

$$(\mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^T) \cdot \tilde{\mathbf{w}} = (\mathbf{c} \otimes \mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{g}_{\ell}) \cdot \text{vec}(\mathbf{V}) \bmod q$$

with implicit lift to  $\mathcal{R}_q$ . Let  $\mathbf{D} := (\mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^T)$  and  $\mathbf{E} := (\mathbf{c} \otimes \mathbf{I}_{m_u/m_{\text{rp}}} \otimes \mathbf{g}_{\ell})$ . The evaluation claims for those constraints are given by:

$$f_{\text{sc}}^{(i)} = \text{MLE}[\mathbf{d}_i] \cdot \text{eq}(\mathbf{p}_w) \cdot \text{MLE}[\hat{\mathbf{w}}] - \text{MLE}[\mathbf{e}_i] \cdot \text{eq}(\mathbf{p}_v) \cdot \text{MLE}[\hat{\mathbf{w}}],$$

where  $\mathbf{p}_w$  and  $\mathbf{p}_v$  are the (public) selectors (within  $\hat{\mathbf{w}}$ ) for  $\tilde{\mathbf{w}}$  and  $\text{vec}(\mathbf{V})$  respectively, and  $\mathbf{d}_i$  and  $\mathbf{e}_i$  are the  $i$ -th rows of  $\mathbf{D}$  and  $\mathbf{E}$  respectively.

We observe that  $k'_{\text{sc}}$  is a power of two, and we let  $\mu' = \log k'_{\text{sc}}$ . Assume, without loss of generality, that these  $k'_{\text{sc}}$  constraints are actually the first  $k'_{\text{sc}}$  constraints out of all  $k_{\text{sc}}$  constraints. Let  $\gamma \in \mathcal{R}_q^{\log k_{\text{sc}}}$  be the random vector used for batching the  $k_{\text{sc}}$  constraints. Therefore, one splits  $\gamma = (\gamma_0, \gamma_1)^T$  where  $\gamma_0 \in \mathcal{R}_q^{\mu - \mu'}$  and  $\gamma_1 \in \mathcal{R}_q^{\mu'}$  and computes:

$$v = \text{eq}(\mathbf{0}, \gamma_0) \cdot \begin{pmatrix} \text{MLE}[\text{tensor}(\gamma_1)^T \mathbf{D}] \cdot \text{eq}(\mathbf{p}_w) \cdot z_0 \\ - \text{MLE}[\text{tensor}(\gamma_1)^T \mathbf{E}] \cdot \text{eq}(\mathbf{p}_v) \cdot z_0 \end{pmatrix} (\mathbf{s}) \bmod q,$$

where  $z_0$  is the claimed evaluation of  $\text{MLE}[\hat{\mathbf{w}}]$  at  $\mathbf{s}$ . We now argue that  $v$  is computable in  $O(r + n_{\text{rp}} \cdot m_{\text{rp}} + \log k_{\text{sc}}) \mathcal{R}_q$  operations. Write  $\gamma_1 = (\gamma'_1, \gamma''_1)$  with  $\gamma'_1 \in \mathcal{R}_q^{\log(m_u/m_{\text{rp}})}$  and  $\gamma''_1 \in \mathcal{R}_q^{\log n_{\text{rp}}}$ , matching the two row-blocks  $\mathbf{I}_{m_u/m_{\text{rp}}}$  and  $\mathbf{J}$  of  $\mathbf{D}$ . Since the multilinear-extension

tensor factors over any partition of its variables,  $\text{tensor}(\gamma_1) = \text{tensor}(\gamma'_1) \otimes \text{tensor}(\gamma''_1)$ . Distributing this through the Kronecker factors of  $\mathbf{D} = \mathbf{I}_{m_u/m_{rp}} \otimes \mathbf{J} \otimes \mathbf{g}_{\ell'}^T$ , the batched row is itself a Kronecker product:

$$\text{tensor}(\gamma_1)^T \mathbf{D} = \text{tensor}(\gamma'_1)^T \otimes (\text{tensor}(\gamma''_1)^T \mathbf{J}) \otimes \mathbf{g}_{\ell'}^T,$$

and analogously  $\text{tensor}(\gamma_1)^T \mathbf{E}$  factors through  $\mathbf{c}$ ,  $\mathbf{I}_{m_u/m_{rp}}$ , and  $\mathbf{g}_\ell$ . Hence the batched row is never obtained by materialising the full  $k'_{sc} \times \tilde{m}_w$  matrix: only the single unstructured factor  $\text{tensor}(\gamma''_1)^T \mathbf{J}$  is computed directly, at cost  $O(n_{rp} \cdot m_{rp})$ , while  $\text{tensor}(\gamma'_1)^T$  and  $\mathbf{g}_{\ell'}^T$  are already row-tensor structured. Each batched row is therefore row-tensor structured, so its multilinear extension factors over the variables and  $\text{MLE}[\cdot](\mathbf{s})$  reduces to a product of per-factor affine evaluations, exactly as in Section 6.2. Collecting the costs,  $\mathbf{J}$  contributes  $O(n_{rp} \cdot m_{rp})$ , the folding-challenge factor  $\mathbf{c}$  in  $\mathbf{E}$  contributes  $O(r)$ , and the batching tensor  $\text{tensor}(\gamma_1)$  together with  $\text{eq}(\mathbf{0}, \gamma_0)$  contributes  $O(\log k_{sc})$ , while the gadget and identity factors cost  $O(1)$ . Hence  $v$  is computable in  $O(r + n_{rp} \cdot m_{rp} + \log k_{sc}) \mathcal{R}_q$  operations.

For “fine” projections, the same idea applies, but due to the different structure of the projection constraints, the evaluation of the MLE at  $\mathbf{s}$  can be done in  $O(r + n_{rp} \cdot m_{rp}/\varphi + \log k_{sc}) \mathcal{R}_q$  operations.

*Communication.* The communication cost of  $\Pi^{\text{lin}}$  is dominated by the cost of sending the evaluation points for the sumcheck claims, which is  $O(\mu) \mathbb{F}_{q^a}$  elements. Equivalently, that is  $O(\log m_w / \log \lambda) \mathcal{R}_q$  elements.  $\square$

### 8.3 Example: Polynomial Commitments

We next investigate the asymptotic performance of a succinct argument obtained by composing our RoKs.

*$\Xi^{\text{lin}}$  relation induced by polynomial commitment construction.* For concreteness, we pick as an example polynomial commitment schemes (PCS) as our target application. Recall that a PCS allows the prover to commit to (the coefficients of) a polynomial and can later prove that the committed polynomial evaluates to some claimed value at a given point. We provide a sketch of how a PCS can be built from a succinct argument for  $\Xi_{\text{COM}}^{\text{lin}}$ . Let the PCS commitment key consist of a commitment key of COM and a random row-tensor matrix  $\mathbf{F} \in \mathcal{R}_q^{n \times \bigotimes_{i \in [\mu]} d_i}$  where  $m_w = \prod_{i \in [\mu]} d_i$ , for some choice of  $n, m_w$ . Suppose the (in general multivariate) polynomial  $p$  to be committed has coefficients (w.r.t. some tensor-structured basis of the polynomial ring) given by  $\text{vec}(\mathbf{W}) \in \mathcal{R}^{m_w \rho}$ . To commit to  $p$ , the prover recursively commits to  $\mathbf{W} \in \mathcal{R}^{m_w \times \rho}$ . That is, they compute  $\mathbf{V} = \mathbf{F} \cdot \mathbf{W} \bmod q$ ,  $\mathbf{Y} = \mathbf{G}_\ell^{-1}(\mathbf{V})$  and  $(\text{com}, \text{aux}) \leftarrow \text{COM.Commit}(\text{ck}, \text{vec}(\mathbf{Y}))$ . The value  $\text{com}$  is then released as a commitment of  $p$ . To prove a polynomial evaluation claim  $p(\mathbf{v}) = t \bmod q$ , the prover expresses  $p(\mathbf{v}) = (\mathbf{r}^T \otimes \mathbf{I}^T) \cdot \text{vec}(\mathbf{W})$  where  $\mathbf{l} \in \mathcal{R}^{m_w/\rho}$  and  $\mathbf{r} \in \mathcal{R}^\rho$  consist of evaluations of some tensor-structured basis polynomials at  $\mathbf{v}$ . This gives a  $\Xi^{\text{lin}}$  relation:

$$\left\{ \begin{array}{l} \text{COM.Verify}(\text{ck}, \text{vec}(\mathbf{Y}), \text{com}) = 1 \\ \mathbf{F} \cdot \mathbf{W} = \mathbf{G}_\ell \cdot \mathbf{Y} \bmod q \\ \mathbf{l}^T \cdot \mathbf{W} \cdot \mathbf{r} = t \bmod q \\ \|\mathbf{W}\| \leq \beta_w, \|\mathbf{Y}\| \leq \beta \end{array} \right.$$

with parameters  $(k_{1\text{in}}, k_{1\text{r}}, n, m_w, r, \beta_w, (n_i)_{i \in [k_{1\text{in}}]}) = (1, 1, 0, m_w, \rho, \beta_w, (O(1)))$ .

**Table 5.** Complexity measures of composed succinct argument for  $\Xi^{\text{lin}}$  in number of  $\mathcal{R}_q$  operations/elements (or equivalent) with different shrink factors. All measures are in  $O(\cdot)$  which is omitted.

| Shrink factor                                     | Prover Verifier    |                                     | Comm                                           |
|---------------------------------------------------|--------------------|-------------------------------------|------------------------------------------------|
| $\rho$                                            | $\rho m_w \lambda$ | $\rho \lambda^2 \log_\rho m_w$      | $\log_\rho(m_w) \cdot \log m_w / \log \lambda$ |
| 2                                                 | $m_w \lambda$      | $\lambda^2 \log m_w$                | $\log^2 m_w / \log \lambda$                    |
| $\lambda$                                         | $m_w \lambda^2$    | $\lambda^3 \log m_w / \log \lambda$ | $\log^2 m_w / \log^2 \lambda$                  |
| $(m_w)^{\log m_w / (\log \lambda \log \log m_w)}$ | $o(m_w^2 \lambda)$ | $o(m_w \lambda^2)$                  | $\log \log m_w$                                |

*Composition.* To build a succinct argument for the above  $\Xi^{\text{lin}}$  relation, we sequentially compose the coarse chain  $\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-c}}$  at most  $O(\log_\rho m_w - \log_\rho \lambda)$  times to shrink  $m_w$  down to  $m'_w = O(n_{\text{TP}}) = O(\lambda)$ . We then sequentially compose the fine chain  $\Pi^{\text{lin}} \circ \Pi^{\text{fold-split}} \circ \Pi^{\text{proj-fl}}$  at most  $O(\log_\rho \lambda)$  times to further shrink  $m'_w$  down to  $m''_w = O(1)$ . For prover runtime, we consider only the runtime of the first chain since it dominates the remaining runtime. For verifier runtime and communication, we consider the sum of the two chains since they are of the same order. This costs in total  $O(\rho m_w \lambda + \rho \lambda^2) = O(\rho m_w \lambda)$  prover time,  $O(\rho \lambda^2 \log_\rho m_w)$  verifier time and  $O(\log_\rho(m_w) \cdot (\log \lambda + \log m_w / \log \lambda))$  communication in  $\mathcal{R}_q$  elements (or equivalent). Since we assume that  $m_w \gg \lambda$ , the communication cost is dominated by  $O(\log_\rho(m_w) \cdot \log m_w / \log \lambda)$ .

In Table 5, we summarise the complexity measures of our composed succinct argument for  $\Xi^{\text{lin}}$  when instantiated with different shrink factors  $\rho$  ranging from a constant, e.g. 2, to slightly sublinear, e.g.  $(m_w \lambda)^{1/\log \log m_w}$ . The value of  $\rho = \lambda$  best reflects the performance of our concrete implementation.

Finally, we note that while the above focuses on  $\Xi^{\text{lin}}$  as the starting relation, a very similar exercise can be done with  $\Xi^{\text{sum}}$  as the starting relation which we leave to the reader.

*Remark 5.* While the example composition discussed above allows extracting a witness for  $\Xi_{\text{COM}}^{\text{lin}}$  without slack, applications of PCS can typically tolerate slack in the polynomial evaluation claim, when the committed polynomial is not required to have small coefficients. For those applications, the very first instance of  $\Pi^{\text{proj-c}}$  can be skipped. In this case, if  $\rho = \Omega(\lambda)$ , the prover runtime can be improved to  $O(m_w \lambda)$ .

## 8.4 Comparison with [KLOT25]

Table 6 summarises the asymptotic comparison between our composed succinct argument for  $\Xi^{\text{lin}}$  and the PCS construction from [KLOT25] for different shrink factors  $\rho$ . We are particularly interested in the case of SALSAA instantiated with  $\rho = 2$  and our PCS instantiation with  $\rho = \lambda$ . We limit our focus to viewing the two constructions as PCS and use  $m$  to denote the number of coefficients of the committed polynomial, which corresponds to  $m_w \cdot \rho$  in our construction.

In this case, our prover runtime is asymptotically faster by a factor of  $\lambda$ , our proof size is asymptotically smaller by a factor of  $\log \lambda$ , and our verifier runtime is asymptotically larger by a factor of  $\lambda / \log \lambda$ . We note that the increased verifier runtime is mainly due to upper bounding the cost of operating on random projection constraints without exploiting their structure. In practice, these constraints, especially when expressed as sumcheck constraints (i.e. turned into evaluation claims), are susceptible to optimisations that can significantly reduce the verifier runtime, as we demonstrate in practical evaluation. We leave as an open question whether the asymptotic analysis can be tightened to reflect the practical performance of the verifier.

**Table 6.** Asymptotic comparison between PCS from [KLOT25] and our protocol for different shrink factors  $\rho$ . All measures are in  $O(\cdot)$  which is omitted. Proof sizes are given in bits, whereas prover and verifier times are given in number of  $\mathcal{R}_q$  operations (or equivalent). We consider asymptotic expression for the proof size in bits, since the rings used in the two constructions are different, and thus the proof size in  $\mathcal{R}_q$  elements can be misleading. In particular, the ring used in [KLOT25] is a cyclotomic ring  $\mathcal{R}'_q$  of degree  $O(\lambda \log m / \log \lambda)$ , whereas the ring used in our construction is  $\mathcal{R}_q$  of degree  $O(\lambda)$ . Modulus  $q$  is assumed to be of  $O(\log m)$  bits for both constructions, which is sufficient for the inner product argument from  $\Pi^{\text{fold-split}}$  (and its analogue in [KLOT25]) to work. We use  $m$  as a shortcut for the number of coefficients of the committed polynomial, which corresponds to  $m_{\text{w}} \cdot \rho$  in our construction. For our work, we include the optimisation of skipping the first  $\Pi^{\text{proj-c}}$  when slack is allowed, as mentioned in Remark 5, and apply the same optimisation to [KLOT25] for a fair comparison.

|                  | [KLOT25]                              | Ours                                |
|------------------|---------------------------------------|-------------------------------------|
| $\rho = 2$       |                                       |                                     |
| Prover:          | $m\lambda$                            | $m\lambda$                          |
| Verifier:        | $\lambda^2 \log m$                    | $\lambda^2 \log m$                  |
| Comm:            | $\lambda \log^3 m / \log \lambda$     | $\lambda \log^3 m / \log \lambda$   |
| $\rho = \lambda$ |                                       |                                     |
| Prover:          | $m$                                   | $m$                                 |
| Verifier:        | $\lambda^3 \log m / \log \lambda$     | $\lambda^3 \log m / \log \lambda$   |
| Comm:            | $\lambda^2 \log^3 m / \log^2 \lambda$ | $\lambda \log^3 m / \log^2 \lambda$ |

## 9 Implementation and Experiments

As a baseline we consider the experiments of [NS24], focusing on the polynomial-commitment instantiation for norm-bounded coefficients from  $\mathbb{Z}_q$  embedded into ring elements. For best performance we work with power-of-two cyclotomic rings  $\mathcal{R} = \mathbb{Z}[\zeta_f]$ ,  $f = 2^k$ , which admit efficient arithmetic via the Number-Theoretic Transform (NTT). We take  $\mathcal{R}$  to “almost split” modulo  $q$ , i.e.  $\mathcal{R}_q \cong (\mathbb{F}_{q^2})^{\varphi/2}$ . We do not use the “lift-and-batch” technique (Appendix B): for our concrete parameters its extra interaction rounds and protocol complexity outweigh the communication savings, so projections use solely  $\Pi^{\text{proj-c}}$  and  $\Pi^{\text{proj-f}}$ . Almost-splitting also lets us use  $\mathbb{F}_{q^e}$  ( $e = 2$ ) as a challenge set and batch across the  $\varphi/e$  NTT slots. We select  $q \approx 2^{50}$  so that runtimes benefit from the AVX-512-IFMA instructions available on modern Intel processors. We operate over the  $f$ -th cyclotomic ring for  $f = 256$  (i.e.  $\varphi = 128$ ). All benchmark suites prove a  $\Xi_{\text{COM}}^{\text{lin}}$  relation with random tensor-structured  $\mathbf{l}, \mathbf{r}$ , equivalent to opening a multilinear polynomial at a random point. The witness  $\ell_\infty$  norm is set to  $2^{31}$ , matching [NS24].

### 9.1 Approximate challenge sampling

We select a modulus  $q$  such that  $\mathcal{R}_q \cong (\mathbb{F}_{q^2})^{\varphi/2}$ , i.e. our cyclotomic ring splits into factors of degree 2. This is because:

- The knowledge error of the RoKs we use (e.g.  $\Pi^{\text{proj-c}}$ ,  $\Pi^{\text{proj-f}}$ ,  $\Pi^{\text{lin}}$ ) is at least  $1/q^e$ , where  $e$  is the residue degree of the ring modulo  $q$ . If  $e = 1$ , as in the case of fully splitting power-of-two cyclotomic rings, the knowledge error becomes too large.
- For  $\Pi^{\text{fold-split}}$  we require the existence of a low-norm challenge set [LS18, ALS20, BC25b] so that the difference of any two distinct elements is invertible. We require that the set is large enough to ensure a sufficiently small knowledge error and the operator norm of the challenges is small. Such set cannot exist if the ring fully splits as the largest subfield of such ring is  $\mathbb{Z}_q$  itself, which itself is not large enough to provide a sufficiently small knowledge error. In other work, e.g. [BS23, NS24], the issue is avoided by using “almost-not-splitting” cyclotomic rings,

i.e. rings that split into factors of large degree. Then, as shown in [LS18], low-norm challenges’ differences are always invertible. However, such rings do not support NTT-based multiplication directly, leading to a significant computational overhead<sup>17</sup>. We take a different approach and sample challenges so that the coefficients are sampled uniformly from the ternary set  $\{-1, 0, 1\}$ . The set of challenges with ternary coefficients is not a strong sampling set by definition, but the probability that the difference of two sampled challenges is non-invertible is small. More precisely, the probability of sampling two elements so that their difference is non-invertible is bounded in Lemma 32 of [BL25] by

$$\epsilon = \max_{j \in [k]} \left( \frac{1}{q} + \frac{1}{q} \sum_{t=1}^{q-1} \prod_{i=0}^{k-1} \left| p + (1-p) \cos \left( \frac{2\pi t r_j^i}{q} \right) \right| \right)^e,$$

where  $p$  is the “bias” of the ternary distribution (i.e. the probability of sampling 0, in our case  $p = 1/3$ ),  $r_j$  are the roots of  $\xi^\varphi + 1$  in  $\mathbb{Z}_q$ ,  $k = \varphi/e$  is the number of irreducible factors of  $\xi^\varphi + 1$  in  $\mathbb{Z}_q[X]$ , and  $e$  is the residue degree. These results serve as a direct generalisation of Lemma 3.2 of [ALS20]. Following the heuristic from Table 1 of [ALS20], we expect that  $\epsilon \approx \varphi/(eq^e)$ . Therefore, we select  $q \approx 2^{50}$  such that  $q = \varphi + 1 \bmod f$ , so that  $e = 2$ . Setting  $p = 1/3$ , we expect that the probability of sampling a non-invertible element is around  $2^{-94}$ . Further, since the cardinality of the set of challenges is large enough, during sampling we reject challenges with large operator norm, which allows us to additionally reduce the norm of the folded and extracted witnesses, leading to a smaller proof size. A more detailed, rigorous analysis of the probability of sampling non-invertible elements with ternary coefficients, together with supporting empirical evidence, is available in [GLLO26].

## 9.2 Implementation details

*Ring arithmetic.* Our implementation is done in Rust. The heart of our implementation is efficient cyclotomic ring arithmetic, which is crucial for the performance of our protocols. As mentioned above, we select parameters so that our ring is “almost-splitting”, i.e. it splits into factors of degree 2, which allows us to leverage the incomplete NTT for efficient polynomial multiplication: each slot contains a degree-1 residue polynomial  $(a_i + b_i X)$  in  $\mathbb{Z}_q[X]/(X^2 - \psi_i)$ , and multiplication reduces to  $\varphi/2$  independent products of such pairs.

We provide two variants of the ring arithmetic implementation:

The first variant uses Intel HEXL [BKS<sup>+</sup>21] via a foreign-function interface. Multiplication of degree- $\varphi$  cyclotomic ring elements in incomplete-NTT form is decomposed into 7 separate HEXL calls using the schoolbook formula: 5 element-wise modular multiplications and 2 element-wise modular additions, all operating on vectors of length  $\varphi/2$ . This variant can be viewed as a Rust re-implementation of the ring arithmetic from [CCC<sup>+</sup>25], designed for proof-friendly CKKS-like FHE schemes (with the distinction that we do not support RNS decomposition). In our benchmarks, this approach matches the performance of the original C++ implementation in [CCC<sup>+</sup>25].

The second variant is based on our own pure-Rust re-implementation of the relevant subset of HEXL, targeting NTT-based arithmetic for moduli  $q \approx 2^{50}$ . The performance improvement over the first variant is twofold. First, we *fuse* the entire multiplication into a single AVX-512 kernel: instead of 7 separate passes over the data, our kernel performs a single pass that loads all inputs and writes

<sup>17</sup> To circumvent this issue, in [BS23, NS24] the arithmetic is “lifted” to a higher modulus and then projected back to the original ring.

both outputs per index, significantly reducing memory traffic. Even using the same schoolbook formula (5 modular multiplications per slot), this fusion alone yields a  $1.35\text{--}1.60\times$  speedup. Second, within the fused kernel we apply the Karatsuba identity  $(a_id_i + b_ic_i) = (a_i + b_i)(c_i + d_i) - a_ic_i - b_id_i$ , reducing the number of modular multiplications from 5 to 4 per slot at the cost of one additional modular addition and two modular subtractions. This provides an additional 2–5% improvement, for a combined speedup of  $1.36\text{--}1.63\times$  over the first variant (and thus also over [CCC<sup>+</sup>25]) across degrees  $\varphi \in \{2^7, 2^8, \dots, 2^{14}\}$ . The individual modular multiplications use the same Dekker-based floating-point technique as Intel HEXL, so the per-element arithmetic is identical; the speedup comes entirely from the improved memory access pattern and the reduced multiplication count<sup>18</sup>.

We remark that these observations, although presented in general terms, have been concretely verified in our benchmarks conducted on the same hardware as the rest of our experiments (see below). That being said, our benchmarking machine features a mid-range mobile CPU. Non-consumer workstations, servers, or cloud CPUs with different microarchitectures may exhibit different performance characteristics – for instance, larger L1 caches may reduce the relative benefit of our fused kernel, different AVX-512 implementations may shift absolute throughput, and less aggressive frequency downscaling under AVX-512 workloads could further improve performance. We leave a more detailed analysis across different microarchitectures for future work.

We have open-sourced our ring arithmetic implementation as an independent library<sup>19</sup>. The library is thoroughly tested and benchmarked, and primarily consists of a re-implementation of the relevant subset of Intel HEXL, enhanced with our fused kernel and Karatsuba optimisation. We chose to develop our own implementation rather than modify the original HEXL codebase to avoid the overhead of foreign-function calls between Rust and C++, and to have greater flexibility in applying optimisations. Our re-implementation (within the set of overlapping functionalities) achieves performance on par with the original HEXL.

*Projection arithmetic.* For projections, both coarse and fine variant, we rely on highly-efficient intrinsics from the AVX-512 instruction set to implement the required matrix-vector multiplications, taking advantage of the structure of the projection matrices (for both prover and verifier side). In projections, we deliberately delay using modular reduction as much as possible, allowing us to perform multiple additions and multiplications before reducing the results modulo  $q$ , which leads to significant performance improvements.

*Platform and portability.* Due to the level of optimisations, we limited our focus on Intel microarchitectures with AVX-512-IFMA support, which are widely available in modern Intel CPUs. HEXL does not support any other microarchitectures, except **x86-64**. Our independent implementation of the ring arithmetic does not inherit this limitation and can be executed on any microarchitecture supported by Rust, albeit with a significant performance degradation.

*Non-interactivity and testing.* To obtain non-interactive arguments, we implement the Fiat-Shamir transform using the BLAKE3 hash function [OANWO20]. Our implementation is single-threaded to provide a fair comparison with prior work [NS24] and is intensively tested using unit tests to ensure correctness. Since most competing protocols only have single-threaded implementations, we report single-threaded numbers for fairness. We reviewed all of the protocols and found that none of

<sup>18</sup> Details of the comparison with [CCC<sup>+</sup>25] and the performance of the fused kernel are available at: [https://github.com/lattice-arguments/rokoko/blob/main/benches/comparison\\_ckks.md](https://github.com/lattice-arguments/rokoko/blob/main/benches/comparison_ckks.md)

<sup>19</sup> Available at: <https://github.com/lattice-arguments/rokoko/tree/main/incomplete-rex1>

them is particularly better or worse suited in the multi-threading setting at the protocol design level. Only SALSAA [KLOT25] currently uses multithreading, and running it single-threaded is still fair, since this only requires minor code adaptation.

### 9.3 Parameters selection

We select parameters to achieve  $\lambda = 128$  security against known attacks listed in Lattice Estimator [APS15]. Our codebase is equipped with an “estimator” mode, which allows us to easily experiment with different parameters. Prover, while executed in the “estimator” mode, outputs the estimated security level, accounting for the norm growth during the extraction process. For statistical soundness, we set overall soundness error  $\kappa \approx 2^{-100}$  (mainly impacted by the size of NTT slots) following the industry standards.

Our parameters are selected semi-manually, by preparing the configuration for all test suites. Then, we run the test suites in “estimator” mode and adjust the parameters accordingly.

To reduce communication, we deviate slightly from the protocol description and allow the prover to perform even more aggressive norm decomposition on the “final level” of the recursive commitment. Then, a separate proof for the norm of the final level is provided (as an additional constraint in  $f_{sc}$ ). This allows us to reduce the rank of the final level significantly, leading to smaller proof sizes.

### 9.4 Experiments

Our experiments were performed on the following hardware and software configuration:

**Table 7.** Experimental setup

| Component        | Specification                                   |
|------------------|-------------------------------------------------|
| Processor        | 11th Gen Intel(R) Core(TM) i7-11850H @ 2.50GHz  |
| Cache            | 256KiB L1 cache, 10MiB L2 cache, 24MiB L3 cache |
| Memory           | 64GiB System Memory                             |
| Operating System | Ubuntu 24.04.3 LTS x86_64                       |
| Machine          | Precision 7560                                  |
| Compilers        | Rust 1.95.0-nightly, g++ 13.3.0                 |
| Execution        | Single-threaded                                 |

The results of our benchmarks are summarised in Table 1. We compare the performance of our implementation with the results from [NS24, KLOT25], which were obtained on the same hardware configuration. Values for the remaining SNARKs (where the concrete performance gaps are too significant to be attributed to architectural advantages) are taken from the original papers, and we refer the reader to those papers for details of their experimental setups.

*Remark 6 (Experiments marked with “†”).* Our target platform is the consumer-grade laptop described above. For the most expensive batch ( $\#\mathbb{Z}_q = 2^{30}$ ), however, Greyhound and SALSAA required more than the 64 GiB of RAM available on the laptop, so for a complete comparison we also ran the benchmarks on a server: a  $2 \times 32$ -core Intel Xeon Platinum 8562Y+ with 2048 GiB of DDR5-5600 memory, as reported in Table 8.

The codebase for our implementation, including the benchmarks, is available at: <https://github.com/lattice-arguments/rokoko>.

**Table 8.** Experiments ran on a server-grade Intel Xeon machine.

| Scheme    | $\#\mathbb{Z}_q = 2^{26}$ |        |        |         | $\#\mathbb{Z}_q = 2^{28}$ |         |        |         | $\#\mathbb{Z}_q = 2^{30}$ |          |        |         |
|-----------|---------------------------|--------|--------|---------|---------------------------|---------|--------|---------|---------------------------|----------|--------|---------|
|           | Comm.                     | P      | V      | $ \pi $ | Comm.                     | P       | V      | $ \pi $ | Comm.                     | P        | V      | $ \pi $ |
| Greyhound | 4.16s                     | 1.69s  | 0.49s  | 46      | 21.60s                    | 6.64s   | 0.86s  | 53      | 88.39s                    | 26.81s   | 1.73s  | 53      |
| SALSAA    | 2.03s                     | 75.76s | 0.333s | 1005    | 8.07s                     | 304.43s | 0.436s | 1232    | 32.58s                    | 1282.25s | 0.554s | 1459    |
| RoKoko    | 1.41s                     | 1.66s  | 8.96ms | 157     | 5.89s                     | 3.64s   | 9.35ms | 157     | 24.8s                     | 11.37s   | 15.6ms | 157     |

### 9.5 Comparison with prior work concrete performance

We position ourselves as a significant improvement over the state-of-the-art lattice-based SNARKs, both in terms of asymptotic complexity and concrete performance. We view SALSAA [KLOT25] and Greyhound [NS24] as the most relevant prior works, since they are the only lattice-based SNARKs whose concrete performance numbers are close to ours. Compared to SALSAA, we achieve a significant improvement in prover time and proof size. Verification and commitment times reported in Table 1 suggest that our verifier and commitment are also faster than SALSAA; we note, however, that SALSAA’s verifier implementation is not optimised, so we expect a more optimised implementation would significantly reduce the verification-time gap. Compared to Greyhound, we achieve a significant improvement in verifier time and some improvement in the prover time. Commitment time reported in Table 1 suggests that our commitment is faster than Greyhound, but we attribute this to the fact that Greyhound uses slightly smaller cyclotomic rings, therefore does not benefit that much from the NTT-based multiplication (and in general, Greyhound suffers from overhead of RNS-style ring arithmetic). Proof size of our scheme is larger than Greyhound’s by a constant factor. We believe that this is mainly due to parameters selection, i.e. executing our protocol with additional levels of recursion rank would lead to smaller proof sizes, but we have not explored that direction, since the verifier time scales linearly with the number of levels, and we wanted to keep the verifier time as low as possible.

### 9.6 Towards a general-usage library

We consider our work as an ongoing effort towards building a general-purpose library for lattice-based succinct arguments. We outline several directions for future work.

- (a) For the sake of this work, we restricted our focus to the PCS-like relations. However, our sumcheck implementation is done in a highly-modular fashion, allowing us to easily plug in different relations as the first step of the reductions chain, which we view as a step towards building a general-purpose library for lattice-based succinct arguments.
- (b) Our current API is restricted to calls to the functions within the same process. A natural extension is to support serialisation/deserialisation of proofs, enabling network communication between prover and verifier.
- (c) The parameters are hard-coded in the codebase and picked semi-manually. An important direction is to support dynamic parameter selection, allowing users to specify security levels and other parameters at runtime.
- (d) Our ring arithmetic implementation is optimised for power-of-two cyclotomic rings, splitting into degree-2 factors. A natural direction is to extend the implementation to support more general rings, or more concretely, to extend support for splitting into degree-4 factors, which would allow us to further reduce the knowledge error. With the intention of still using the “native” (i.e.

without defining an auxiliary, fully splitting, composite modulus and then projecting back to the original ring) approach, we expect that the performance degradation for the prover would be at most  $1.4\times$  (since around half time is spent on the element-wise multiplications, which would be slower by a factor of at most  $1.8\times$  following the numbers from [CCC<sup>+</sup>25]) and the verifier performance would negligibly degrade (since the verifier spends most of the time batching the “fine” projections with  $\mathbb{Z}_q$  challenges, which would be unaffected by the change of the ring).

## References

- AAB<sup>+</sup>24. Marius A. Aardal, Diego F. Aranha, Katharina Boudgoust, Sebastian Kolby, and Akira Takahashi. Aggregating falcon signatures with LaBRADOR. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part I*, volume 14920 of *LNCS*, pages 71–106. Springer, Cham, August 2024. [1](#)
- ABPS24. Shahla Atapoor, Karim Bagheri, Hilder V. L. Pereira, and Jannik Spiessens. Verifiable FHE via lattice-based SNARKs. *CiC*, 1(1):24, 2024. [1](#)
- ACFY24. Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. STIR: Reed-solomon proximity testing with fewer queries. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 380–413. Springer, Cham, August 2024. [1.1](#)
- ACFY25. Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed-solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 214–243. Springer, Cham, May 2025. [1.1](#), [1.3](#), [1](#)
- Ach01. Dimitris Achlioptas. Database-friendly random projections. In *Proceedings of the Twentieth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*, PODS ’01, page 274–281, New York, NY, USA, 2001. Association for Computing Machinery. [A](#)
- ACK21. Thomas Attema, Ronald Cramer, and Lisa Kohl. A compressed  $\Sigma$ -protocol theory for lattices. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 549–579, Virtual Event, August 2021. Springer, Cham. [1.1](#), [1.1](#)
- AFLN24. Martin R. Albrecht, Giacomo Fenzi, Oleksandra Lapiha, and Ngoc Khanh Nguyen. SLAP: Succinct lattice-based polynomial commitments from standard assumptions. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part VII*, volume 14657 of *LNCS*, pages 90–119. Springer, Cham, May 2024. [1.1](#)
- AHIV17. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligerio: Lightweight sublinear arguments without a trusted setup. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 2087–2104. ACM Press, October / November 2017. [1](#)
- AHIV23. Scott Ames, Carmit Hazay, Yuval Ishai, and Muthuramakrishnan Venkitasubramaniam. Ligerio: lightweight sublinear arguments without a trusted setup. *DCC*, 91(11):3379–3424, 2023. [1.3](#)
- Ajt96. Miklós Ajtai. Generating hard instances of lattice problems (extended abstract). In *28th ACM STOC*, pages 99–108. ACM Press, May 1996. [1.1](#)
- AL21. Martin R. Albrecht and Russell W. F. Lai. Subtractive sets over cyclotomic rings - limits of Schnorr-like arguments over lattices. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 519–548, Virtual Event, August 2021. Springer, Cham. [1.1](#), [1.1](#)
- ALS20. Thomas Attema, Vadim Lyubashevsky, and Gregor Seiler. Practical product proofs for lattice commitments. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 470–499. Springer, Cham, August 2020. [1.3](#), [9.1](#)
- APS15. Martin Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology*, 9:169–203, 10 2015. <https://github.com/malb/lattice-estimator/tree/66771ec3d331e2021eccf17331a5ed1ff71f3ddb>. [9.3](#)
- BBHR18. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. [1.1](#), [1.3](#), [1](#)
- BC25a. Dan Boneh and Binyi Chen. LatticeFold: A lattice-based folding scheme and its applications to succinct proof systems. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part III*, volume 16247 of *LNCS*, pages 330–362. Springer, Singapore, December 2025. [1.3](#), [2.2](#)
- BC25b. Dan Boneh and Binyi Chen. LatticeFold+: Faster, simpler, shorter lattice-based folding for succinct proof systems. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 327–361. Springer, Cham, August 2025. [1.2](#), [1.3](#), [2.1](#), [6](#), [9.1](#)
- BCFW25. Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. Linear-time accumulation schemes. Cryptology ePrint Archive, Report 2025/753, 2025. [1.3](#)
- BCR<sup>+</sup>19. Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, Cham, May 2019. [1.3](#)
- BCS21. Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. Sumcheck arguments and their applications. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 742–773, Virtual Event, August 2021. Springer, Cham. [1.1](#), [1.1](#)

- BCS23. Jonathan Bootle, Alessandro Chiesa, and Katerina Sotiraki. Lattice-based succinct arguments for NP with polylogarithmic-time verification. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 227–251. Springer, Cham, August 2023. [1.1](#), [C.1](#)
- BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020. [1.3](#)
- BK20. Dan Boneh and Sam Kim. One-time and interactive aggregate signatures from lattices. [https://crypto.stanford.edu/~skim13/agg\\_ots.pdf](https://crypto.stanford.edu/~skim13/agg_ots.pdf), 2020. [1](#)
- BKS<sup>+</sup>21. Fabian Boemer, Sejun Kim, Gelila Seifu, Fillipe D. M. de Souza, and Vinodh Gopal. Intel HEXL: Accelerating homomorphic encryption with intel AVX512-IFMA52. Cryptology ePrint Archive, Report 2021/420, 2021. [9.2](#)
- BL17. Carsten Baum and Vadim Lyubashevsky. Simple amortized proofs of shortness for linear relations over polynomial rings. Cryptology ePrint Archive, Report 2017/759, 2017. [1.1](#)
- BL25. Katharina Boudgoust and Aleksandra Lapiha. Leftover hash lemma(s) over cyclotomic rings. Cryptology ePrint Archive, Report 2025/1080, 2025. [9.1](#)
- BLNS20. Jonathan Bootle, Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. A non-PCP approach to succinct quantum-safe zero-knowledge. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 441–469. Springer, Cham, August 2020. [1.1](#)
- BLS19. Jonathan Bootle, Vadim Lyubashevsky, and Gregor Seiler. Algebraic techniques for short(er) exact lattice-based zero-knowledge proofs. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 176–202. Springer, Cham, August 2019. [1.3](#)
- BMNW25a. Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Accumulation without homomorphism. In Raghu Meka, editor, *ITCS 2025*, volume 325, pages 23:1–23:25. LIPIcs, January 2025. [1.3](#)
- BMNW25b. Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Arc: Accumulation for reed-solomon codes. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 128–160. Springer, Cham, August 2025. [1.3](#)
- BS23. Ward Beullens and Gregor Seiler. LaBRADOR: Compact proofs for R1CS from module-SIS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 518–548. Springer, Cham, August 2023. [1.1](#), [4](#), [1.1](#), [1.2](#), [2.1](#), [5.1](#), [5](#), [6](#), [9.1](#), [17](#), [A](#)
- CCC<sup>+</sup>25. Ignacio Cascudo, Anamaria Costache, Daniele Cozzo, Dario Fiore, Antonio Guimarães, and Eduardo Soria-Vazquez. Verifiable computation for approximate homomorphic encryption schemes. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 643–677. Springer, Cham, August 2025. [9.2](#), [18](#), [\(d\)](#)
- Cha13. Ioannis Chatzigeorgiou. Bounds on the lambert function and their application to the outage analysis of user cooperation. *IEEE Communications Letters*, 17(8):1505–1508, 2013. [8.1](#)
- CHM<sup>+</sup>20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020. [1.3](#)
- CLM23. Valerio Cini, Russell W. F. Lai, and Giulio Malavolta. Lattice-based succinct arguments from vanishing polynomials - (extended abstract). In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 72–105. Springer, Cham, August 2023. [1.1](#), [2.1](#), [1](#)
- CMNW24. Valerio Cini, Giulio Malavolta, Ngoc Khanh Nguyen, and Hoeteck Wee. Polynomial commitments from lattices: Post-quantum security, fast verification and transparent setup. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 207–242. Springer, Cham, August 2024. [1.1](#), [1](#)
- CN11. Yuanmi Chen and Phong Q. Nguyen. BKZ 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 1–20. Springer, Berlin, Heidelberg, December 2011. [8.1](#)
- CT10. Alessandro Chiesa and Eran Tromer. Proof-carrying data and hearsay arguments from signature cards. In Andrew Chi-Chih Yao, editor, *ICS 2010*, pages 310–331. Tsinghua University Press, January 2010. [1.3](#)
- FKNP24. Giacomo Fenzi, Christian Knabenhans, Ngoc Khanh Nguyen, and Duc Tu Pham. Lova: Lattice-based folding scheme from unstructured lattices. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part IV*, volume 15487 of *LNCS*, pages 303–326. Springer, Singapore, December 2024. [1.3](#)
- FMN24. Giacomo Fenzi, Hossein Moghaddas, and Ngoc Khanh Nguyen. Lattice-based polynomial commitments: Towards asymptotic and concrete efficiency. *Journal of Cryptology*, 37(3):31, July 2024. [6](#), [6](#), [6](#)

- GBK<sup>+</sup>24. Mariana Gama, Emad Heydari Beni, Jiayi Kang, Jannik Spiessens, and Frederik Vercauteren. Blind zkSNARKs for private proof delegation and verifiable computation over encrypted data. Cryptology ePrint Archive, Report 2024/1684, 2024. [1](#)
- GGPR13. Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013. [1.3](#)
- GHL22. Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part I*, volume 13275 of *LNCS*, pages 458–487. Springer, Cham, May / June 2022. [1.1](#), [A](#)
- GLLO26. Albert Garreta, Helger Lipmaa, Urmaz Luhaäär, and Michał Osadnik. Cyclo: Lightweight lattice-based folding via partial range checks. In Joan Daemen and Emmanuel Thomé, editors, *Advances in Cryptology – EUROCRYPT 2026*, pages 33–62, Cham, 2026. Springer Nature Switzerland. [2.2](#), [9.1](#)
- GLS<sup>+</sup>23. Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 193–226. Springer, Cham, August 2023. [1.3](#), [1](#)
- Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. [1.3](#)
- GSW23. Riddhi Ghosal, Amit Sahai, and Brent Waters. Non-interactive publicly-verifiable delegation of committed programs. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 575–605. Springer, Cham, May 2023. [1](#)
- GWC19. Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. [1.3](#)
- HMZ25. Mi-Ying Miryam Huang, Xinyu Mao, and Jiapeng Zhang. Sublinear proofs over polynomial rings. Cryptology ePrint Archive, Report 2025/199, 2025. [2.2](#)
- HSS24. Intak Hwang, Jinyeong Seo, and Yongsoo Song. Concretely efficient lattice-based polynomial commitment from standard assumptions. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 414–448. Springer, Cham, August 2024. [1](#)
- Kil92. Joe Kilian. A note on efficient zero-knowledge proofs and arguments (extended abstract). In *24th ACM STOC*, pages 723–732. ACM Press, May 1992. [1](#)
- KLNO24a. Michael Kloof, Russell W. F. Lai, Ngoc Khanh Nguyen, and Michał Osadnik. RoK, paper, SISsors – toolkit for lattice-based succinct arguments. Cryptology ePrint Archive, Report 2024/1972, 2024. [2](#)
- KLNO24b. Michael Kloof, Russell W. F. Lai, Ngoc Khanh Nguyen, and Michał Osadnik. RoK, paper, SISsors toolkit for lattice-based succinct arguments - (extended abstract). In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 203–235. Springer, Singapore, December 2024. [1.1](#), [2.1](#), [3.4](#), [2](#), [6](#), [8](#)
- KLNO25. Michael Kloof, Russell W. F. Lai, Ngoc Khanh Nguyen, and Michał Osadnik. RoK and roll - verifier-efficient random projection for  $\tilde{O}(\lambda)$ -size lattice arguments - (extended abstract). In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part III*, volume 16247 of *LNCS*, pages 297–329. Springer, Singapore, December 2025. [1.1](#), [1.2](#), [1](#), [2.1](#), [2.2](#), [1](#), [5](#), [5.3](#), [6](#), [8](#), [B](#)
- KLOT25. Shuto Kuriyama, Russell W. F. Lai, Michał Osadnik, and Lorenzo Tucci. SALSAA – sumcheck-aided lattice-based succinct arguments and applications. Cryptology ePrint Archive, Paper 2025/2124, 2025. [1.1](#), [1.2](#), [1](#), [2.1](#), [2.2](#), [2.2](#), [2.3](#), [2.3](#), [8](#), [8.4](#), [6](#), [9.2](#), [9.4](#), [9.5](#)
- KP16. Yael Tauman Kalai and Omer Paneth. Delegating RAM computations. In Martin Hirt and Adam D. Smith, editors, *TCC 2016-B, Part II*, volume 9986 of *LNCS*, pages 91–118. Springer, Berlin, Heidelberg, October / November 2016. [1](#)
- KP23. Abhiram Kothapalli and Bryan Parno. Algebraic reductions of knowledge. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 669–701. Springer, Cham, August 2023. [3.4](#)
- KST21. Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. Cryptology ePrint Archive, Report 2021/370, 2021. [1.3](#)
- Lee21. Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 1–34. Springer, Cham, November 2021. [1](#)

- LFKN92. Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *J. ACM*, 39(4):859–868, October 1992. [1.1](#)
- LLZ<sup>+</sup>25. Fengrun Liu, Haofei Liang, Tianyu Zhang, Yuncong Hu, Xiang Xie, Haisheng Tan, and Yu Yu. HasteBoots: Proving FHE bootstrapping in seconds. Cryptology ePrint Archive, Report 2025/261, 2025. [1](#)
- LM23. Russell W. F. Lai and Giulio Malavolta. Lattice-based timed cryptography. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part V*, volume 14085 of *LNCS*, pages 782–804. Springer, Cham, August 2023. [1](#)
- LNP22. Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Maxime Plançon. Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 71–101. Springer, Cham, August 2022. [1.1](#), [1.3](#)
- LNS21. Vadim Lyubashevsky, Ngoc Khanh Nguyen, and Gregor Seiler. Shorter lattice-based zero-knowledge proofs via one-time commitments. In Juan Garay, editor, *PKC 2021, Part I*, volume 12710 of *LNCS*, pages 215–241. Springer, Cham, May 2021. [1.1](#)
- LNSW13. San Ling, Khoa Nguyen, Damien Stehlé, and Huaxiong Wang. Improved zero-knowledge proofs of knowledge for the ISIS problem, and applications. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 107–124. Springer, Berlin, Heidelberg, February / March 2013. [1.3](#)
- LS18. Vadim Lyubashevsky and Gregor Seiler. Short, invertible elements in partially splitting cyclotomic rings and applications to lattice-based zero-knowledge proofs. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 204–224. Springer, Cham, April / May 2018. [9.1](#)
- MBKM19. Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2111–2128. ACM Press, November 2019. [1.3](#)
- Mic94. Silvio Micali. CS proofs (extended abstracts). In *35th FOCS*, pages 436–453. IEEE Computer Society Press, November 1994. [1](#)
- MR09. Daniele Micciancio and Oded Regev. *Lattice-based Cryptography*, pages 147–191. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009. [8.1](#)
- NOZ26. Ngoc Khanh Nguyen, George O’Rourke, and Jiapeng Zhang. Hachi: Efficient lattice-based multilinear polynomial commitments over extension fields. Cryptology ePrint Archive, Paper 2026/156, 2026. [2.2](#)
- NS24. Ngoc Khanh Nguyen and Gregor Seiler. Greyhound: Fast polynomial commitments from lattices. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 243–275. Springer, Cham, August 2024. [1.1](#), [1.1](#), [1.2](#), [1](#), [2.3](#), [9](#), [9.1](#), [17](#), [9.2](#), [9.4](#), [9.5](#)
- NS25. Wilson Nguyen and Srinath Setty. Neo: Lattice-based folding scheme for CCS over small fields and pay-per-bit commitments. Cryptology ePrint Archive, Report 2025/294, 2025. [1.3](#)
- OANWO20. Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. Blake3: the high-speed cryptographic hash, 2020. [9.2](#)
- OKC<sup>+</sup>25. Michal Osadnik, Darya Kaviani, Valerio Cini, Russell W. F. Lai, and Giulio Malavolta. Papercraft: Lattice-based verifiable delay function implemented. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 1603–1621. IEEE Computer Society Press, May 2025. [1](#), [1](#)
- PHGR13. Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013. [1.3](#)
- Val08. Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, March 2008. [1.3](#)
- WTs<sup>+</sup>18. Riad S. Wahby, Ioanna Tzialla, abhi shelat, Justin Thaler, and Michael Walfish. Doubly-efficient zkSNARKs without trusted setup. In *2018 IEEE Symposium on Security and Privacy*, pages 926–943. IEEE Computer Society Press, May 2018. [1.3](#)
- XZZ<sup>+</sup>19. Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 733–764. Springer, Cham, August 2019. [1.3](#)
- ZLW<sup>+</sup>25. Zhelei Zhou, Yun Li, Yuchen Wang, Zhaomin Yang, Bingsheng Zhang, Cheng Hong, Tao Wei, and Wenguang Chen. ZHE: Efficient zero-knowledge proofs for HE evaluations. In Marina Blanton, William

Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 3328–3346. IEEE Computer Society Press, May 2025. [1](#)

## A On the Asymptotic Johnson–Lindenstrauss Conjecture

The committed random projection of Section 5 relies on the ring Johnson–Lindenstrauss lemma (Lemma 5). Its *concrete* parameters, projection dimension  $n_{\text{rp}} = 256$  and norm bounds  $\alpha_{\text{rp}} = \sqrt{30}$ ,  $\beta_{\text{rp}} = \sqrt{337}$ ,  $b = 125$  at failure probability  $\kappa_{\text{rp}} = 2^{-128}$ , are exactly those proven in [BS23, Lemmas 4.1 and 4.2]. For the *asymptotic* analysis of the recursive composition (Section 8), however, we additionally need to know how these parameters scale as the failure probability  $\kappa$  shrinks with the security parameter. This scaling is the content of the following conjecture, which is used in Lemma 5 and, through it, in the asymptotic efficiency claims of Sections 5 and 6.

*Conjecture 1 (Asymptotic Johnson–Lindenstrauss scaling).* Let  $\chi$  be the distribution of Lemma 5 and let  $q \in \mathbb{N}$ . There is a choice of projection dimension  $n_{\text{rp}}(\kappa) = \Theta(\log(1/\kappa))$  for which the parameters of Lemma 5 can be taken as

$$\alpha_{\text{rp}}(\kappa) = \Theta(\sqrt{\log(1/\kappa)}), \quad \beta_{\text{rp}}(\kappa) = \Theta(\sqrt{\log(1/\kappa)}), \quad b(\kappa) = \Theta(\sqrt{\log(1/\kappa)}).$$

The conjectured scaling matches the second-moment behaviour of the projection  $\mathbf{J}\mathbf{w}$ . Fix a vector  $\mathbf{w}$  and let  $\mathbf{r}$  be a row of  $\mathbf{J} \leftarrow_{\$} \chi^{n_{\text{rp}} \times m_{\text{rp}}}$ . The entries  $r_i$  are i.i.d., with zero mean and  $\mathbb{E}[r_i^2] = \frac{1}{2}$ , so the cross terms in the inner product vanish in expectation and

$$\mathbb{E}[\langle \mathbf{r}, \mathbf{w} \rangle^2] = \sum_i \mathbb{E}[r_i^2] w_i^2 = \frac{1}{2} \|\mathbf{w}\|_2^2.$$

Summing over the  $n_{\text{rp}}$  rows gives  $\mathbb{E}[\|\mathbf{J}\mathbf{w}\|_2^2] = \frac{n_{\text{rp}}}{2} \|\mathbf{w}\|_2^2$ , so the typical scale of  $\|\mathbf{J}\mathbf{w}\|_2 / \|\mathbf{w}\|_2$  is  $\sqrt{n_{\text{rp}}/2} = \Theta(\sqrt{n_{\text{rp}}})$ . The squared norm  $\|\mathbf{J}\mathbf{w}\|_2^2$  is a sum of  $n_{\text{rp}}$  independent sub-exponential variables, so a standard concentration inequality (Bernstein-type) gives

$$\Pr \left[ \left| \|\mathbf{J}\mathbf{w}\|_2^2 - \frac{n_{\text{rp}}}{2} \|\mathbf{w}\|_2^2 \right| > \epsilon \cdot \frac{n_{\text{rp}}}{2} \|\mathbf{w}\|_2^2 \right] \leq \exp(-\Theta(\epsilon^2 n_{\text{rp}})).$$

Forcing the right-hand side below the target failure probability  $\kappa$  requires  $\epsilon = \Theta(\sqrt{\log(1/\kappa)/n_{\text{rp}}})$ . Choosing  $n_{\text{rp}} = \Theta(\log(1/\kappa))$  makes  $\epsilon = \Theta(1)$ : the multiplicative distortion stays within a *constant-factor window* around the mean scale  $\sqrt{n_{\text{rp}}/2}$ , so both endpoints  $\alpha_{\text{rp}}(\kappa)$  and  $\beta_{\text{rp}}(\kappa)$  are of order  $\sqrt{n_{\text{rp}}} = \Theta(\sqrt{\log(1/\kappa)})$ . Any sub-logarithmic  $n_{\text{rp}}$  would instead force  $\epsilon \rightarrow \infty$  and a vanishing lower endpoint  $\alpha_{\text{rp}}$ , while a super-logarithmic  $n_{\text{rp}}$  would only shrink the window further at a higher communication cost;  $n_{\text{rp}} = \Theta(\log(1/\kappa))$  is thus the natural choice.

It remains to address the reduction modulo  $q$ , i.e. that the verifier observes  $\|\mathbf{J}\mathbf{w} \bmod q\|_2$  rather than  $\|\mathbf{J}\mathbf{w}\|_2$ . We sketch how the proof of [BS23, Lemma 4.2] (which strengthens [GHL22, Corollary 3.3]) would behave asymptotically; it requires fixing a number of scaling constants, and our conjecture is that this is possible. The proof proceeds by a case distinction:

**Case 1:** The condition  $\|\mathbf{w}\| < q/10$  in [BS23, Section A] should scale as  $\beta_1(\kappa) = \tilde{\beta}_1(\kappa)\sqrt{n} = \theta(\sqrt{n}) = \theta(\sqrt{\log(1/\kappa)})$  for a constant  $\beta_1(\kappa)$ . The proof step relies on a bound for a 1-dimensional JL projection and a union bound, which (heuristically) has suitable scaling.

**Case 2:** The condition  $\|\mathbf{w}\|_\infty \geq q/60$  in [BS23, Section A] should scale as  $\alpha_2(\kappa) = \tilde{\alpha}_2(\kappa)\sqrt{n} = \theta(n)$  for a constant  $\tilde{\alpha}_2(\kappa)$ . The bound  $k = 29$  on summands in  $\sum_{i=0}^{29} \binom{256}{i}$  scales (via Chernoff bound) again with  $\tilde{\alpha}'\sqrt{n}$  for suitable  $c'$ . A suitable parameter choice of  $\tilde{\beta}_2$  and  $\tilde{\alpha}'$  should allow the conclusion of this case to hold.

**Case 3:** The condition  $\|\mathbf{w}\| \geq q/10$  and  $\|\mathbf{w}\|_\infty < q/60$  in [BS23, Section A] leads to scalings  $\tilde{\alpha}_1\sqrt{n}$  and  $\tilde{\beta}_2\sqrt{n}$ , for constants  $\tilde{\alpha}_1 \leq \tilde{\beta}_1$ ,  $\tilde{\alpha}_2 \leq \tilde{\beta}_2$ . The employed anti-concentration argument (based on Berry–Esseen’s theorem) depends on the ratio between  $\tilde{\alpha}_1$  and  $\tilde{\beta}_2$ . Again, by a suitable parameter choice, the conclusion of the case should hold, because the probability term  $p = 0.39 + 2 \cdot 0.15 < 1$  that a row is bad would asymptotically be a constant  $p = p(\alpha_1, \beta_2) < 1$ . hence  $n = \theta(\log(\kappa))$  suffices for failure probability  $\theta(\kappa)$ .

The argument above is heuristic in exactly the sense of [GHL22]: it treats the discrete projection distribution  $\chi$  over  $\{-1, 0, 1\}$  as if it were a Gaussian with the same mean and variance. Such a heuristic is justified by a very similar variant of Johnson–Lindenstrauss lemma from [Ach01]. For a genuinely Gaussian  $\mathbf{J}$ , the squared norm  $\|\mathbf{J}\mathbf{w}\|_2^2$  is exactly  $\frac{1}{2}\|\mathbf{w}\|_2^2$  times a  $\chi^2$ -distributed variable with  $n_{\text{rp}}$  degrees of freedom, and the scaling  $\alpha_{\text{rp}}, \beta_{\text{rp}}, b = \Theta(\sqrt{\log(1/\kappa)})$  then follows rigorously from standard  $\chi^2$  tail bounds.

## B Committed Fine Random Projection with “Lift-and-Batch”

As discussed in Section 5.3, the RoK  $\Pi^{\text{proj-f}}$  presented in Fig. 3 ultimately results in a communication cost of  $\tilde{\Omega}(\lambda^2) \mathcal{R}_q$  elements when composed. To reduce communication cost to  $\tilde{O}(\lambda) \mathcal{R}_q$  elements, an idea borrowed from the unstructured projection protocol of [KLNO25] is to assume and exploit a  $d$ -smooth tower structure of  $\mathcal{R}$ , i.e. assume that

$$\mathcal{R} = \mathcal{R}_c \supset \mathcal{R}_{c-1} \supset \dots \supset \mathcal{R}_1 \supset \mathcal{R}_0 = \mathbb{Z}. \quad \text{and} \quad [\mathcal{R}_{i+1} : \mathcal{R}_i] \leq d.$$

The trick is to *gradually* lift the trace value from  $\mathcal{R}_i$  to  $\mathcal{R}_{i+1}$ , while *simultaneously* applying batch verification to reduce the elements in  $\mathcal{R}_i$  to fewer elements in  $\mathcal{R}_{i+1}$ , and so on.

More concretely, consider the prover computing  $\mathbf{r}_0 = \mathbf{r} \in \mathcal{R}^{n_{\text{bat},0}}$  as in Fig. 3. Instead of sending  $\mathbf{r}_0$  in plain, which would be expensive, the prover sends its trace relative to  $\mathcal{R}_0 = \mathbb{Z}$  and receives from the verifier a random challenge  $\mathbf{Z}_0 \in \mathcal{R}_0^{n_{\text{bat},1} \times n_{\text{bat},0}}$ . Using the challenge, prover batches  $\mathbf{r}_0 \in \mathcal{R}^{n_{\text{bat},0}}$  into  $\mathbf{r}_1 \in \mathcal{R}^{n_{\text{bat},1}}$  and sends its trace relative to  $\mathcal{R}_1$ . Continuing in this way, the prover eventually arrives at  $\mathbf{r}_c \in \mathcal{R}_c^{n_{\text{bat},c}}$  where  $\mathcal{R}_c = \mathcal{R}$ , which is small enough to be sent in plain. For suitable choices of parameters, the sketched protocol has  $O(1)$  communication complexity measured in  $\mathcal{R}_q$  elements.

To ensure soundness of batching, we further require that each intermediate extension  $\mathcal{R}_i$  admits a “relatively large” subfield modulo  $q$ . Concretely, we write  $\mathcal{Q}_i = \mathcal{R}_i/(q\mathcal{R}_i) \cong (\mathbb{F}_{q^{e_i}})^{\varphi_i/e_i}$ . Given  $n_{\text{bat},i+1}$  rows in  $\mathbf{Z}_i$  used to batch  $n_{\text{bat},i}$  constraints, we can manage to lose a knowledge error proportional to  $(n_{\text{bat},i}/q^{e_i})^{n_{\text{bat},i+1}}$  for each  $i$ . For favourable rings, e.g. if some intermediate  $\mathcal{Q}_i$  is a field of size  $2^\lambda$ , we observe that communication for the batching phase is bounded by  $c|\mathcal{R}_q|$ , where  $c$  is the tower height.

Due to complexity of the notation and the fundamental similarity to the unstructured projections in [KLNO25], we omit the full protocol description and analysis here. The envisioned protocol follows the same structure as Fig. 3, with more stages of batching and lifting interleaved as described in [KLNO25].

We will refer to this protocol as  $\Pi^{\text{proj-fl}}$  in the asymptotic analysis. We do not use that protocol in our concrete instantiations (or rather, we view  $\Pi^{\text{proj-f}}$  as a special case of  $\Pi^{\text{proj-fl}}$  with tower height  $c = 1$ ).

**Lemma 12.** *Let  $\mathcal{R} = \mathcal{R}_c \supset \dots \supset \mathcal{R}_0 = \mathbb{Z}$  be a tower of rings and  $\mathbf{n}_{\text{bat}} = (n_{\text{bat},0}, \dots, n_{\text{bat},c}) \in \mathbb{N}^{c+1}$  be batching parameters. Let  $\mathbf{e} \in \mathbb{N}^{c+1}$  be the vector of extension degrees, i.e.  $\mathbf{e}^T = (e_0, \dots, e_c)$*

where  $e_i$  is the residue degree of  $q$  in  $\mathcal{R}_i$ . Let  $\ell, \ell' \in \mathbb{N}$  be gadget decomposition parameters and  $\text{par}_{\text{com}}, \text{par}'_{\text{com}}$  be parameters for COM so that  $\beta_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}_{\text{com}}]$  and  $\beta'_{\mathbf{x}} = \beta_{\mathbf{x}}[\text{par}'_{\text{com}}]$ . Let  $m_{\text{rp}} | \varphi m_{\mathbf{w}}$ . The RoK  $\Pi_{n_{\text{bat}}, \ell, \ell', \text{par}_{\text{com}}, \text{par}'_{\text{com}}}^{\text{proj-fl}}$  satisfies the following properties:

- It is  $\epsilon$ -complete, with  $\epsilon = (\varphi m_{\mathbf{w}}/m_{\text{rp}})\kappa_{\text{rp}}$ , for  $\Xi_{\text{COM}}^{\text{lin}} \rightarrow \Xi_{\text{COM}}^{\text{lin}}$  with parameter changes

$$k_{\text{lin}} \mapsto k_{\text{lin}} + 2, \quad n \mapsto n + n_{\text{bat},c}$$

where  $(n_i, \text{par}_{\text{com},i}, m_{\mathbf{y},i}, \beta_{\mathbf{x},i}, \beta_{\mathbf{y},i})$  for  $i \in \{k_{\text{lin}}, k_{\text{lin}} + 1\}$  is given by

$$\begin{cases} \left(0, \text{par}_{\text{com}}, \ell m_{\frac{n_{\text{rp}}}{m_{\text{rp}}}}, \beta_{\mathbf{x}}, \text{dcmp}_{\ell}(\beta_{\text{rp}} \cdot \beta_{\mathbf{w}})\right) & i = k_{\text{lin}}, \\ \left(n_{\text{bat},c}, \text{par}'_{\text{com}}, \ell' n_{\text{bat},c}, \beta'_{\mathbf{x}}, \text{dcmp}_{\ell'}(\sqrt{\varphi n_{\text{bat}}} q/2)\right) & i = k_{\text{lin}} + 1. \end{cases}$$

- It is  $\kappa$ -knowledge-sound with

$$\kappa = \sum_{i \in [c]} (n_{\text{bat},i}/q^{e_i})^{n_{\text{bat},i+1}} + (\varphi m_{\mathbf{w}}/m_{\text{rp}})\kappa_{\text{rp}},$$

where  $\text{cmp}(\beta'_{\mathbf{y},k_{\text{lin}}})/\alpha_{\text{rp}} < q/b$  and

$$\begin{aligned} & \Xi_{\text{sis}}[\text{par}_{\text{sis}}] \vee \Xi_{\text{COM}}^{\text{lin}} \left[ \begin{array}{l} k_{\text{lin}}, k_{\text{lr}}, n, m_{\mathbf{w}}, r, \text{cmp}(\beta'_{\mathbf{y},k_{\text{lin}}})/\alpha_{\text{rp}}, \\ (n_i, \text{par}_{\text{com},i}, m_{\mathbf{y},i}, \beta'_{\mathbf{x},i}, \beta'_{\mathbf{y},i})_{i \in [k_{\text{lin}}]} \end{array} \right] \\ \leftarrow & \Xi_{\text{COM}}^{\text{lin-rel}} \left[ \begin{array}{l} k_{\text{lin}} + 2, k_{\text{lr}}, n + n_{\text{bat},c}, m_{\mathbf{w}}, r, \beta'_{\mathbf{w}}, \\ (n_i, \text{par}_{\text{com},i}, m_{\mathbf{y},i}, \beta'_{\mathbf{x},i}, \beta'_{\mathbf{y},i})_{i \in [k_{\text{lin}}+2]}, \varrho' \end{array} \right] \end{aligned}$$

where

$$\begin{aligned} \text{par}_{\text{sis}} \supseteq & \text{par}_{\text{break}}[\text{par}_{\text{com},k_{\text{lin}}}, m_{\mathbf{y},k_{\text{lin}}}, \beta'_{\mathbf{y},k_{\text{lin}}}, \beta'_{\mathbf{x},k_{\text{lin}}}] \\ & \cup \text{par}_{\text{break}}[\text{par}_{\text{com},k_{\text{lin}}+1}, m_{\mathbf{y},k_{\text{lin}}+1}, \beta'_{\mathbf{y},k_{\text{lin}}+1}, \beta'_{\mathbf{x},k_{\text{lin}}+1}] \cup \{(n_0, m_{\mathbf{w}}, 2 \cdot \beta'_{\mathbf{w}}\varrho')\} \end{aligned}$$

- It has the following efficiency measures:

- Prover runtime:  $O(m_{\mathbf{w}} r n_{\text{rp}} \varphi) \mathbb{Z}$  operations,  $O\left(\sum_{i \in [c]} n_{\text{bat},i} m r\right) \mathcal{R}_q$  operations and the cost of computing  $\text{com}_{k_{\text{lin}}}$  and  $\text{com}_{k_{\text{lin}}+1}$ .
- Verifier runtime:  $O(n_{\text{rp}} m_{\text{rp}})$  samplings from  $\chi$ ,  $O(n_{\text{bat},0} \log(\varphi m_{\mathbf{w}}/m_{\text{rp}}))$  samplings from  $\mathbb{Z}_q$ ,  $O(n_{\text{bat},i+1})$  samplings from  $\mathcal{R}_{i+1,q}$  for  $i \in [c-1]$ , and  $O(n_{\text{bat},c}) \mathcal{R}_q$  operations.
- Prover-to-verifier communication:  $|\text{com}_{k_{\text{lin}}}| + |\text{com}_{k_{\text{lin}}+1}| + \sum_{i \in [c]} O(n_{\text{bat},i} \cdot \log |\mathcal{R}_{c,q}|)$ .

## C Generic sumcheck

We recall sumchecks and expand on their use with NTT-based batching over  $\mathbb{F}_{q^e}$ , as applied in Section 7.

### C.1 Basic sumcheck

We follow the sumcheck notion for modules from [BCS23], which we briefly recall. Let  $M$  be a (commutative)  $R$ -module. Let  $g \in M[x_0, \dots, x_{\ell-1}] = M[\mathbf{x}]$  be a polynomial with coefficients in  $M$ . Let  $\mathcal{C} \subseteq R$  be a set with invertible differences in  $R$ , a.k.a. a strong sampling set. Let  $H \subseteq R$ , usually  $H = \{0, 1\}$ .

The usual sumcheck language consists of tuples

$$\text{stmt} = (g, y) \in M[x_0, \dots, x_{\ell-1}] \times M \quad \text{with} \quad \sum_{\mathbf{x} \in H^\ell} g(\mathbf{x}) = y$$

However, we generalise it slightly, by introducing a “ $\mathcal{C}$ -linear” map<sup>20</sup>  $\Phi: N \rightarrow M$ , i.e.  $\Phi(a \cdot c) = \Phi(a) \cdot c$  for all  $a \in N$  and  $c \in \mathcal{C}$ . Hence, our sumcheck language  $\Xi_{R,N,M,\mathcal{C},\ell}^{\text{sc}}$  consists of tuples  $\text{stmt} = (\Phi, g, y)$  such that

- $\Phi: N \rightarrow M$  is a  $\mathcal{C}$ -linear map;
- $g \in N[x_0, \dots, x_{\ell-1}] = N[\mathbf{x}]$  is a polynomial with coefficients in  $N$ ;
- $y = \Phi(\sum_{\mathbf{x} \in H^\ell} g(\mathbf{x}))$  is the summation claim.

If unspecified, we assume  $M = N$ ,  $\Phi = \text{id}$  and  $H = \{0, 1\}$ . The sumcheck protocol is a reduction (of knowledge) from  $\Xi_{R,N,M,\mathcal{C},H,\ell}^{\text{sc}}$  to a polynomial evaluation claim  $\Xi_{R,N,M,\mathcal{C},H,\ell}^{\text{polyeval}}$ , where  $\Xi_{R,N,M,\mathcal{C},H,\ell}^{\text{polyeval}}$  consists of statements  $\text{stmt} = (\Phi, g, \mathbf{c}, v)$  such that

- $\Phi: N \rightarrow M$  is a  $\mathcal{C}$ -linear map;
- $g \in N[x_0, \dots, x_{\ell-1}] = N[\mathbf{x}]$  is a polynomial with coefficients in  $N$ ;
- $\mathbf{c} \in \mathcal{C}^\ell$  is a vector of (random) challenges;
- $v = \Phi(g(\mathbf{c}))$  is the evaluation claim.

In the following, we usually leave the languages  $\Xi^{\text{sc}}$  and  $\Xi^{\text{polyeval}}$  implicit, as they will always be clear from the context (namely from  $f$  and the stated sum over  $f$ ). Moreover, if  $N = R$ ,  $M = \mathcal{C} = \mathbb{F} \subseteq R$ ,  $H = \{0, 1\}$  we use the shorthand notation

$$\Xi_{R,\mathbb{F},\ell}^{\text{sc}} := \Xi_{R,N,M,\mathcal{C},\ell}^{\text{sc}} \quad \text{and} \quad \Xi_{R,\mathbb{F},\ell}^{\text{polyeval}} := \Xi_{R,N,M,\mathcal{C},H,\ell}^{\text{polyeval}}.$$

We give formal statements for the sumcheck protocol next.

**Definition 4 (Basic sumcheck with linear map).** *Let  $R$  be a non-trivial ring,  $N, M$  be  $R$ -modules,  $\mathcal{C} \subseteq R$  be a subset with invertible difference (a.k.a. strong sampling set) and  $H = \{0, 1\}$ . Let  $g \in N[x_1, \dots, x_\ell]$  and  $y' \in M$ . Let  $\Phi: N \rightarrow M$  be an “ $\mathcal{C}$ -linear” map (i.e.  $\Phi(a \cdot c) = \Phi(a) \cdot c$  for all  $a \in N$ ,  $c \in \mathcal{C}$ ). The sumcheck protocol in Fig. 7 with  $\mathcal{C}$ -linear map  $\Phi$  is a public-coin reduction of knowledge from a summation claim*

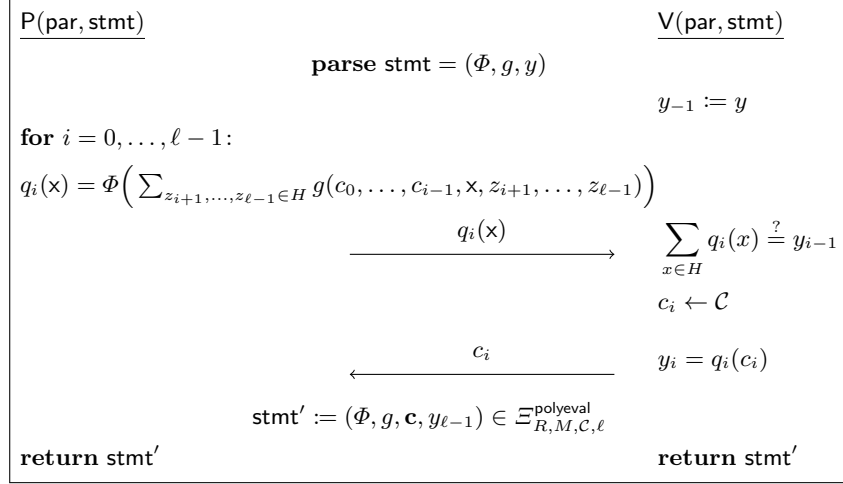
$$\text{stmt} = (\Phi, g, y) \quad \text{where} \quad \Phi \left( \sum_{(x_0, \dots, x_{\ell-1}) \in H^\ell} g(x_0, \dots, x_{\ell-1}) \right) = y$$

*to an evaluation claim  $\Phi(g(\mathbf{c})) = v$ , i.e.*

$$\text{stmt} = (\Phi, g, \mathbf{c}, v) \in \Xi_{R,N,M,\mathcal{C},\ell}^{\text{polyeval}}$$

*where  $\mathbf{c} = (c_0, \dots, c_{\ell-1}) \in \mathcal{C}^\ell$  are the verifier’s challenges.*

<sup>20</sup> More precisely,  $\mathcal{C}$ -linear for the subring  $C \subseteq R$  generated by  $\mathcal{C}$ .



**Fig. 7.** The basic sumcheck protocol (supporting a  $\mathcal{C}$ -linear map  $\Phi$ ).

*Remark 7.* The basic sumcheck verifier does not need access to  $g$ ; one-time evaluation access  $\mathbf{c} \mapsto \Phi(g(\mathbf{c}))$  suffices to run the verification. More abstractly, it suffices to convince the verifier that  $\Phi(g(\mathbf{c})) = v$  via a polynomial evaluation proof.

We include the linear map  $\Phi$  in the sumcheck protocol to capture batch-verification techniques, in particular to make batch verification with the NTT isomorphism  $\mathcal{R}_q \cong \mathbb{F}_q^{\varphi/e}$  more explicit.

*Remark 8 (Eliminating the linear map  $\Phi$ ).* For any  $\Phi: N \rightarrow M$ , we can define  $\Phi(g) \in M[\mathbf{x}]$ , which is obtained from  $g \in M[\mathbf{x}]$  by coefficient-wise application of  $\Phi$ . For  $\mathcal{C}$ -linear  $\Phi$ , it is easy to see that  $\Phi(g(\mathbf{c})) = \Phi(g)(\mathbf{c})$ . A brief inspection then shows that the sumcheck protocol for  $(\Phi, g, y)$  (that reduces to  $(\Phi, g, \mathbf{c}, y_0)$ ) is equivalent to a standard sumcheck for  $(\text{id}, \Phi(g), y)$  (that reduces to  $(\text{id}, \Phi(g), \mathbf{c}, y_0)$ ).

**Lemma 13.** *The sumcheck protocol with linear map  $\Phi$  is a perfectly complete reduction of knowledge. For an instance  $\Phi: N \rightarrow M$ ,  $g \in N[x_0, \dots, x_{\ell-1}]$ ,  $y \in M$  as in Definition 4, the (knowledge) soundness error  $\kappa$  is bounded by  $\kappa \leq \frac{\sum_{i \in [\ell]} \deg_{x_i}(g)}{|\mathcal{C}|} \leq \frac{\ell \deg(g)}{|\mathcal{C}|}$ , where  $\deg(g)$  denotes the maximal individual degree.*

*The communication consists of  $\sum_{i \in [\ell]} (\deg_{x_i}(g) + 1)$  elements in  $M$  from the prover,  $\ell$  challenges in  $\mathcal{C}$  from the verifier. Assuming the prover has access to the evaluation table of  $g$  on  $H^\ell$ , the prover performs  $O\left(\frac{d}{|H|-1} \cdot |H|^\ell\right)$  operations in  $N$  and  $\sum_{i \in [\ell]} (\deg_{x_i}(g) + 1)$  applications of  $\Phi$ , where  $d = \max_i \deg_{x_i}(g)$ . The computation of the verifier is dominated by evaluating  $\ell$  polynomials of degrees  $\deg_{x_i}(g)$ , respectively, at  $|H|$  points.*

*Proof.* Perfect correctness follows by the direct inspection of the protocol: if  $\Phi(\sum_{\mathbf{x} \in H^\ell} g(\mathbf{x})) = y$  holds, then each partial sum  $q_i$  is correctly computed and all verifier checks pass. For soundness, we observe that since  $\Phi$  is  $\mathcal{C}$ -linear, the round polynomials  $q_i(x) \in M[x]$  have the same individual degree bounds as in the standard (i.e.  $\Phi = \text{id}$ ) sumcheck, so the Schwartz–Zippel argument applies directly to the  $q_i$ 's.

In every round, the verifier gets the partially evaluated sum  $q_i(x)$ , asserts that  $\sum_{x \in H} q_i(x) = y_{i-1}$ , and randomly assigns  $x_i$  with  $c_i$ . If a (partial) summation claim is false, say  $y_{i-1}^* \neq y_{i-1}$ , then

$\sum_{x \in H} q_i(x) = y_{i-1}$  only holds if the prover's message  $q_i^*(\mathbf{x})$  differs from the honestly computed  $q_i(\mathbf{x})$  of the current round. Then  $q_i(c_i) = y_i \neq q_i^*(c_i)$  except with probability at most  $\frac{\deg_{x_i}(g)}{|\mathcal{C}|}$  by the Schwartz–Zippel lemma. (To apply Schwartz–Zippel, we need the invertible difference property of  $\mathcal{C}$ , which ensures unique interpolation even if  $R$  is not a field.)

For the prover's runtime, we use the standard bookkeeping technique. Before round  $i$ , the prover maintains a table  $T_i: H^{\ell-i} \rightarrow N$  of size  $|H|^{\ell-i}$ , defined by

$$T_i(x_i, \dots, x_{\ell-1}) = g(c_0, \dots, c_{i-1}, x_i, \dots, x_{\ell-1}).$$

The initial table  $T_0$  is the evaluation of  $g$  on  $H^\ell$ , which is given by assumption. In round  $i$ , the prover must determine the univariate polynomial

$$q_i^\circ(\mathbf{x}) := \sum_{(x_{i+1}, \dots, x_{\ell-1}) \in H^{\ell-i-1}} g(c_0, \dots, c_{i-1}, \mathbf{x}, x_{i+1}, \dots, x_{\ell-1}) \in N[\mathbf{x}]$$

of degree  $d_i = \deg_{x_i}(g)$ , and then set  $q_i = \Phi \circ q_i^\circ$ . Since  $q_i^\circ$  has degree  $d_i$ , it is determined by its values at any  $d_i + 1$  points. For each  $\alpha \in H$ , the value  $q_i^\circ(\alpha)$  is obtained by summing the  $|H|^{\ell-i-1}$  table entries  $T_i(\alpha, x_{i+1}, \dots, x_{\ell-1})$ . For each of the remaining  $d_i + 1 - |H|$  evaluation points  $\alpha \notin H$ , the value  $g(c_0, \dots, c_{i-1}, \alpha, x_{i+1}, \dots, x_{\ell-1})$  is obtained for every  $(x_{i+1}, \dots, x_{\ell-1})$  by interpolating the  $|H|$  stored values in the variable  $x_i$ , at cost  $O(d_i)$  per tuple. Hence, computing  $q_i^\circ$  costs  $O(d_i \cdot |H|^{\ell-i-1})$  operations in  $N$ . After receiving  $c_i$ , the prover updates the table to  $T_{i+1}$  by evaluating the univariates at  $c_i$ , at the same asymptotic cost. Summing over all rounds with  $d = \max_i d_i$ ,

$$\sum_{i=0}^{\ell-1} O(d \cdot |H|^{\ell-i-1}) = O\left(d \cdot \sum_{j=0}^{\ell-1} |H|^j\right) = O\left(\frac{d}{|H|-1} \cdot |H|^\ell\right),$$

where the geometric series is dominated by its largest term, showing that the total cost is at most a factor  $\frac{|H|}{|H|-1}$  over the first round alone. Additionally,  $\Phi$  is applied  $\sum_i (d_i + 1)$  times in total to produce the messages  $q_i = \Phi \circ q_i^\circ$ .  $\square$

## C.2 Exploiting ring structure and NTT

By the Chinese Remainder Theorem, the splitting of the cyclotomic polynomial  $\Phi_f$  modulo  $q$  into  $\varphi/e$  irreducible factors of degree  $e$  induces a ring isomorphism  $\text{NTT}: \mathcal{R}_q \rightarrow \mathbb{F}_{q^e}^{\varphi/e}$ .

Let  $M = \mathcal{R}_q$  and let  $\mathcal{R} \cong \mathbb{F}_{q^e}^{\varphi/e}$  be the NTT decomposition. Write  $\mathbb{F} = \mathbb{F}_{q^e}$  as a shorthand. Observe that all operations in sumcheck are algebraic. Hence, by applying  $\text{NTT}: \mathcal{R} \rightarrow \mathbb{F}^{\varphi/e}$ , we can run sumcheck over  $\mathbb{F}^{\varphi/e}$  (with component-wise operations). If we need the resulting claim over  $\mathcal{R}$ , it can be recovered via  $\text{NTT}^{-1}$ .

Switching to the NTT domain increases our implementation efficiency. In fact, we can further optimise the sumcheck by running it over  $\mathbb{F}$  instead of  $\mathbb{F}^{\varphi/e}$ . For this, we assume that  $\mathcal{C} \subseteq \mathbb{F}$ . Then, we can view  $M = \mathcal{R} = \mathbb{F}^{\varphi/e}$  as a  $\mathbb{F}$ -vector space of dimension  $\varphi/e$ . By choosing a suitable random  $\mathbb{F}$ -linear map  $\Phi: \mathbb{F}^{\varphi/e} \rightarrow \mathbb{F}$ , we can replace the sumcheck claim

$$\sum_{\mathbf{x} \in H^n} g(\mathbf{x}) = y \in \mathbb{F}^{\varphi/e}$$

with the batched version

$$\Phi\left(\sum_{\mathbf{x} \in H^n} g(\mathbf{x})\right) = \Phi(y)$$

This is a reduction of knowledge with soundness error depending on the choice of  $\Phi$ .

- For  $\Phi$  defined as  $\Phi : \delta^T \circ \text{NTT}$  for uniformly random  $\delta \leftarrow \mathbb{F}^{\varphi/e}$ , the knowledge error is  $\frac{1}{|\mathbb{F}|} = q^{-e}$ .
- For random tensor product, i.e.  $\Phi_i = \text{eq}(\text{bin}(i), \mathbf{r})$  where  $\mathbf{r} \leftarrow \mathbb{F}^{\lceil \log(\varphi/e) \rceil}$ , the knowledge error is  $\frac{\lceil \log(\varphi/e) \rceil}{|\mathbb{F}|} = q^{-e} \cdot \lceil \log(\varphi/e) \rceil$ .

Both claims are consequences of the Schwartz–Zippel lemma.

The sumcheck reduction with random  $\Phi$  reduces the summation claim (over  $\mathcal{R}_q \cong \mathbb{F}_{q^e}^{\varphi/e}$ ) to an evaluation claim  $\Phi(g(\mathbf{c})) = v$ , where  $v \in \mathbb{F}$  and  $\mathbf{c} \in \mathbb{F}$  are the verifier's random challenges. As our expressions are of the form  $g(\mathbf{c}) = f(\text{MLE}[\mathbf{w}], \text{MLE}[\overline{\mathbf{w}}])(\mathbf{c})$  for  $f \in (R[\mathbf{z}])[\mathbf{x}_0, \mathbf{x}_1]$ , we find:

$$\Phi(f(\text{MLE}[\mathbf{w}], \text{MLE}[\overline{\mathbf{w}}]))(\mathbf{c}) = \Phi(f(\text{MLE}[\mathbf{w}], \text{MLE}[\overline{\mathbf{w}}])(\mathbf{c})) = \Phi(g(\mathbf{c})) = v.$$

In particular, for any  $\mathbf{c} \in \mathbb{F}_{q^e}^{\text{len}(\mathbf{z})} \subseteq \mathcal{R}_q^{\text{len}(\mathbf{z})}$ , this expression can be verified as  $\Phi(f(z_0, z_1)(\mathbf{c}))$  given  $z_0 = \text{MLE}[\mathbf{w}](\mathbf{c})$  and  $z_1 = \text{MLE}[\overline{\mathbf{w}}](\mathbf{c})$ , where the claimed equalities for  $z_0, z_1$  may be delegated to the prover again, while exploiting the identity

$$z_1 = \text{MLE}[\overline{\mathbf{w}}](\mathbf{c}) \iff \overline{z_1} = \text{MLE}[\mathbf{w}](\overline{\mathbf{c}}).$$